Alma Mater Studiorum · Università di Bologna

Scuola di Ingegneria
Corso di Laurea in Automation Engineering



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Laboratory of Automation Systems

Implementation and Control of a DC Motor

Academic Year 2025/2026

Professors:
Prof. Claudio Melchiorri
Prof. Davide Samorì

Students:
Andrea Gallo
(Mat. 0001117356)
Vittorio Ragazzini
(Mat. 0001116031)
Leonardo Lodolini
(Mat. 0001116950)

June 15, 2026

Contents

Introduction **3**
0.1 Objective of the Report . . . 3
0.2 Structure of the Report . . . 4

1 Experimental Setup **5**

2 DC Motor Model **7**
2.1 Motor Model . . . 7
2.2 Motor parameters from datasheet . . . 8
2.3 Speed measurement filtering . . . 8

3 Cascade Control of the DC Motor **12**
3.1 Cascade Control for a DC Motor . . . 12
3.1.1 Cascade control of our electric motor . . . 13
3.2 PI controller for Current Loop . . . 14
3.2.1 Discretization . . . 15
3.2.2 Implementation . . . 16
3.2.3 Current loop anti-windup implementation . . . 19
3.3 PI controller for Speed Tracking . . . 21
3.3.1 Discretization and Implementation . . . 23
3.3.2 Speed loop anti-windup implementation . . . 25
3.3.3 Final PI control scheme . . . 26
3.3.4 Trajectory tracking and feedforward action . . . 29

4 Parameter Estimation Using Least Squares **32**
4.1 Motivation for Parameter Estimation . . . 32
4.2 Least Squares Method . . . 32
4.3 Discrete-Time Formulation of the Problem . . . 34
4.4 Estimation of Electrical Parameters . . . 35
4.4.1 Estimation of the Armature Resistance . . . 36
4.4.2 Estimation of the Armature Inductance . . . 36
4.5 Estimation of Mechanical Parameters . . . 37
4.5.1 Estimation of the Equivalent Inertia . . . 37
4.5.2 Estimation of the Viscous Friction Coefficient . . . 38
4.5.3 Interpolation of the Values of b . . . 38
4.5.4 Simulink transposition of the interpolated value of b . . . 43
4.6 Validation of the Estimated Parameters . . . 45
4.6.1 Electrical Subsystem Validation . . . 45
4.6.2 Mechanical Subsystem Validation . . . 45

4.7 PI Controller Designed with Estimated Parameters 46
4.7.1 Inner Current Loop Design 46
4.7.2 Outer Speed Loop Design 47
4.7.3 Performance Evaluation with Fixed Estimated Friction 47
4.7.4 Performance Evaluation with Interpolated Friction 50
4.8 Comparison Between Nominal and Estimated Parameters 53
4.9 Improvements Obtained on the Real System 53

5 Advanced Discrete-Time Controllers 55
5.1 Introduction to Discrete-Time Control 55
5.2 Deadbeat Controller 55
5.2.1 Theoretical Principle of Deadbeat Control 55
5.2.2 Plant Discretization and Controller Design 56
5.2.3 Anti-Windup Implementation 57
5.2.4 Simulation Results: Control Effort and Ringing Effect 58
5.2.5 Experimental Results on the DC Motor 60
5.3 Dahlin Controller 63
5.3.1 Theoretical Principle of Dahlin Control 63
5.3.2 Plant Discretization, Controller Design and Tuning 63
5.3.3 Anti-Windup Implementation and Simulation Results 64
5.3.4 Experimental Results on the DC Motor 66

6 Final Comparison Between Controllers 70
6.1 Comparison Criteria 70
6.2 Performance in Unloaded Condition 71
6.3 Performance in Loaded Condition 74
6.4 Final Discussion 76

7 Conclusions 77

Introduction

0.1 Objective of the Report

The primary objective of this report is to document the complete workflow concerning the design, implementation, and experimental validation of various digital control strategies applied to a direct current (DC) motor. The project aims to bridge the gap between continuous-time theoretical modeling and real-world discrete-time implementation, utilizing a LAUNCHXL-F28379D microcontroller interfaced with MATLAB and Simulink.

In a theoretical framework, control algorithms are often designed assuming ideal linear systems. However, real-world actuations introduce several physical constraints and non-idealities, such as sensor quantization noise, computational delays, and strict actuator saturation limits. Therefore, a major focus of this work is not only the synthesis of the controllers but also the management of these physical limitations. This is achieved through the implementation of digital low-pass filters for sensor data and the design of robust anti-windup mechanisms to prevent system instability during voltage or current saturation.

Specifically, the report pursues the following intermediate goals:

- **System Modeling and Filtering:** Defining an accurate electro-mechanical model of the DC motor and properly conditioning the feedback signals (e.g., encoder measurements).
- **Cascade PI Control:** Designing a classic multi-loop architecture featuring an inner current loop and an outer speed loop, enhanced with feedforward actions to improve tracking performance.
- **System Identification:** Applying offline Least Squares (LS) estimation to accurately identify the real electro-mechanical parameters of the motor, thereby improving the analytical tuning of the controllers.
- **Advanced Discrete-Time Control:** Synthesizing and deploying advanced analytical digital controllers, namely the Deadbeat and Dahlin algorithms, directly in the discrete-time domain.
- **Performance Comparison:** Comparing the dynamic behavior, control effort, and robustness of the implemented strategies under different operating conditions (e.g., loaded and unloaded scenarios).

Ultimately, this document serves as a comprehensive guide demonstrating how to progressively upgrade a motor control system from a basic uncompensated setup to an advanced, fully robust digital control architecture.

0.2 Structure of the Report

To provide a clear and logical progression from system modeling to the implementation of advanced control algorithms, this report is organized into seven main chapters:

- **Chapter 1: Experimental Setup.** Introduces the hardware components utilized in the laboratory, detailing the LAUNCHXL-F28379D microcontroller board, the PWM signal generation, and the speed sensing mechanics.
- **Chapter 2: DC Motor Model.** Presents the theoretical mathematical model of the DC motor and the nominal parameters extracted from the datasheet. It also details the design and discretization of the low-pass filter required to properly handle the speed measurements.
- **Chapter 3: Cascade Control of the DC Motor.** Details the design, discretization, and implementation of the classical multi-loop PI control scheme. It covers the synthesis of both the inner current loop and the outer speed loop, integrating anti-windup mechanisms and feedforward compensation for trajectory tracking.
- **Chapter 4: Parameter Estimation Using Least Squares.** Explains the offline estimation of the motor's electrical (R_a, L_a) and mechanical (J, b) parameters. It demonstrates how updating the mathematical model with these real values significantly improves the overall control performance.
- **Chapter 5: Advanced Discrete-Time Controllers.** Explores direct discrete-time synthesis through the implementation of Deadbeat and Dahlin controllers. A strong emphasis is placed on managing actuator saturation and high-frequency ringing effects through the design of dedicated polynomial anti-windup schemes.
- **Chapter 6: Final Comparison Between Controllers.** Provides a comprehensive comparative analysis of the implemented control architectures (Cascade PI, Deadbeat, and Dahlin), evaluating their transient behavior, steady-state accuracy, and control effort.
- **Chapter 7: Conclusions.** Summarizes the main results achieved during the project, discusses the practical advantages and limitations of each approach, and suggests possible future improvements.

Chapter 1

Experimental Setup

The experimental setup used in this work is based on a DC motor test bench controlled through a microcontroller board. The system is composed of two mechanically coupled motors, an encoder for speed and position measurement, a PWM power stage for the motor supply, and a control board used to execute the control algorithms in real time.

One of the motors is used as the controlled DC motor, while the second motor is mechanically connected to the same shaft and can be used to emulate an external load. In this way, it is possible to test the behavior of the controller both in no-load conditions and in the presence of an additional resistant torque.

The control algorithm is implemented on the LAUNCHXL-F28379D board. This board executes the discrete-time control routines and generates the PWM signals required to drive the motor. Since the control is implemented digitally, all the controllers designed in continuous time must be discretized before being uploaded to the microcontroller.

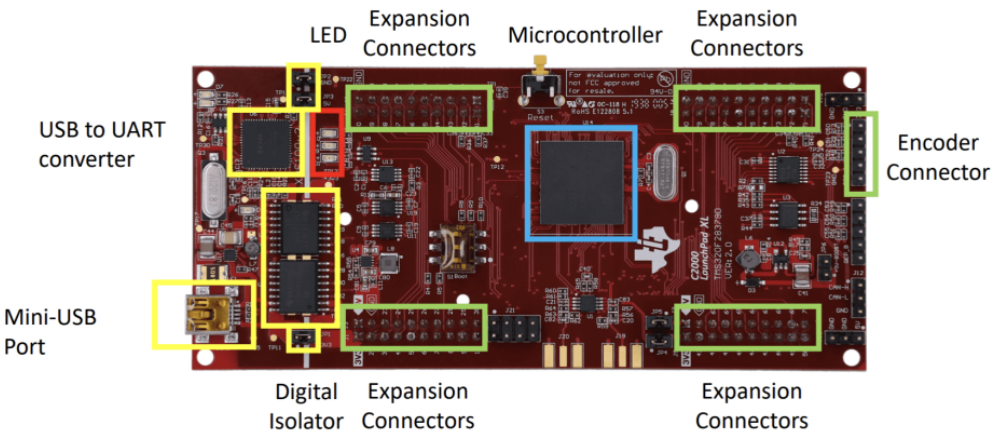


Figure 1.1: LAUNCHXL-F28379D control board

The motor voltage is regulated through PWM modulation. The controller computes the required voltage reference, and the PWM block converts this value into a duty cycle. By changing the duty cycle, the average voltage applied to the DC motor is adjusted, allowing the current and speed to be controlled.

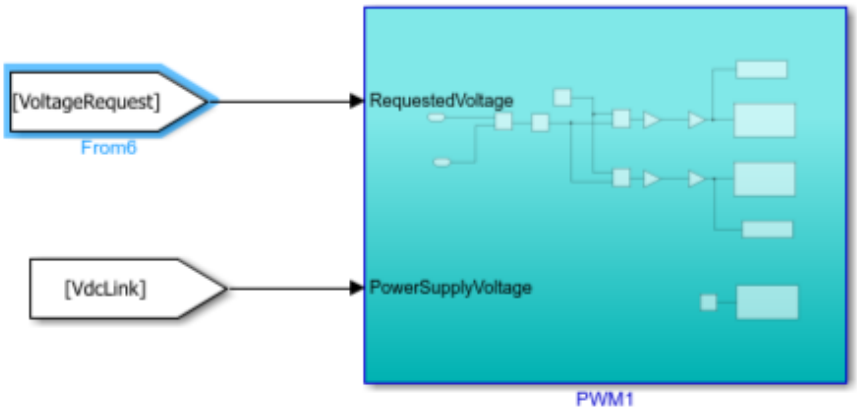


Figure 1.2: PWM block used to generate the motor driving signal

The motor speed is measured using an incremental encoder connected to the motor shaft. The encoder has a resolution of 2048 counts per revolution. Since quadrature decoding is used, the effective number of counts per revolution becomes:

$$2048 \cdot 4 = 8192 \quad \text{counts/revolution} \tag{1.1}$$

The encoder signal is sampled every $100\,\mu s$, corresponding to a sampling time of:

$$T_s = 100\,\mu s = 1 \cdot 10^{-4}\,s \tag{1.2}$$

At a speed of $1500\,rpm$, the expected number of encoder counts per second is:

$$\frac{8192 \cdot 1500}{60} = 204800 \quad \text{counts/s} \tag{1.3}$$

Therefore, during one sampling period, the expected number of counts is approximately:

$$204800 \cdot 1 \cdot 10^{-4} \simeq 20 \quad \text{counts/sample} \tag{1.4}$$

This shows that even an error of one encoder count can produce a significant variation in the estimated speed. For example, at this operating point, one count corresponds to about 5% of the measured value, which is approximately $75\,rpm$. For this reason, the raw speed measurement is affected by quantization noise and must be filtered before being used as feedback for the speed controller.

Chapter 2

DC Motor Model

2.1 Motor Model

The first step for the design of the control system is the definition of a mathematical model of the DC motor. The model describes the relationship between the armature voltage, the armature current, the generated torque and the motor speed. In a DC motor, the electrical and mechanical dynamics are coupled through the motor constant.

The electrical equation of the armature circuit is:

$$v_a(t) = R_a i_a(t) + L_a \frac{di_a(t)}{dt} + e_m(t) \tag{2.1}$$

where $v_a(t)$ is the armature voltage, $i_a(t)$ is the armature current, R_a and L_a are the armature resistance and inductance, and $e_m(t)$ is the back electromotive force. The back-EMF is proportional to the motor speed:

$$e_m(t) = k_m \omega_m(t) \tag{2.2}$$

The electromagnetic torque generated by the motor is proportional to the armature current:

$$C_m(t) = k_m i_a(t) \tag{2.3}$$

The mechanical dynamics are described by the torque balance on the rotor:

$$J \frac{d\omega_m(t)}{dt} = C_m(t) - C_r(t) - b\omega_m(t) \tag{2.4}$$

where J is the total inertia, b is the viscous friction coefficient and $C_r(t)$ is the load torque. In this model, the same motor constant k_m is used for both the torque constant and the back-EMF constant, considering SI units.

By applying the Laplace transform and neglecting the load torque for the transfer function derivation, the motor model can be written as:

$$V_a(s) = R_a I_a(s) + sL_a I_a(s) + k_m \Omega_m(s) \tag{2.5}$$

$$sJ\Omega_m(s) = k_m I_a(s) - b\Omega_m(s) \tag{2.6}$$

From these equations, the transfer function between the armature voltage and the motor speed is obtained:

$$G_\omega(s) = \frac{\Omega_m(s)}{V_a(s)} = \frac{k_m}{(R_a + sL_a)(sJ + b) + k_m^2} \tag{2.7}$$

This model highlights the interaction between the electrical part of the motor and the mechanical part. The armature voltage determines the current, the current generates torque, and the torque produces the motor speed. At the same time, the motor speed generates a back-EMF that opposes the applied voltage. This complete model was then implemented in Simulink and used as the basis for the design of the cascade current and speed controllers.

2.2 Motor parameters from datasheet

The parameters used for the motor model and for the controller design were obtained from the datasheet of the PMB 1524 DC motor. The main nominal data are the rated voltage, rated current and rated speed:

$$V_n = 24\,V, \qquad I_n = 1.9\,A, \qquad N_n = 1500\,rpm \tag{2.8}$$

For the electrical model, the terminal resistance and the armature inductance were used:

$$R_m = 3.20\,\Omega, \qquad L_a = 4.1\,mH \tag{2.9}$$

The torque constant and the main time constants provided by the datasheet are:

$$K_t = 0.11\,Nm/A, \qquad T_e = 1.3\,ms, \qquad T_m = 85\,ms \tag{2.10}$$

The datasheet gives the rotor inertia of one motor as:

$$J = 0.352\,kg\,cm^2 \tag{2.11}$$

Converting this value into SI units:

$$J = 0.352 \cdot 10^{-4} = 3.52 \cdot 10^{-5}\,kg\,m^2 \tag{2.12}$$

In the experimental setup, two motors are mechanically connected. Therefore, the equivalent inertia considered in the model was obtained by doubling the inertia of a single motor:

$$J_{eq} = 2J = 2 \cdot 3.52 \cdot 10^{-5} = 7.04 \cdot 10^{-5}\,kg\,m^2 \tag{2.13}$$

This equivalent inertia was then used in the design of the speed controller, since the controller has to take into account the total mechanical inertia of the coupled motor system.

2.3 Speed measurement filtering

The speed measured by the encoder is affected by quantization effects and high-frequency noise. For this reason, a first-order low-pass filter was applied before using the measured speed as feedback for the speed PI controller:

$$G_f(s) = \frac{1}{\tau_f s + 1} \tag{2.14}$$

The filter time constant was chosen as $\tau_f = 0.002\,s$, as a compromise between noise reduction and delay introduced in the feedback loop. A larger value of τ_f would produce a smoother signal, but also a higher delay, which could worsen the transient response of the speed loop.

The filter was discretized using the backward Euler approximation:

$$s \simeq \frac{1 - z^{-1}}{T_s} \tag{2.15}$$

By applying this approximation, the recursive equation becomes:

$$\omega_f[k] = \frac{T_s}{T_s + \tau_f}\omega_m[k] + \frac{\tau_f}{T_s + \tau_f}\omega_f[k-1] \tag{2.16}$$

where $\omega_m[k]$ is the measured speed and $\omega_f[k]$ is the filtered speed. The corresponding Matlab implementation is:

```
function out = LowPassFilter(Ts, Tao, In)

persistent OutPrecedente

if isempty(OutPrecedente)
    OutPrecedente = 0;
end

% Discretized low-pass filter using backward Euler method
a = Ts/(Ts + Tao);
b = Tao/(Ts + Tao);

out = a*In + b*OutPrecedente;

OutPrecedente = out;

end
```

Listing 2.1: Low-pass filter implementation

In the code, `Tao` corresponds to the filter time constant τ_f, while `OutPrecedente` stores the previous filtered value.

Figure 2.1 shows the measured and filtered speed during a speed transient. The filtered signal follows the measured speed without introducing a significant delay, while reducing the high-frequency oscillations due to encoder quantization.

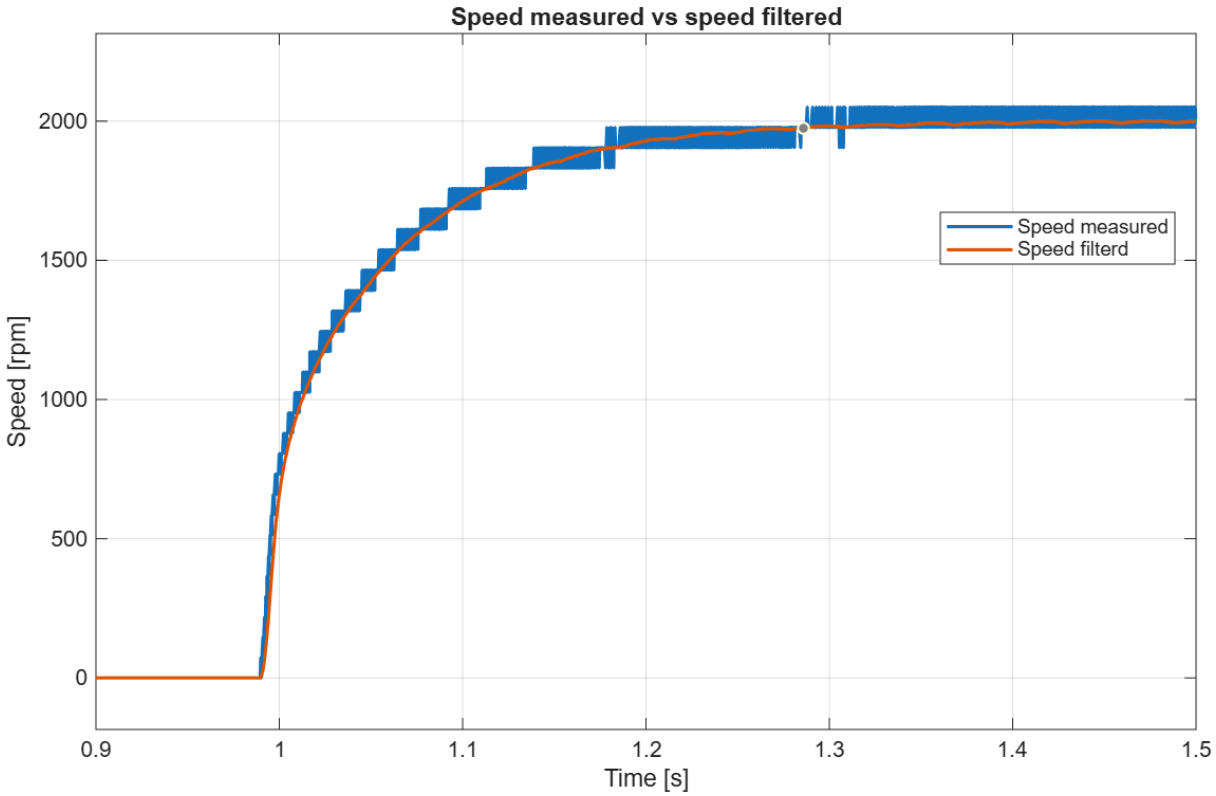


Figure 2.1: Measured and filtered speed during a speed transient

The zoomed view in Figure 2.2 highlights the filtering effect around steady state. The measured speed presents fast oscillations, whereas the filtered speed is smoother and remains close to the average measured value. Therefore, the chosen filter reduces measurement noise while keeping the feedback delay limited.

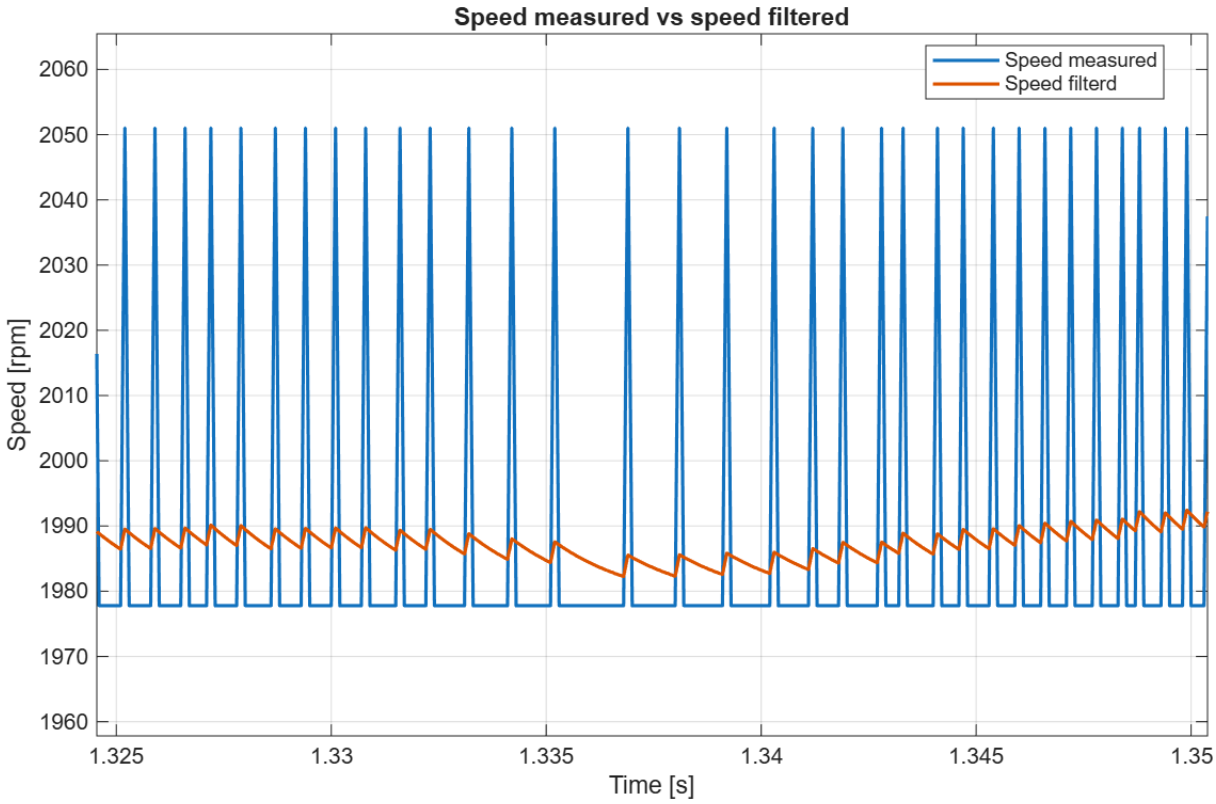


Figure 2.2: Zoomed view of the measured and filtered speed at steady state

Chapter 3

Cascade Control of the DC Motor

3.1 Cascade Control for a DC Motor

The first control strategy implemented for the DC motor is based on a cascade control architecture. Cascade control is particularly useful when the dynamics of the plant can be divided into two main parts, characterized by different time constants. In this case, the electrical dynamics of the motor are much faster than the mechanical dynamics; therefore, it is possible to control them separately by using two nested control loops.

In a general cascade control scheme, the inner loop is designed to regulate the fastest dynamic of the system, while the outer loop controls the slower variable. This structure allows the controller to improve the overall performance of the system, since the inner loop can quickly reject disturbances and compensate for variations before they affect the outer loop.

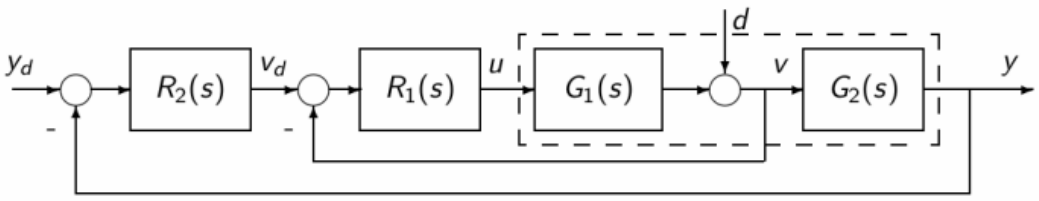


Figure 3.1: Classical cascade control scheme

The main advantages of this control architecture are the local compensation of disturbances acting on the inner loop, a higher robustness with respect to parameter variations, and the possibility of applying saturation and anti-windup strategies separately on each loop.

Considering a generic cascade structure composed of two plant dynamics $G_1(s)$ and $G_2(s)$, and two controllers $R_1(s)$ and $R_2(s)$, the overall transfer function can be written as:

$$G_{t1}(s) = \frac{R_1(s)R_2(s)G_1(s)G_2(s)}{1 + R_1(s)G_1(s) + R_1(s)R_2(s)G_1(s)G_2(s)} \tag{3.1}$$

This expression can be rearranged as:

$$G_{t1}(s) = \frac{1}{1 + \frac{1}{R_2(s)G_2(s)}\left(1 + \frac{1}{R_1(s)G_1(s)}\right)} \tag{3.2}$$

If the inner loop is sufficiently fast, it is possible to assume that:

$$R_1(s)G_1(s) \gg 1 \tag{3.3}$$

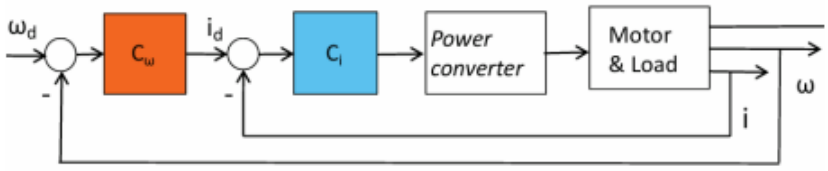
in the frequency range of interest for the outer loop. Under this assumption, the overall transfer function mainly depends on the external control loop:

$$G_{t1}(s) \simeq \frac{1}{1 + \frac{1}{R_2(s)G_2(s)}} \tag{3.4}$$

This property will be exploited in the following sections for the design of the current and speed controllers of the DC motor.

3.1.1 Cascade control of our electric motor

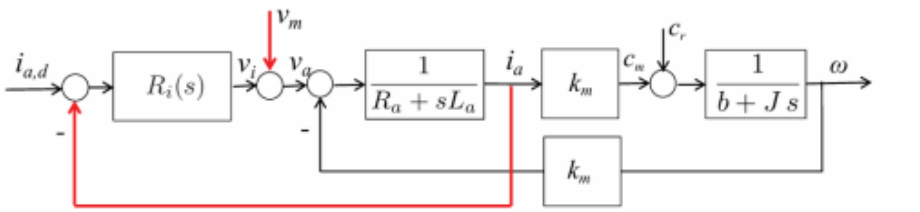
As we can clearly see in the previous section, thanks to the separate electrical and mechanical dynamics, the control of an electric motor can be naturally implemented with a cascade structure composed of a current control loop and a velocity control loop.



The inner control loop is the current control which is equivalent to a direct torque control because of the relationship $C_m(t) = k_m i(t)$, although there is not a direct measurement of the torque. It is also very useful in preventing overcurrents both in the converter and in the motor, especially at motor start-up. The second outer control loop is the velocity control, when considering the design of its controller we will exploit the consideration made earlier that the inner current loop can be considered as an ideal torque actuator. In general, in order to consider the dynamics of the inner loop negligible with respect to the outer loop, the current loop must be significantly faster than the speed loop. In this implementation, the current-loop time constant was chosen as $\tau_i = 1\,ms$, while the speed-loop time constant was chosen as $\tau_\omega = 10\tau_i = 10\,ms$. Therefore, the speed loop is about ten times slower than the current loop, which preserves the cascade control structure.

3.2 PI controller for Current Loop

The classical block scheme for the current control loop is:



The red feedforward term $v_m(t) = k_m\omega(t)$ is very important, it acts as back-EMF compensation. Remember that the back-EMF effect is often the dominant term in the armature voltage. This feedforward action requires the knowledge of the motor velocity ω, and it is important as it allows to fully apply the control on the electrical armature, and also to avoid overcurrents in case of small errors in the current loop. Indeed, if we do not compensate the back-EMF, we are making the electrical load active, and much more responsive to errors in the current loop. Now we choose to have a null steady state error with a current controller of the PI type:

$$R_i(s) = K_{pi}\left(1 + \frac{1}{T_{ii}s}\right) = \frac{K_{pi}}{T_{ii}} \cdot \frac{1 + T_{ii}s}{s} \tag{3.5}$$

Therefore, if

$$G_e(s) = \frac{1}{R_a + sL_a} = \frac{1/R_a}{1 + s\tau_a}, \qquad \tau_a = \frac{L_a}{R_a} \tag{3.6}$$

the open loop transfer function results:

$$G_i(s) = R_i(s)G_e(s) = \frac{K_{pi}}{T_{ii}} \cdot \frac{1 + T_{ii}s}{s} \cdot \frac{1/R_a}{1 + s\tau_a} \tag{3.7}$$

Let's choose $T_{ii} = \tau_a$ in order to cancel the electric pole:

$$G_i(s) = R_i(s)G_e(s) = \frac{K_{pi}}{R_a T_{ii} s} \tag{3.8}$$

We see that the system is now TYPE 1, so the steady state error to a step reference is null. The closed loop function is:

$$F_i(s) = \frac{i_a(s)}{i_{ad}(s)} = \frac{G_i(s)}{1 + G_i(s)} = \frac{1}{1 + \frac{1}{G_i(s)}} = \frac{1}{1 + \frac{R_a T_{ii} s}{K_{pi}}} = \frac{1}{1 + \tau_i s} \tag{3.9}$$

By assigning a desired time constant τ_i for the current loop, it results:

$$\frac{R_a T_{ii}}{K_{pi}} = \tau_i \qquad \Rightarrow \qquad K_{pi} = \frac{L_a}{\tau_i} \tag{3.10}$$

Therefore, the two parameters of the PI current controller are:

$$T_{ii} = \tau_a = \frac{L_a}{R_a}, \qquad K_{pi} = \frac{L_a}{\tau_i} \tag{3.11}$$

3.2.1 Discretization

After the design of the continuous-time PI controller, it was necessary to discretize it in order to implement it on the microcontroller. Since the control algorithm is executed in discrete time with the same period of the PWM, the sampling time used in the implementation is:

$$T_s = T_{PWM} = \frac{1}{f_{PWM}} = \frac{1}{10^4} = 1 \cdot 10^{-4}\, s \tag{3.12}$$

Starting from the continuous-time PI controller:

$$R(s) = K_{pi}\left(1 + \frac{1}{T_{ii}s}\right) \tag{3.13}$$

the Tustin substitution, also called bilinear transform, is:

$$s = \frac{2}{T_s} \cdot \frac{1 - z^{-1}}{1 + z^{-1}} \tag{3.14}$$

By substituting this expression into the continuous-time controller, we obtain:

$$R(z) = K_{pi}\left(1 + \frac{T_s}{2T_{ii}} \cdot \frac{1 + z^{-1}}{1 - z^{-1}}\right) \tag{3.15}$$

Bringing everything under a common denominator:

$$R(z) = K_{pi}\left(\frac{(1 - z^{-1}) + \frac{T_s}{2T_{ii}}(1 + z^{-1})}{1 - z^{-1}}\right) \tag{3.16}$$

which can be rewritten as:

$$R(z) = K_{pi}\left(\frac{1 + \frac{T_s}{2T_{ii}} + \left(\frac{T_s}{2T_{ii}} - 1\right) z^{-1}}{1 - z^{-1}}\right) \tag{3.17}$$

This discrete transfer function has the following form:

$$R(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1}}{1 - z^{-1}} \tag{3.18}$$

where:

$$b_0 = K_{pi}\left(1 + \frac{T_s}{2T_{ii}}\right), \qquad b_1 = K_{pi}\left(\frac{T_s}{2T_{ii}} - 1\right) \tag{3.19}$$

The motor parameters used in the Matlab script are:

$$L_a = 0.0041\, H, \qquad R_a = 3.20\, \Omega \tag{3.20}$$

The integral time of the current PI controller was chosen equal to the electrical time constant, in order to cancel the electrical pole:

$$T_{ii} = \tau_a = \frac{L_a}{R_a} = \frac{0.0041}{3.20} = 1.28 \cdot 10^{-3}\, s \tag{3.21}$$

The desired closed-loop time constant of the current loop was chosen as:

$$\tau_i = 1 \cdot 10^{-3}\, s \tag{3.22}$$

Therefore, the proportional gain is:

$$K_{pi} = \frac{L_a}{\tau_i} = \frac{0.0041}{1 \cdot 10^{-3}} = 4.1 \tag{3.23}$$

Using these numerical values:

$$T_s = 1 \cdot 10^{-4}\, s, \qquad T_{ii} = 1.28 \cdot 10^{-3}\, s, \qquad K_{pi} = 4.1 \tag{3.24}$$

the discrete coefficients become:

$$b_0 = 4.1 \left(1 + \frac{1 \cdot 10^{-4}}{2 \cdot 1.28 \cdot 10^{-3}}\right) \simeq 4.26 \tag{3.25}$$

$$b_1 = 4.1 \left(\frac{1 \cdot 10^{-4}}{2 \cdot 1.28 \cdot 10^{-3}} - 1\right) \simeq -3.94 \tag{3.26}$$

The controller can be written as the following recursive equation:

$$Y[k] = b_0 X[k] + b_1 X[k-1] + Y[k-1] \tag{3.27}$$

For the software implementation, the same controller was written in incremental form:

$$Y[k] = Y[k-1] + K_{pi}\left(X[k] - X[k-1]\right) + \frac{K_{pi} T_s}{2T_{ii}}\left(X[k] + X[k-1]\right) \tag{3.28}$$

This final expression corresponds to the equation implemented inside the Matlab function block.

3.2.2 Implementation

The discrete PI controller was implemented in Matlab using a function with persistent variables. These variables are used to store the previous input and output values of the controller between two consecutive sampling instants.

In the implemented function, the variable `in` represents the controller input, which is the current error, while `out` represents the controller output, namely the requested voltage. The variables `in_prev` and `out_prev` store the values of the previous sampling step.

The implemented equation is:

$$\text{out}(k) = \frac{K_p T_s}{2T_i}\left(\text{in}(k) + \text{in}(k-1)\right) + K_p\left(\text{in}(k) - \text{in}(k-1)\right) + \text{out}(k-1) \tag{3.29}$$

which is exactly the incremental form obtained from the Tustin discretization. In the Matlab code, this is written as:

```
out = (Kp*Ts/(2*Ti))*(in + in_prev) + ...
      Kp*(in - in_prev) + out_prev;
```

Listing 3.1: Discrete PI implementation

The implementation also includes an enable signal. If the controller is enabled, the PI output is computed and the internal states are updated. If the controller is disabled, the output is set to zero and the previous states are reset:

```
if (Enable == 1)
    out = (Kp*Ts/(2*Ti))*(in + in_prev) + ...
          Kp*(in - in_prev) + out_prev;

    out_prev = out;
    in_prev = in;
else
    out = 0;
    out_prev = 0;
    in_prev = 0;
end
```

Listing 3.2: PI enable logic

With this implementation, the current loop is discretized consistently with the controller design and is suitable for real-time execution on the microcontroller with sampling time $T_s = 100\,\mu s$.

The Simulink implementation of the current loop is shown in Figure 3.2. The current reference is compared with the measured current and the resulting error is given as input to the discrete PI block. The controller receives the parameters previously computed, namely K_{pi}, T_{ii} and the sampling time T_s. In addition, the enable signal and the saturation limits are provided to the block, so that the controller can be activated only when required and the requested voltage can be limited within the admissible range.

The output of the PI block represents the voltage contribution required to regulate the armature current. After this, the back-EMF compensation term $k_m\omega$ is added, using the measured motor speed. In this way, the final voltage request sent to the PWM includes both the feedback action of the current controller and the compensation of the motor back electromotive force.

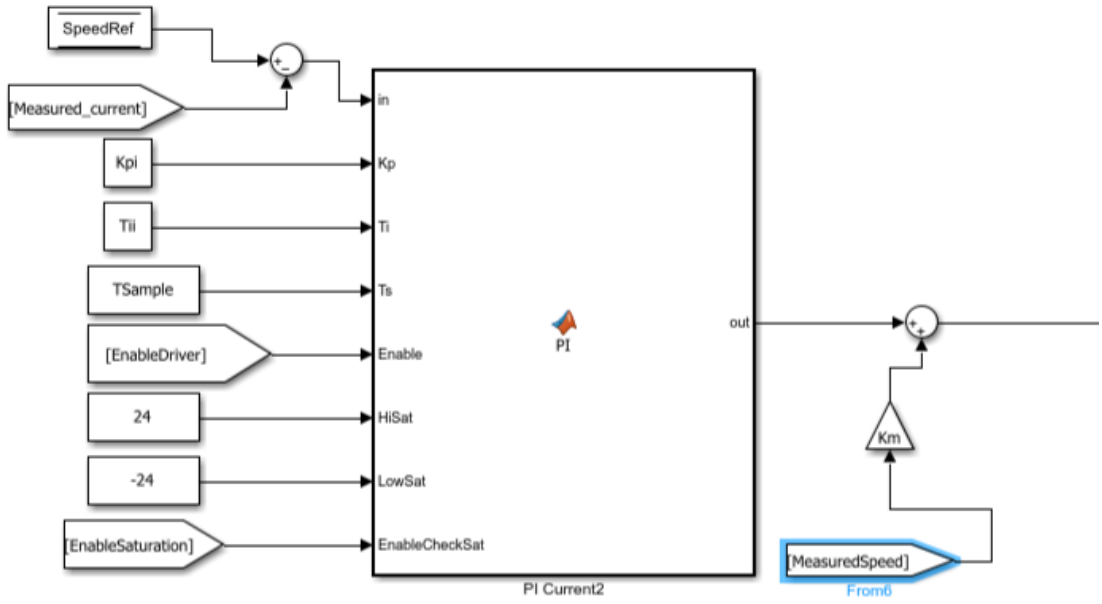


Figure 3.2: Current PI implementation in Simulink

After the implementation of the current PI controller, the inner loop was tested by applying a step reference of $1\,A$. The aim of this test was to verify that the measured

current was able to track the desired reference and that the requested voltage remained within the admissible range.

From Figure 3.3, it can be observed that the measured current follows the reference with a very fast transient. When the current reference changes from $0\,A$ to $1\,A$, the measured current reaches the desired value in a short time, with only a small overshoot. After the transient, the current remains close to the reference, with some oscillations mainly due to measurement noise and to the switching behavior of the PWM converter.

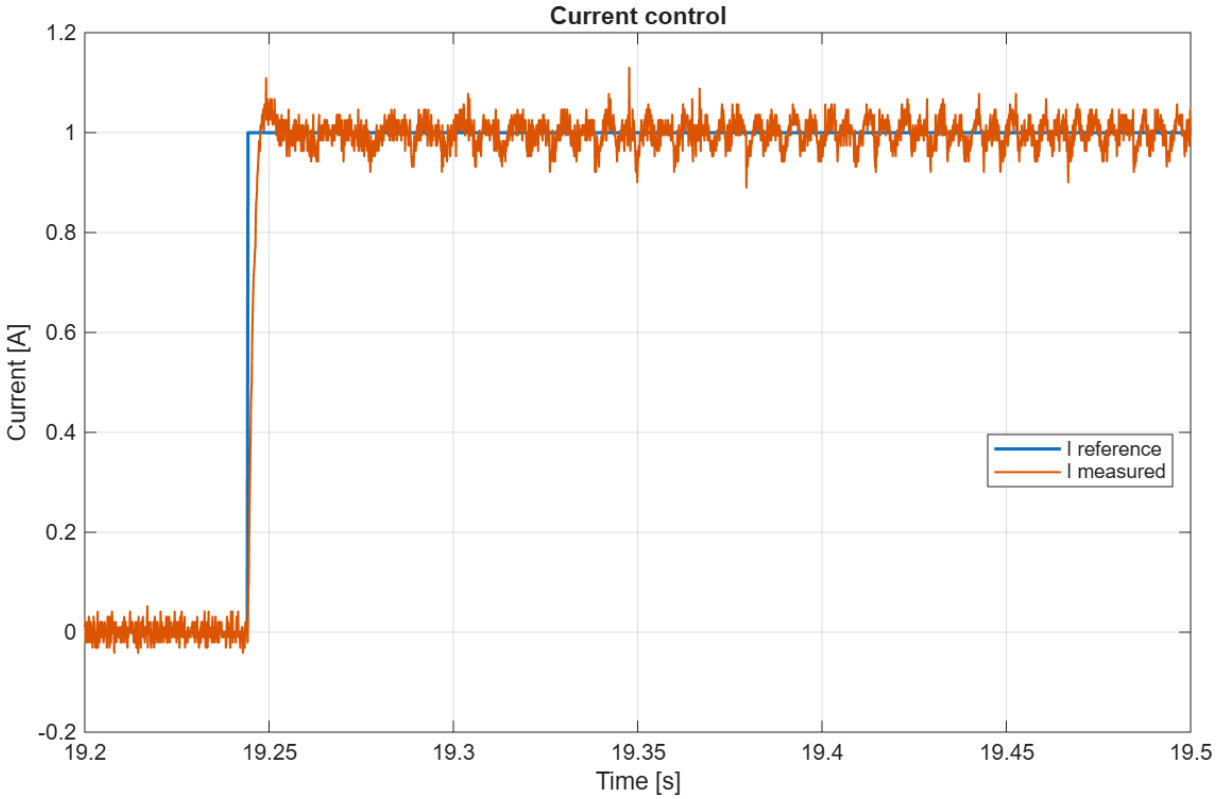


Figure 3.3: Current response of the inner loop

The requested voltage shown in Figure 3.4 confirms the behavior of the current loop. At the beginning of the step, the controller requires a fast increase of the voltage in order to rapidly bring the current to the desired value. After the first transient, the requested voltage settles around a nearly constant value of about 4.5–$5\,V$, with small oscillations caused by the regulation action and by the current measurement noise.

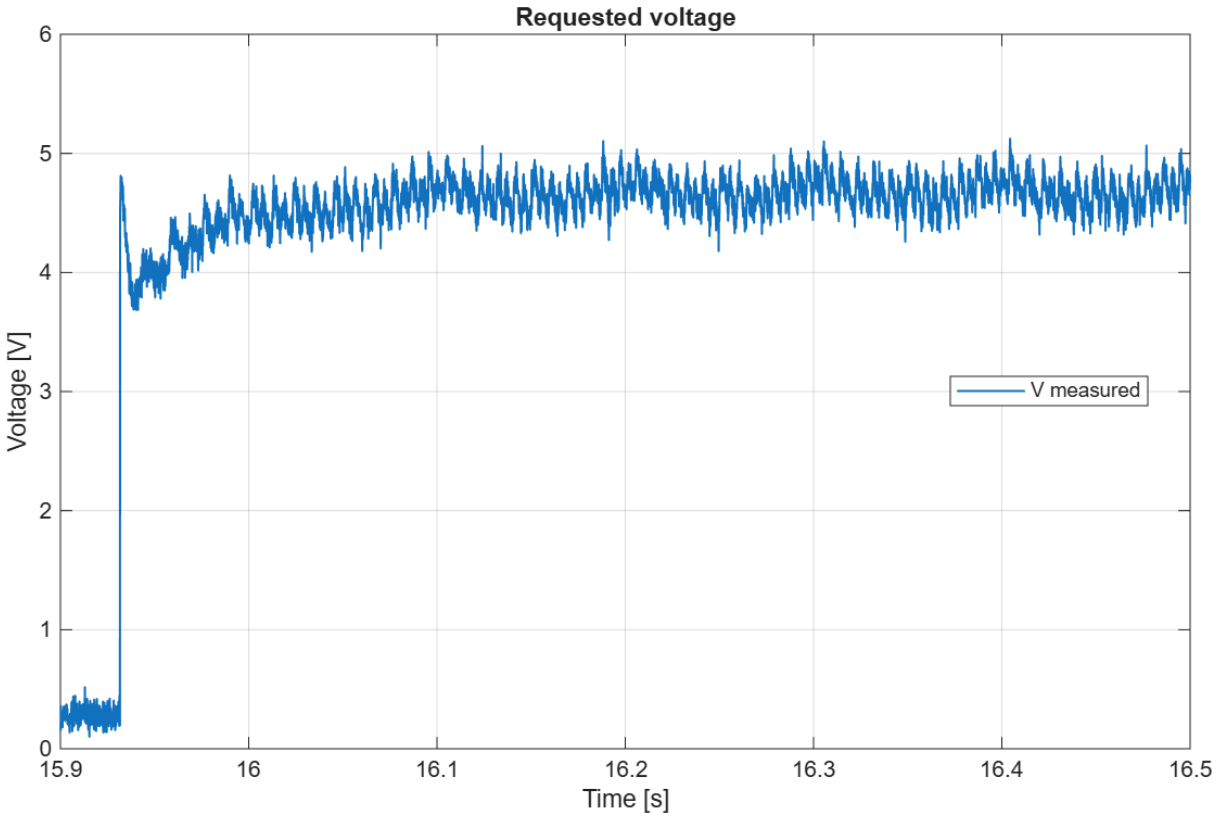


Figure 3.4: Requested voltage during current control

In this test, the voltage does not reach the saturation limit. Therefore, the results show the nominal behavior of the current loop: the response is fast, stable and characterized by a small steady-state error. However, in more demanding operating conditions, for example with larger current references or higher load torque, the requested voltage may reach the physical limits of the actuator. For this reason, an anti-windup strategy was also implemented, as described in the following section.

3.2.3 Current loop anti-windup implementation

During the current control implementation, particular attention was given to the saturation of the requested voltage. Since the motor is driven through a PWM stage, the voltage that can be applied to the armature is physically limited by the available DC-link voltage. Therefore, when the controller requests a voltage higher than this limit, the actuator cannot provide it and the signal is saturated.

This condition can produce the wind-up phenomenon. When saturation occurs, the control system keeps trying to reduce the tracking error, but the actuator is already working at its maximum admissible value. As a consequence, the internal control action may continue to increase even though the real voltage applied to the motor is limited. This can cause overshoot, slower recovery and a worse transient response when the system exits the saturation region.

In the implementation, the anti-windup is obtained by saving the saturated output as the previous output of the controller. In this way, at the following sampling instant, the regulator starts again from the voltage that was actually applied to the motor, instead of starting from a value that cannot be physically generated by the actuator.

```
% Saturation and anti-windup
if (EnableCheckSat == 1)
    if out > HiSat
        out = HiSat;
    elseif out < LowSat
        out = LowSat;
    end

    % Anti-windup:
    % the saturated output is saved in out_prev
end

% Update controller states
out_prev = out;
in_prev = in;
```

Listing 3.3: Saturation and anti-windup implementation

The effect of this implementation can be observed in the following experimental results. In Figure 3.5, the current reference is set to 1 A. At the beginning of the test, the measured current rapidly increases and reaches the reference value. After the initial transient, however, the measured current decreases and remains below the reference. This is due to the saturation of the available voltage: the actuator is not able to provide more voltage, so the desired current cannot be maintained for the whole duration of the test.

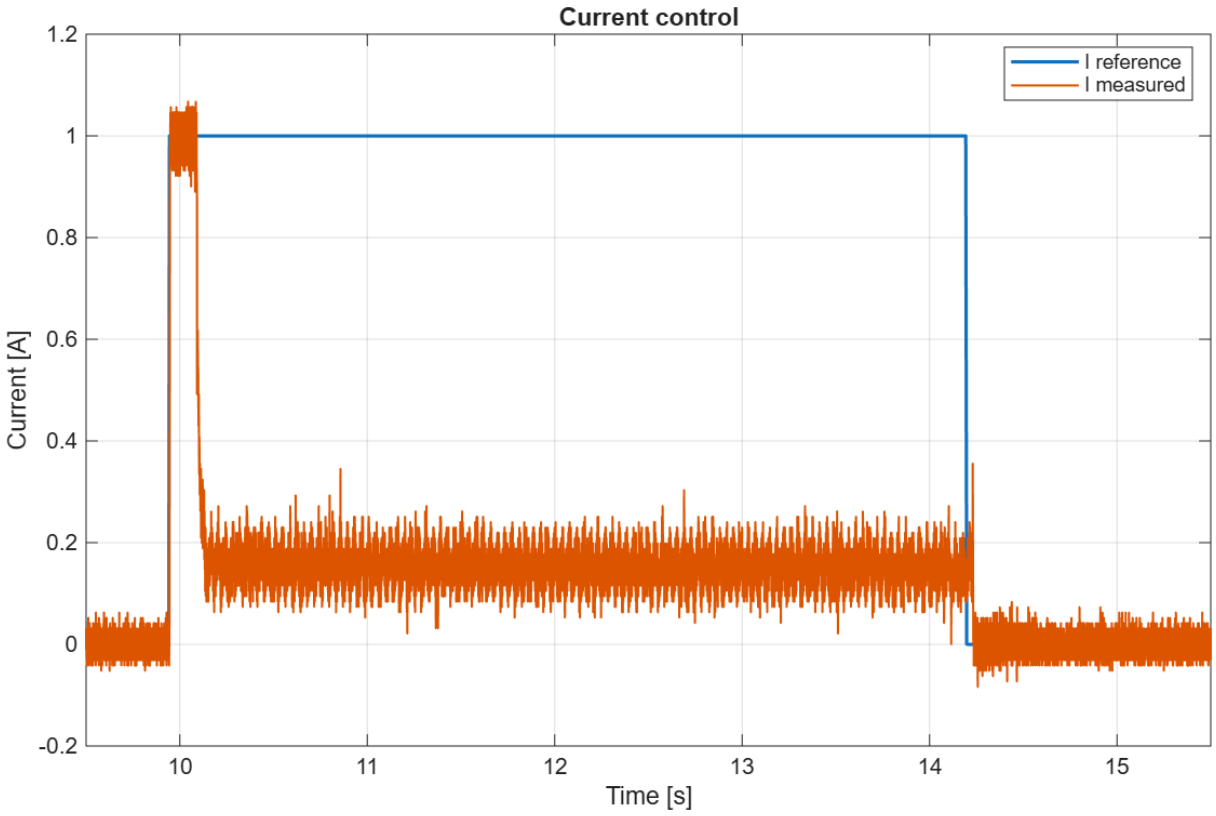


Figure 3.5: Current response during voltage saturation

This behavior is confirmed by Figure 3.6. The requested voltage reaches the DC-link voltage and remains saturated for most of the transient. This shows that the system is operating at the physical voltage limit. Thanks to the anti-windup implementation, the controller does not continue to accumulate an unrealistic control action beyond this limit.

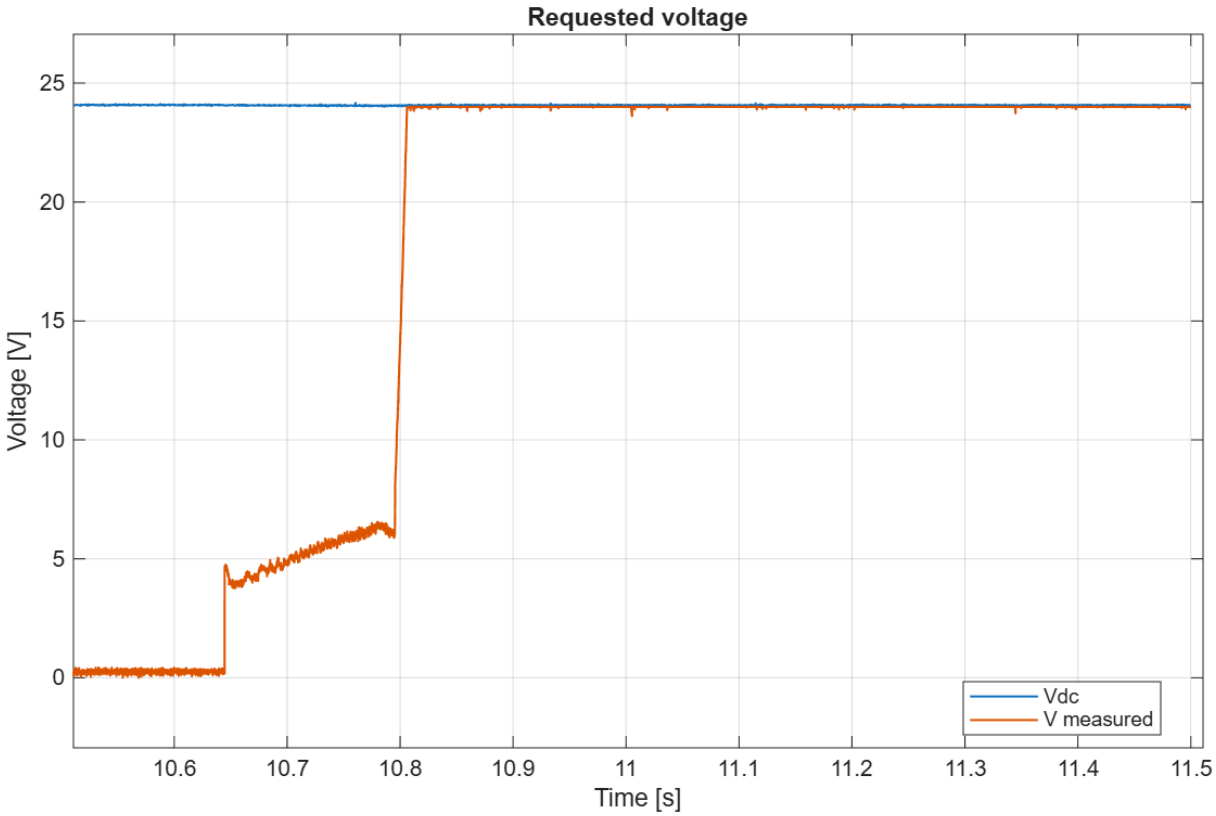


Figure 3.6: Requested voltage with PI anti-windup

When the current reference returns to zero, the measured current also decreases quickly towards zero. This indicates that the saturated phase did not produce a long recovery time. Therefore, the anti-windup implementation allows the system to handle the voltage saturation correctly, preventing the excessive accumulation of control action and improving the behavior after saturation.

3.3 PI controller for Speed Tracking

After the implementation and testing of the current loop, the external speed loop was added. The aim of this second loop is to make the motor speed follow the desired reference. The output of the speed controller is the current reference for the inner loop, which means that the speed controller indirectly controls the motor torque.

The complete block diagram used for the design of the speed loop is shown in Figure 3.7.

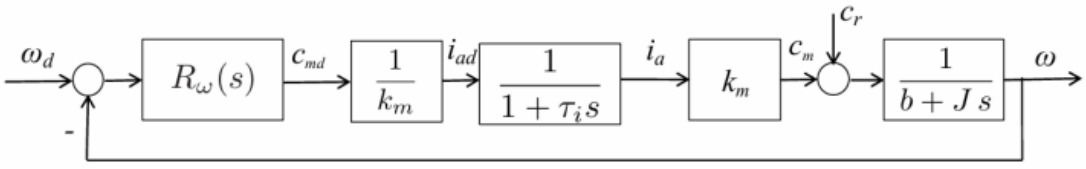


Figure 3.7: Complete block diagram for the cascade control of the DC motor

Through the motor torque constant k_m, this current reference generates the electromagnetic torque required to accelerate the load inertia J.

The mechanical dynamics of the DC motor are described by:

$$G_m(s) = \frac{1}{b + Js} = \frac{1/b}{1 + \frac{J}{b}s} = \frac{1/b}{1 + \tau_m s} \tag{3.30}$$

where:

$$\tau_m = \frac{J}{b} \tag{3.31}$$

In the design of the speed controller, a simplified procedure was used. Since the current loop is much faster than the mechanical dynamics, the closed-loop current dynamics can be neglected in the frequency range of interest for the speed loop:

$$\tau_i \simeq 0 \qquad \Rightarrow \qquad F_i(s) \simeq 1 \tag{3.32}$$

Therefore, the open-loop transfer function of the speed loop can be approximated as:

$$G_\omega(s) = R_\omega(s) F_i(s) k_m G_m(s) \simeq R_\omega(s) k_m G_m(s) \tag{3.33}$$

Using a PI controller for the speed loop:

$$R_\omega(s) = K_{p\omega}\left(1 + \frac{1}{T_{i\omega}s}\right) = \frac{K_{p\omega}}{T_{i\omega}} \frac{1 + T_{i\omega}s}{s} \tag{3.34}$$

the open-loop transfer function becomes:

$$G_\omega(s) \simeq \frac{K_{p\omega}k_m}{T_{i\omega}} \frac{1 + T_{i\omega}s}{s} \frac{1/b}{1 + \tau_m s} \tag{3.35}$$

As done for the current loop, the integral time of the speed PI controller is chosen in order to cancel the mechanical pole:

$$T_{i\omega} = \tau_m \tag{3.36}$$

With this choice, the open-loop transfer function becomes:

$$G_\omega(s) = \frac{K_{p\omega}k_m}{bT_{i\omega}s} \tag{3.37}$$

and the closed-loop transfer function is:

$$F_\omega(s) = \frac{\omega(s)}{\omega_d(s)} = \frac{G_\omega(s)}{1 + G_\omega(s)} = \frac{1}{1 + \frac{1}{G_\omega(s)}} = \frac{1}{1 + \frac{bT_{i\omega}}{K_{p\omega}k_m}s} = \frac{1}{1 + \tau_\omega s} \tag{3.38}$$

By assigning a desired time constant τ_ω for the speed loop, the proportional gain of the speed PI controller is:

$$K_{p\omega} = \frac{bT_{i\omega}}{\tau_\omega k_m} = \frac{J}{\tau_\omega k_m} \tag{3.39}$$

Therefore, the parameters of the speed PI controller are:

$$T_{i\omega} = \tau_m, \qquad K_{p\omega} = \frac{J}{\tau_\omega k_m} \tag{3.40}$$

The speed loop must be slower than the current loop in order to preserve the cascade structure. For this reason, in our implementation the speed-loop time constant was chosen as:

$$\tau_\omega = 10\tau_i \tag{3.41}$$

Since the current-loop time constant was:

$$\tau_i = 1 \cdot 10^{-3}\, s \tag{3.42}$$

the speed-loop time constant becomes:

$$\tau_\omega = 10 \cdot 10^{-3} = 1 \cdot 10^{-2}\, s = 10\, ms \tag{3.43}$$

3.3.1 Discretization and Implementation

The PI transfer function and its discretization are the same as the ones used for the current controller. The Tustin method was again adopted, and the same Matlab function was used for the software implementation. The only difference is the choice of the PI parameters, since in this case the controller acts on the speed error and its output is the current reference for the inner current loop.

The integral time of the speed PI controller was chosen equal to the mechanical time constant:

$$T_{i\omega} = \tau_m = 0.085\, s \tag{3.44}$$

Using the motor parameters defined in the Section 2.2, with:

$$J = 2 \cdot 0.0000352 = 7.04 \cdot 10^{-5}\, kg\, m^2, \qquad k_m = 0.11\, Nm/A \tag{3.45}$$

the proportional gain of the speed PI controller is:

$$K_{p\omega} = \frac{J}{\tau_\omega k_m} = \frac{7.04 \cdot 10^{-5}}{0.01 \cdot 0.11} \simeq 0.064 \tag{3.46}$$

Therefore, the parameters used for the speed PI controller are:

$$T_{i\omega} = 0.085\, s, \qquad K_{p\omega} \simeq 0.064 \tag{3.47}$$

The obtained speed response is shown in Figure 3.8. The test was performed by applying a step reference of $800\, rpm$ with a DC-link voltage of $24\, V$. The measured speed reaches the reference very quickly, showing that the cascade controller is able to provide a fast transient response. A small overshoot is present just after the rising edge, but the speed rapidly settles close to the desired value.

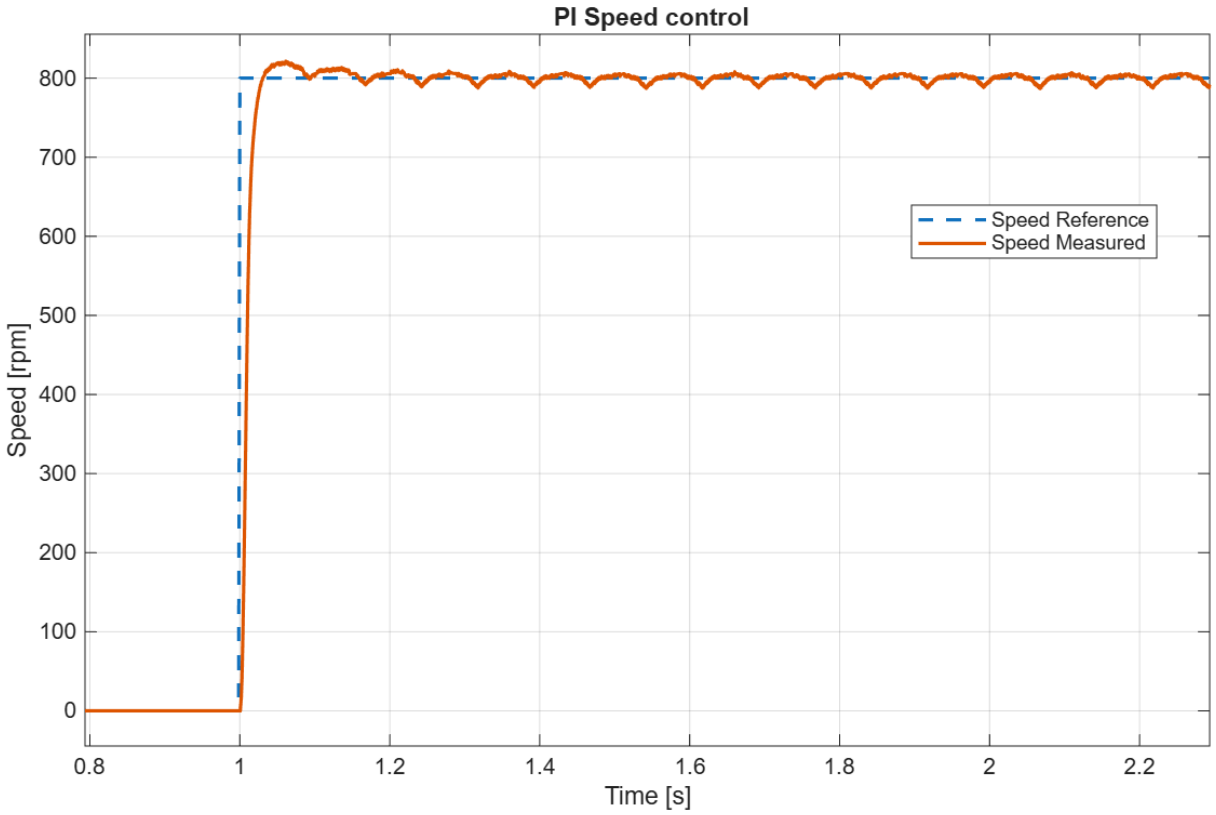


Figure 3.8: PI speed control response at $800\,rpm$ with $24\,V$ supply

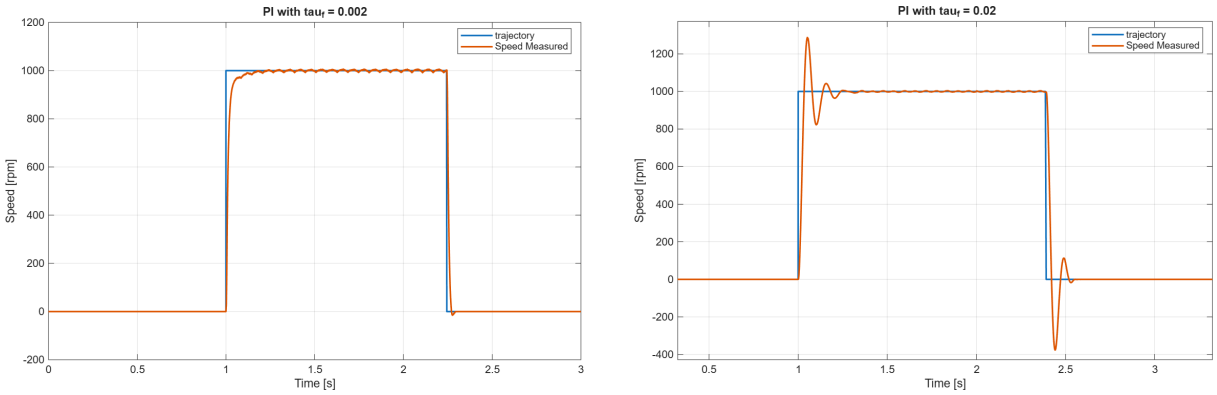
After the transient, the measured speed remains close to the reference. Small periodic oscillations can be observed around the steady-state value. These oscillations are mainly due to measurement noise, encoder quantization and the switching behavior of the PWM. However, their amplitude is limited and the average value of the speed remains very close to the imposed reference.

Overall, the result confirms that the external speed loop is correctly coordinated with the inner current loop. The current loop is sufficiently fast to be considered almost instantaneous from the point of view of the speed controller, while the speed PI provides a stable and accurate tracking of the reference.

The measured speed is obtained from the encoder and is affected by noise and quantization effects. For this reason, a low-pass filter was analyzed. The choice of the filter time constant τ_f is important because a larger value reduces noise but also introduces delay in the feedback loop.

Two values of τ_f were tested:

$$\tau_f = 0.002\,s \qquad \text{and} \qquad \tau_f = 0.02\,s \tag{3.48}$$



(a) $\tau_f = 0.002\,s(chosen)$ (b) $\tau_f = 0.02\,s$

Figure 3.9: Effect of the filter time constant on the speed response

From Figure 3.9, it can be observed that with $\tau_f = 0.002\,s$ the speed follows the reference with a fast and stable response. A small undershoot appears after the falling edge of the speed reference. This is mainly due to the motor inertia, the finite bandwidth of the cascaded control loop, and the delay introduced by the speed measurement filter. Since it is small and quickly damped, it does not indicate controller instability.

On the other hand, with $\tau_f = 0.02\,s$, the larger delay introduced by the filter causes a significant overshoot and oscillations during the transient. For this reason, $\tau_f = 0.002\,s$ was selected as the best compromise between noise reduction and closed-loop stability.

3.3.2 Speed loop anti-windup implementation

After implementing the anti-windup strategy in the current loop, the same approach was also applied to the speed controller. In this case, the output of the speed PI is the current reference for the inner current loop. Therefore, when the required current exceeds the allowed limits, the output of the speed controller must be saturated to avoid unrealistic current requests.

As for the current loop, the anti-windup action is obtained by saving the saturated output as the previous output of the controller. In this way, the integral action does not continue to accumulate error when the controller output is already limited by the saturation block.

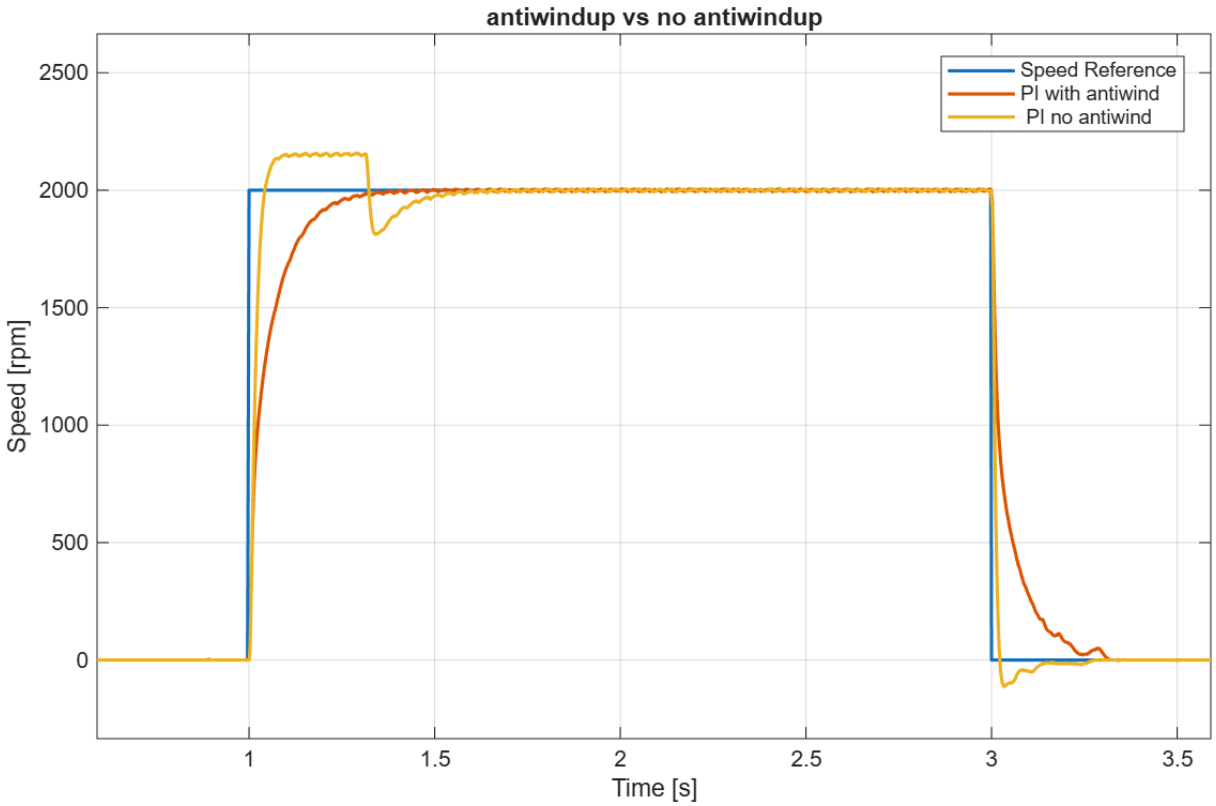


Figure 3.10: Comparison between speed control with and without anti-windup

Figure 3.10 shows the comparison between the speed response with and without anti-windup for a reference of 2000 rpm. Without anti-windup, the controller output saturates during the initial transient and the integral action keeps accumulating error. As a consequence, the response shows a large overshoot, reaching a speed higher than the imposed reference, and then requires more time to settle.

With the anti-windup implementation, the overshoot is eliminated. The response is slightly slower during the rising transient, because the controller does not allow the integral action to grow beyond the saturation limit, but the behavior is more regular and stable. When the reference returns to zero, the response with anti-windup decreases more smoothly, while the response without anti-windup shows a negative undershoot due to the previous accumulation of the integral action.

Overall, the comparison confirms that the anti-windup strategy improves the behavior of the speed loop during saturation, reducing overshoot and avoiding excessive recovery times after large reference variations.

3.3.3 Final PI control scheme

After the design and implementation of both the current and speed controllers, the final cascade PI control scheme was built in Simulink. The complete model includes the outer speed loop, which generates the current reference, and the inner current loop, which generates the requested voltage for the motor.

The scheme also includes the saturation, the anti-windup logic and the back-EMF compensation term. In this way, the final control structure is able to regulate the motor speed while keeping the current and voltage requests within the admissible limits.

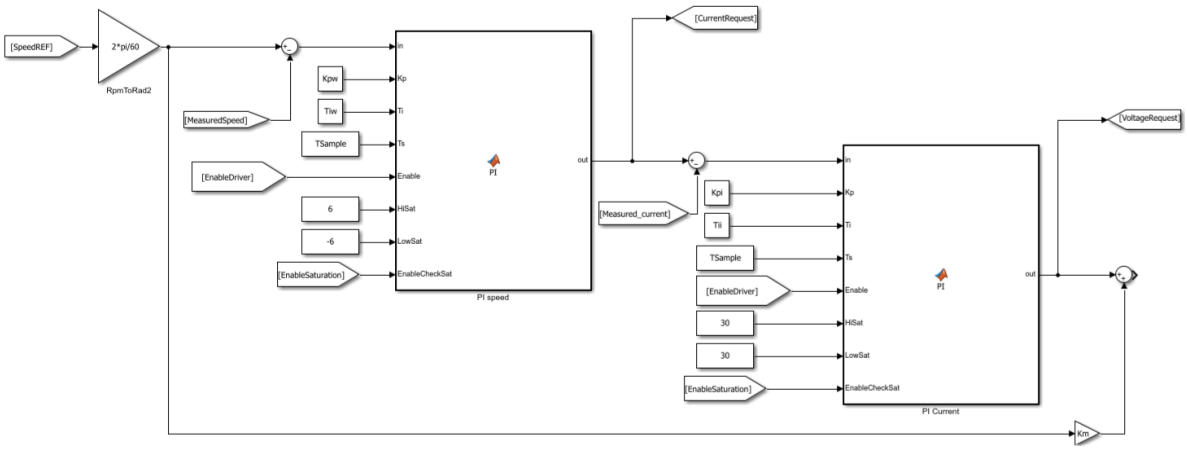


Figure 3.11: Final Simulink implementation of the cascade PI control

This final implementation was then used to perform the experimental tests on the motor and to evaluate the tracking performance of the complete cascade controller.

PI control with and without load

The final PI cascade controller was tested both with and without load connected to the motor. The aim of this test was to verify the robustness of the controller when an additional resistant torque is applied to the motor shaft.

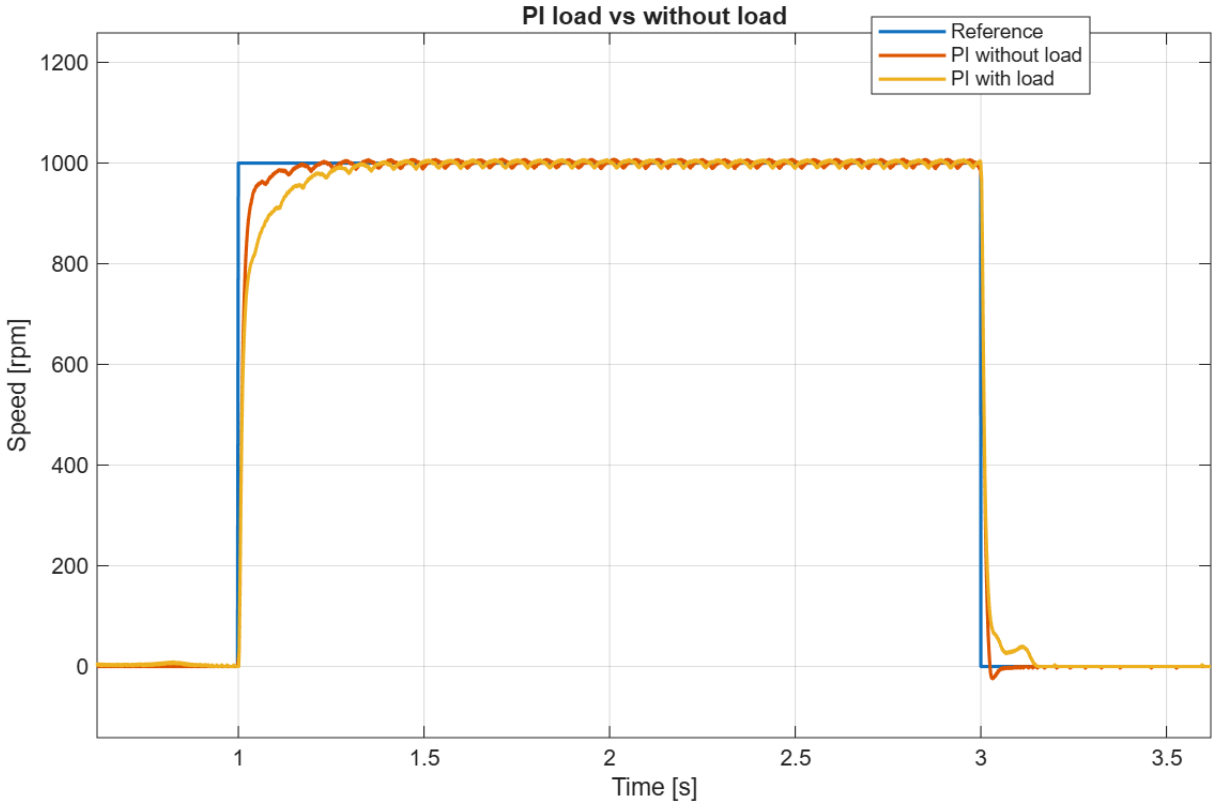


Figure 3.12: PI speed control response with and without load

From Figure 3.12, it can be observed that both responses reach the reference value of 1000 rpm with zero steady-state error. Without load, the speed reaches the reference

faster, since the motor has to overcome only its own inertia and friction. When the load is connected, the transient is slightly slower because the motor must provide a higher torque.

However, after the transient, the loaded response also reaches the desired speed and remains close to the reference. This shows that the PI controller is able to compensate the additional resistant torque introduced by the load. When the reference returns to zero, the loaded case presents a slightly slower decay and a small residual oscillation, but the system remains stable and returns correctly to rest.

Overall, the test confirms that the cascade PI controller works properly also in presence of load, with only a limited degradation of the transient response.

As shown in Figure 3.12, a small undershoot can be observed when the speed reference returns to zero. This behavior is due to an data mistake: the plotted signal was taken from a `simout` block referring to the PI controller without anti-windup compensation, instead of the final PI controller with anti-windup. However, this does not affect the validity of the considerations made above.

Effect of different current saturation limits

A second test was performed by changing the saturation limit imposed on the current reference generated by the speed controller. The current saturation limits the maximum torque that the motor can produce, since the electromagnetic torque is proportional to the armature current.

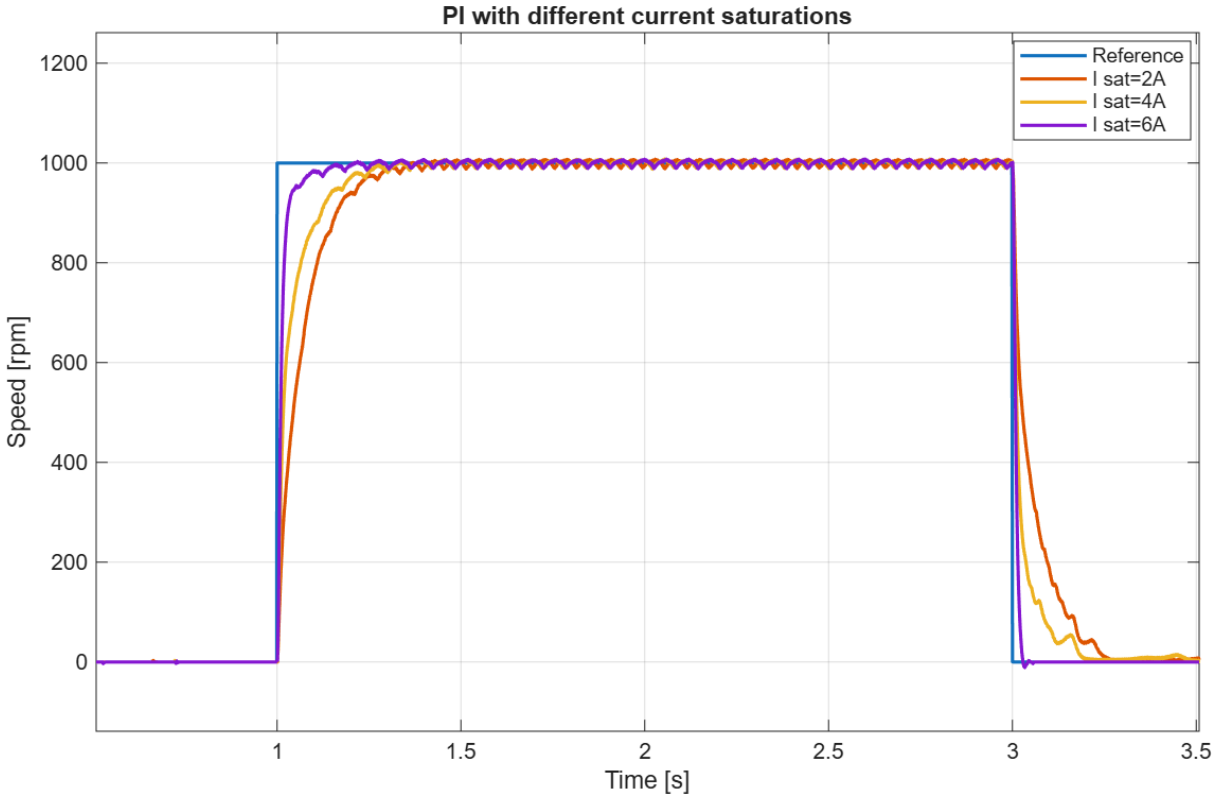


Figure 3.13: PI speed control response with different current saturation limits

Figure 3.13 compares the speed response for three different current saturation values: $2\,A$, $4\,A$ and $6\,A$. As expected, a higher current saturation allows the controller to request a larger torque, resulting in a faster transient response.

With $I_{sat} = 2\,A$, the acceleration is slower and the motor takes more time to reach the reference. With $I_{sat} = 4\,A$, the response becomes faster, while with $I_{sat} = 6\,A$ the speed reaches the reference in the shortest time. During steady state, all three responses remain close to $1000\,rpm$, showing that the saturation mainly affects the transient behavior and not the final tracking accuracy.

When the reference returns to zero, the same effect can be observed: lower current saturation values produce a slower deceleration, while higher saturation allows the motor to reduce its speed more rapidly. Therefore, the current saturation represents a trade-off between response speed and limitation of the current stress on the motor and converter. As shown in Figure 3.13, a small undershoot can be observed when the speed reference returns to zero. This behavior is due to an data mistake: the plotted signal was taken from a `simout` block referring to the PI controller without anti-windup compensation, instead of the final PI controller with anti-windup. However, this does not affect the validity of the considerations made above.

3.3.4 Trajectory tracking and feedforward action

After testing the PI cascade controller on step references, the controller was tested on more complex speed trajectories. The aim of these tests was to evaluate the ability of the controller to follow references with ramps, fast changes and different speed levels.

Figure 3.14 shows the response of the PI controller without feedforward compensation on a trajectory with positive and negative speed values. The measured speed follows the reference quite well during most of the trajectory. Small delays can be observed during the fastest transitions, especially at the beginning of the ramps, but the tracking remains stable and accurate.

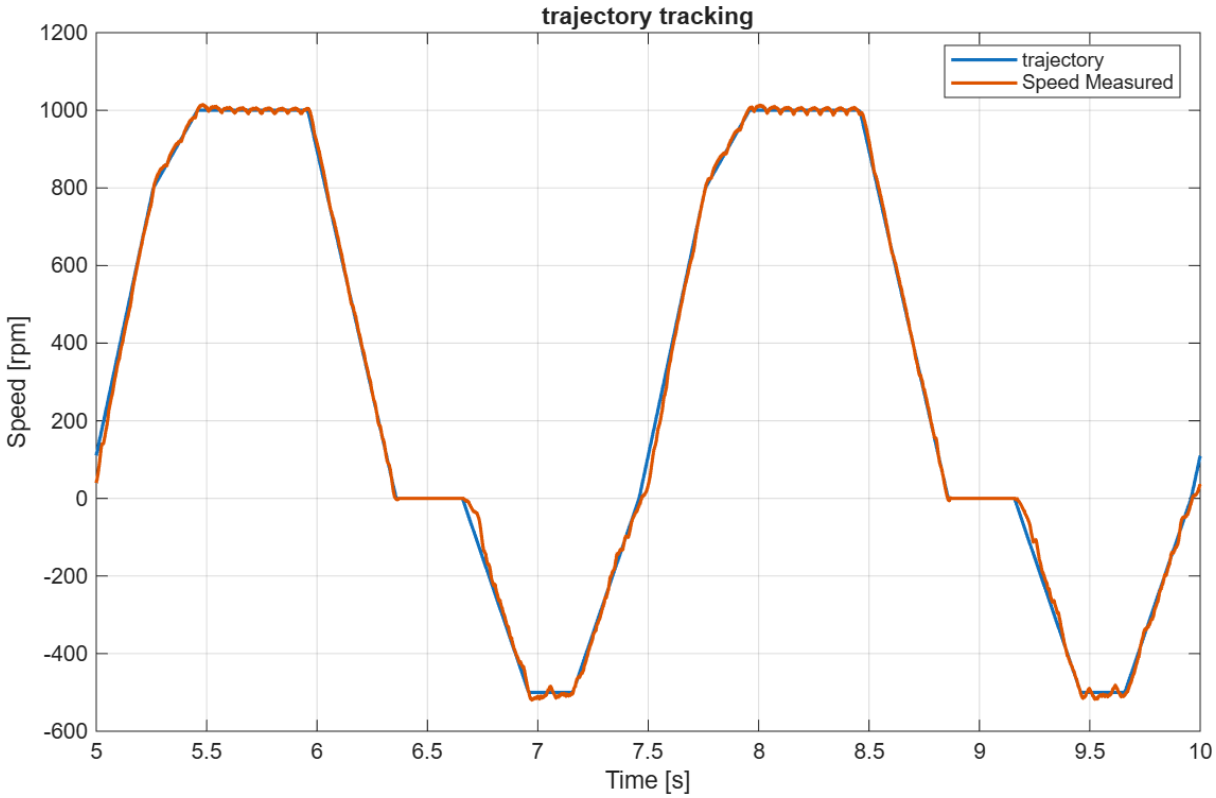


Figure 3.14: Trajectory tracking with PI controller without feedforward compensation

To improve the tracking performance on faster trajectories, a feedforward action was also introduced. The idea is to add a contribution based on the desired trajectory, so that part of the required control action is generated in advance and not only after the appearance of a tracking error.

Starting from the mechanical equation of the motor:

$$J\dot{\omega}(t) + b\omega(t) = k_m i(t) \tag{3.49}$$

we simplified the model by neglecting the viscous friction term, thus assuming:

$$b = 0 \tag{3.50}$$

With this approximation, the mechanical equation becomes:

$$J\dot{\omega}(t) = k_m i(t) \tag{3.51}$$

Therefore, the feedforward current needed to follow the desired trajectory was computed as:

$$i_{ff}(t) = \frac{J\dot{\omega}_d(t)}{k_m} \tag{3.52}$$

where $\dot{\omega}_d(t)$ is the desired acceleration obtained from the reference trajectory. This feedforward contribution is added to the output of the speed PI controller. In this way, during acceleration and deceleration phases, the controller already receives part of the current required to follow the trajectory, reducing the delay with respect to a pure feedback control.

The comparison between the PI controller without and with feedforward compensation is shown in Figure **??**. Without feedforward, the measured speed follows the reference with a visible delay during the acceleration and deceleration phases. With feedforward compensation, the response becomes more prompt and the measured speed follows the ramps more closely.

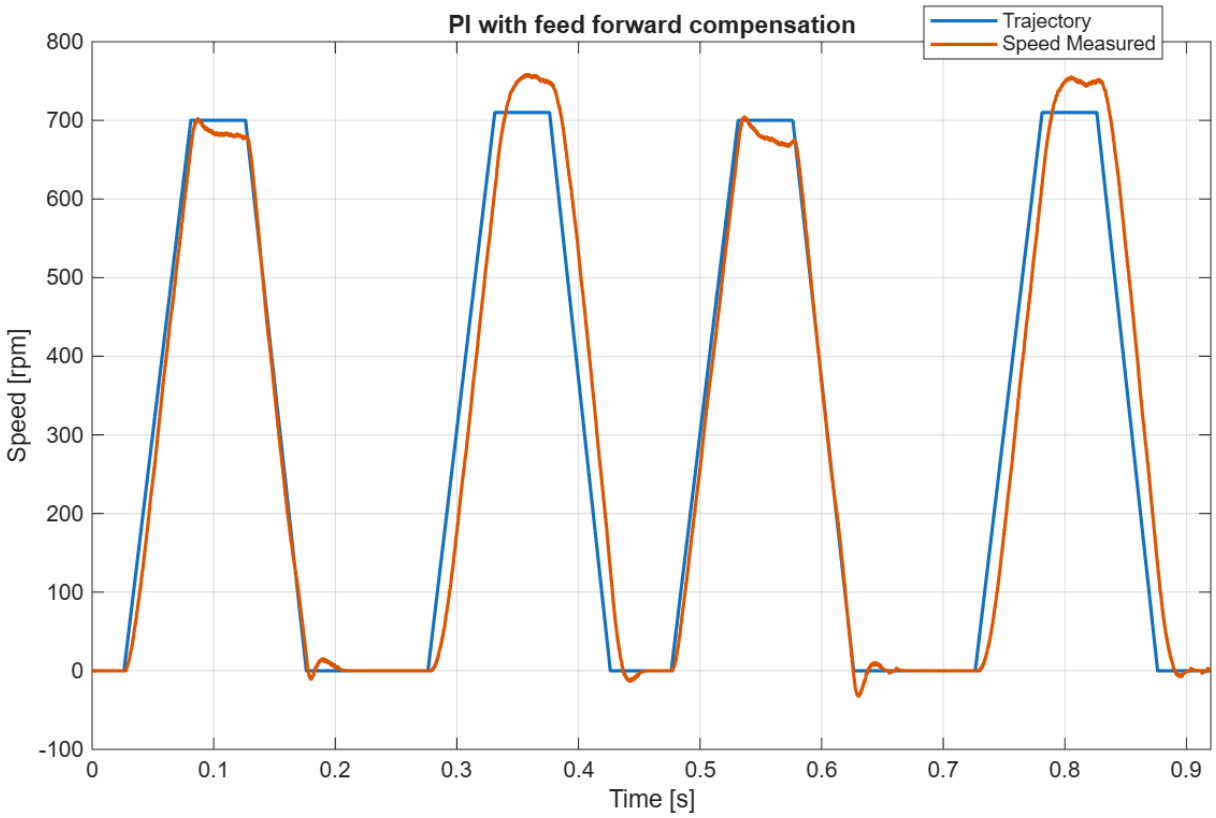


Figure 3.15: Comparison between PI tracking with and without feedforward compensationn

As shown in Figure 3.15, the effect of the feedforward compensation can be observed in the first and third speed peaks. In these intervals, the feedforward action is enabled and the measured speed follows the reference trajectory more closely, reducing the tracking delay during the acceleration and deceleration phases.

On the other hand, during the second and fourth peaks, the feedforward contribution was intentionally disabled. For this reason, the response is slower and a larger tracking error can be observed, especially during the transient phases.

Therefore, the feedforward contribution is particularly useful when the reference changes rapidly, since it helps the controller anticipate the required action and improves the dynamic tracking performance of the speed control loop.

Chapter 4

Parameter Estimation Using Least Squares

4.1 Motivation for Parameter Estimation

The design of an effective control system relies heavily on the accuracy of the mathematical model used to describe the physical plant. For a DC motor, the governing dynamic equations depend on specific electrical and mechanical parameters, namely the armature resistance R, the armature inductance L, the equivalent inertia J, and the viscous friction coefficient b.

Typically, these values are provided by the manufacturer in the motor's datasheet. However, relying exclusively on nominal datasheet values can lead to suboptimal control performances or even instability. Nominal parameters are subject to manufacturing tolerances and do not account for the specific operating conditions of the system. For instance, the armature resistance R increases significantly as the motor heats up during operation. Similarly, the mechanical parameters J and b can vary due to the addition of external loads or wear-related phenomena in the bearings and brushes over the motor's lifespan.

Therefore, to design a robust, high-performance controller—especially when implementing advanced techniques such as feed-forward compensation or fine-tuned PI regulators—it is necessary to perform an experimental parameter estimation. This system identification procedure bridges the gap between the theoretical model and the actual physical system, allowing the control algorithm to adapt to the real and current state of the plant.

4.2 Least Squares Method

To experimentally estimate the unknown parameters of the DC motor, the Least Squares (LS) algorithm was used. This method is a technique used to estimate the parameters of a mathematical model by minimizing the error between the experimental data and the model's predictions.

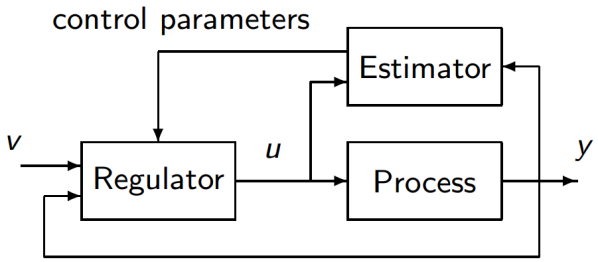


Figure 4.1: Block diagram of the Least Squares estimation scheme.

The fundamental assumption of the Least Squares approach is that the system model is linear with respect to the unknown parameters, meaning it can be written in the form:

$$y(t) = \varphi^T(t)\theta \tag{4.1}$$

where:

- $y(t)$ is the measured output of the system at time t;
- $\theta = [\theta_1, \theta_2, \ldots, \theta_n]^T$ is the vector containing the unknown parameters to be estimated;
- $\varphi(t) = [\varphi_1(t), \varphi_2(t), \ldots, \varphi_n(t)]^T$ is the regression vector, which contains known variables such as past inputs and past outputs.

In a real scenario, the measured output is affected by noise or modeling approximations. Therefore, the actual measurement can be written as:

$$y(t) = \varphi^T(t)\theta + \varepsilon(t) \tag{4.2}$$

where $\varepsilon(t)$ represents the estimation error (or residual) at time t.

Given a set of N experimental measurements, we can define a system of equations in matrix form:

$$Y = \Phi\theta + \mathcal{E} \tag{4.3}$$

where $Y \in \mathbb{R}^{N\times 1}$ is the vector of all collected outputs, $\Phi \in \mathbb{R}^{N\times n}$ is the regression matrix constructed by stacking all the regression vectors $\varphi^T(t)$, and $\mathcal{E}$ is the error vector.

The objective of the Least Squares method is to find the optimal parameter vector $\hat{\theta}$ that minimizes the cost function $J(\theta)$, defined as the sum of the squared estimation errors:

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{N}\varepsilon_i^2 = \frac{1}{2}\mathcal{E}^T\mathcal{E} = \frac{1}{2}(Y - \Phi\theta)^T(Y - \Phi\theta) \tag{4.4}$$

By differentiating the cost function $J(\theta)$ with respect to θ and setting the derivative to zero, we obtain the normal equation. Assuming that the matrix $\Phi^T\Phi$ is non-singular (which implies that the system is persistently excited), the optimal estimate $\hat{\theta}$ can be computed algebraically using the pseudo-inverse formulation:

$$\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^T Y \tag{4.5}$$

This matrix computation forms the core of the MATLAB estimation scripts used to extract the physical parameters of the DC motor from the sampled input-output data.

4.3 Discrete-Time Formulation of the Problem

The electrical and mechanical subsystems of the DC motor are governed by continuous-time differential equations. However, data acquisition systems and digital controllers operate on discrete-time signals, sampled at a constant time interval, denoted as the sampling period T_s. Therefore, to apply the Least Squares estimation technique described in the previous section, it is necessary to convert the continuous-time equations into a discrete-time difference equation form.

Both the electrical and mechanical dynamics of the DC motor can be mathematically modeled as first-order differential equations. In general terms, a continuous-time linear first-order system can be expressed as:

$$\dot{y}(t) = a_c y(t) + b_c u(t) \tag{4.6}$$

where $y(t)$ is the system output, $u(t)$ is the input excitation signal, and a_c, b_c are the continuous-time physical parameters.

To translate Eq. (4.6) into the discrete-time domain, the derivative $\dot{y}(t)$ must be approximated. In order to do so the *Backward Euler* (or implicit Euler) numerical integration scheme is chosen. The continuous derivative at time $t = kT_s$ is approximated using the current and the previous sampled points:

$$\dot{y}(t) \approx \frac{y(k) - y(k-1)}{T_s} \tag{4.7}$$

where $y(k)$ denotes the sampled value $y(kT_s)$; $[y(k) = y(kT_s)]$.

Substituting this approximation into the continuous-time model yields:

$$\frac{y(k) - y(k-1)}{T_s} = a_c y(k) + b_c u(k) \tag{4.8}$$

By algebraically rearranging the terms to isolate the current output $y(k)$ on the left-hand side, we obtain:

$$y(k) - T_s a_c y(k) = y(k-1) + T_s b_c u(k) \tag{4.9}$$

$$(1 - T_s a_c) y(k) = y(k-1) + T_s b_c u(k) \tag{4.10}$$

$$y(k) = \frac{1}{1 - T_s a_c} y(k-1) + \frac{T_s b_c}{1 - T_s a_c} u(k) \tag{4.11}$$

Eq. (4.11) represents a standard linear difference equation, which expresses the system's current output as a linear combination of the past output and the current input. This formulation can be compactly rewritten by defining two discrete-time coefficients, α_1 and α_2:

$$y(k) = \alpha_1 y(k-1) + \alpha_2 u(k) \tag{4.12}$$

where:

$$\alpha_1 = \frac{1}{1 - T_s a_c}, \quad \alpha_2 = \frac{T_s b_c}{1 - T_s a_c} \tag{4.13}$$

The difference equation formulated in Eq. (4.12) matches the linear regression form required by the Least Squares algorithm, $y(k) = \varphi^T(k)\theta$. For a generic window of N

samples, the regression vector $\varphi(k)$ and the unknown parameter vector θ are structured as follows:

$$\varphi(k) = \begin{bmatrix} y(k-1) \\ u(k) \end{bmatrix}, \quad \theta = \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} \tag{4.14}$$

Once the optimal discrete-time parameter vector $\hat{\theta} = [\alpha_1, \alpha_2]^T$ is computed via the pseudo-inverse formula, the original continuous-time physical parameters a_c and b_c can be uniquely reconstructed by inverting the relations:

$$a_c = \frac{\alpha_1 - 1}{\alpha_1 T_s}, \quad b_c = \frac{\alpha_2}{\alpha_1 T_s} \tag{4.15}$$

This mathematical framework establishes the exact procedure utilized in the estimation routines to map the discrete coefficients identified by MATLAB back into the physical constants of the DC motor.

4.4 Estimation of Electrical Parameters

The electrical dynamics of a DC motor are governed by Kirchhoff's voltage law, which yields the following continuous-time differential equation:

$$v(t) = Ri(t) + L\frac{di(t)}{dt} + e_a(t) \tag{4.16}$$

where $v(t)$ is the applied armature voltage, $i(t)$ is the armature current, and $e_a(t)$ is the back-electromotive force (back-EMF). The back-EMF is directly proportional to the motor's angular velocity $\omega(t)$ through the torque constant K_m, such that $e_a(t) = K_m\omega(t)$.

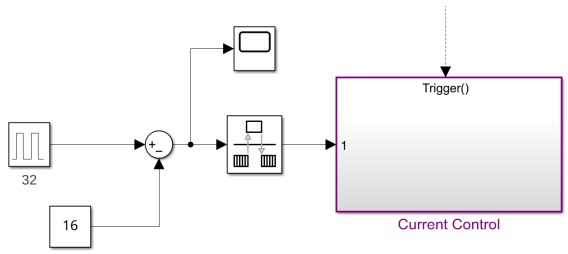


Figure 4.2: Input for the tracing model

(The constant which is subtracted from the pulse generator must be equal to the pulse generator's amplitude divided by 2, so that the average value is null)

To accurately estimate the electrical parameters (resistance R and inductance L) using the Least Squares method, it is necessary to isolate the electrical subsystem from the mechanical one. This is achieved by imposing $\omega(t) = 0$, which consequently sets $e_a(t) = 0$. Experimentally, this condition was satisfied by feeding the motor with a voltage step and acquiring the data strictly during the very first phase of the transient ($N = 1000$ samples with a sampling time $T_s = 0.0001$ s), right before the rotor started spinning due to its mechanical inertia.

Under this blocked-rotor assumption, Eq. (4.16) simplifies to:

$$v(t) = Ri(t) + L\frac{di(t)}{dt} \tag{4.17}$$

$[(e_a = K_m\omega = 0)]$
By rearranging the terms to match the standard first-order form $\dot{y}(t) = a_c y(t) + b_c u(t)$ introduced in the previous section, we obtain:

$$\frac{di(t)}{dt} = -\frac{R}{L}i(t) + \frac{1}{L}v(t) \tag{4.18}$$

Comparing Eq. (4.18) with the generic formulation, the continuous-time physical parameters map to the system constants as follows:

$$a_c = -\frac{R}{L}, \quad b_c = \frac{1}{L} \tag{4.19}$$

By running the Least Squares algorithm on the acquired voltage and current dataset, the discrete-time parameter vector $\hat{\theta} = [\alpha_1, \alpha_2]^T$ was identified. From these discrete coefficients, R and L were extracted.

4.4.1 Estimation of the Armature Resistance

The armature resistance R dictates the steady-state current absorption of the blocked motor. To extract it from the estimated discrete-time model, we rely on the continuous-to-discrete mapping derived previously:

$$a_c = \frac{\alpha_1 - 1}{\alpha_1 T_s}, \quad b_c = \frac{\alpha_2}{\alpha_1 T_s} \tag{4.20}$$

By substituting these expressions into the relation $R = -\frac{a_c}{b_c}$, the dependency on the sampling time T_s cancels out, yielding the direct estimation formula:

$$R = -\frac{\frac{\alpha_1 - 1}{\alpha_1 T_s}}{\frac{\alpha_2}{\alpha_1 T_s}} = \frac{1 - \alpha_1}{\alpha_2} \tag{4.21}$$

This exact relation was implemented in the MATLAB script, yielding an experimental resistance value of $R_{estim} = 3.1532\ \Omega$. This result differs slightly from the nominal datasheet value (3.2 Ω).

4.4.2 Estimation of the Armature Inductance

The armature inductance L characterizes the electrical transient speed of the motor. Similarly to the resistance, it can be mathematically isolated by exploiting the relation $L = \frac{1}{b_c}$. Substituting the discrete-time equivalent for b_c, we obtain:

$$L = \frac{1}{\frac{\alpha_2}{\alpha_1 T_s}} = \frac{\alpha_1}{\alpha_2} T_s \tag{4.22}$$

This formula allowed the extraction of the estimated inductance, resulting in $L_{estim} = 0.0061$ H (6.1 mH). This value is noticeably higher than the nominal parameter (4.1 mH), which further justifies the necessity of the experimental parameter identification. A comprehensive dynamic validation of these electrical parameters, including a graphical comparison against the datasheet model, will be presented in the next section.

4.5 Estimation of Mechanical Parameters

The mechanical dynamics of the DC motor are represented by the torque balance equation:

$$C_m(t) = J\dot{\omega}(t) + b\omega(t) \tag{4.23}$$

where $\omega(t)$ is the angular velocity, J is the total equivalent inertia of the rotor, b is the viscous friction coefficient, and $C_m(t)$ is the generated motor torque. The generated torque is directly proportional to the armature current $i(t)$ through the torque constant K_m, such that $C_m(t) = K_m i(t)$.

By rearranging Eq. (4.23) to isolate the derivative of the output variable $\omega(t)$, the system can be written in the standard first-order form:

$$\dot{\omega}(t) = -\frac{b}{J}\omega(t) + \frac{1}{J}C_m(t) \tag{4.24}$$

Comparing this to the generic continuous-time formulation $\dot{y}(t) = a_c y(t) + b_c u(t)$, the mechanical parameters map to the system constants as:

$$a_c = -\frac{b}{J}, \quad b_c = \frac{1}{J} \tag{4.25}$$

In the MATLAB identification script, the input to the algorithm was the torque vector ($K_m \cdot i$) and the output was the angular velocity vector (ω). By running the Least Squares algorithm over the transient dataset ($N = 20000$ samples), the discrete-time coefficients α_1 and α_2 were obtained.

4.5.1 Estimation of the Equivalent Inertia

The equivalent inertia J is extracted using the relation $J = \frac{1}{b_c}$. Substituting the discrete-time equivalent for b_c derived from the Backward Euler discretization, we obtain:

$$J = \frac{1}{\frac{\alpha_2}{\alpha_1 T_s}} = \frac{\alpha_1}{\alpha_2} T_s \tag{4.26}$$

This formulation corresponds exactly to the mathematical computation implemented in the MATLAB routine. From a physical point of view, the equivalent inertia J is an intrinsic mechanical property of the rotor assembly and must remain strictly constant regardless of the applied voltage. However, due to the experimental nature of the acquired data, measurement noise, and minor unmodeled non-idealities, the estimated values exhibited slight fluctuations across the different test runs. Therefore, to ensure robustness and mitigate these experimental variations, the procedure was repeated across multiple voltage levels, and the final equivalent inertia was computed as the average of the values extracted from the first five valid tests (14, 20, 23, 32, and 38 V). Higher voltage tests (44 V and 48 V) were deliberately excluded from this average to prevent drive saturation from affecting the calculation.

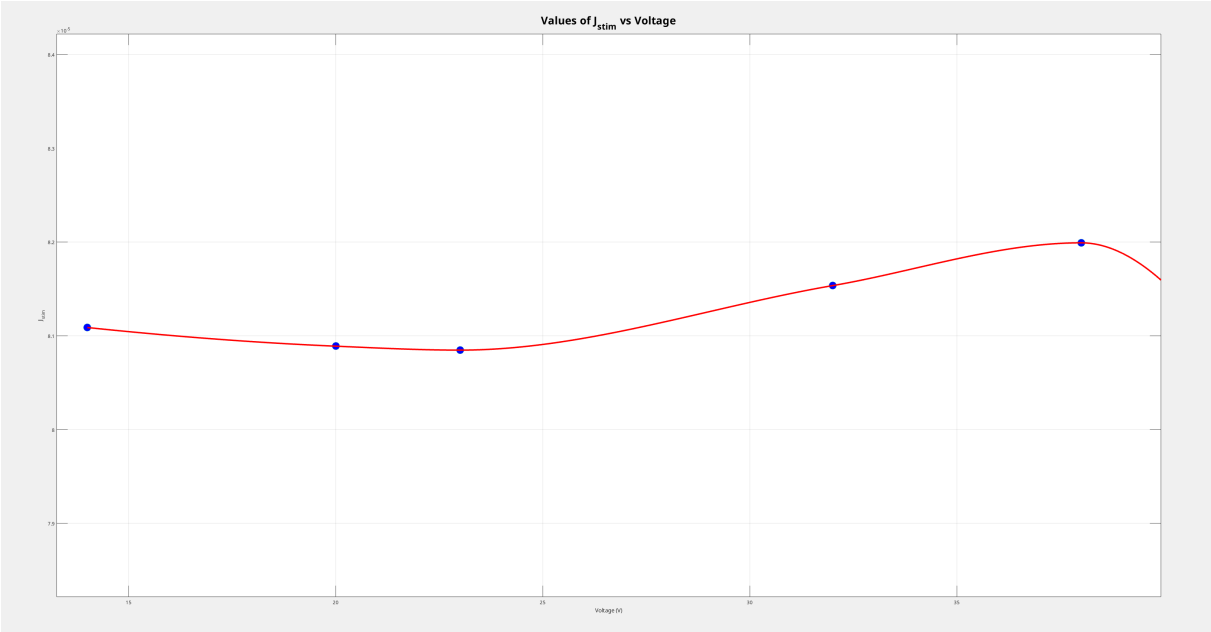


Figure 4.3: Estimated values of J

This averaging procedure yielded a final estimated inertia of $J_{estim} = 8.127 \times 10^{-5}$ kg m^2.

4.5.2 Estimation of the Viscous Friction Coefficient

Similarly to the inertia, the viscous friction coefficient b can be analytically reconstructed. From the relation $b = -J \cdot a_c$, and substituting the previously found expression for J and the discrete mapping for a_c:

$$b = -\left(\frac{\alpha_1 T_s}{\alpha_2}\right)\left(\frac{\alpha_1 - 1}{\alpha_1 T_s}\right) = \frac{1 - \alpha_1}{\alpha_2} \tag{4.27}$$

This formula yields a single, globally averaged value of the viscous friction coefficient over the considered data window. However, as investigated in the following subsection, this parameter cannot be realistically treated as a pure constant.

4.5.3 Interpolation of the Values of b

During the estimation of the mechanical parameters, it was observed that the viscous friction coefficient b varies significantly depending on the operating conditions, particularly the motor's speed and direction of rotation.

Initially, the system was excited with a symmetric square wave voltage reference. Under these conditions, the steady-state armature current exhibited different magnitudes during the positive and negative rotation phases, despite the angular velocity reaching the same absolute value in both directions. Since, at steady state ($\dot{\omega} \approx 0$), the motor torque strictly balances the friction forces ($C_m = b \cdot \omega$), this discrepancy strongly suggested an asymmetric mechanical friction.

To ensure this behavior was not caused by electrical offsets, the physical connections at the motor terminals were swapped. The phenomenon reversed, confirming that the asymmetry was entirely related to the mechanics of the motor, likely due to wear-related

phenomena in the bearings or brushes resulting from heavier usage in one specific rotational direction over its lifespan.

While the alternating square wave tests highlighted the presence of the asymmetry, extracting the exact value of b from these specific datasets wouldn't have been accurate. In fact, continuously reversing the motor's direction makes the pure steady-state friction estimation less reliable.

Least Squares Estimation of the Symmetric Friction Curve

Despite this limitation for finding the exact asymmetric values, the alternating square wave datasets were initially used to estimate an average friction profile. To do this, the standard Least Squares (LS) algorithm was applied to the entire data vector of each test (from 14 V to 48 V).

Using the discrete-time parameters $[\alpha_1, \alpha_2]^T$ found by the algorithm, the friction coefficient was calculated directly in the MATLAB script using the following formula:

$$b = \frac{1 - \alpha_1}{\alpha_2} \tag{4.28}$$

Because the LS method minimizes the overall error across the whole dataset, it naturally calculates a single, constant value for b that best fits the entire test. Since the analyzed data included both the positive and negative rotations of the square wave, the algorithm mixed the two behaviors. This means it effectively averaged out the physical asymmetry, returning a single "mean" friction value for each voltage level.

These average points (saved in the `b_stim_vec` array in MATLAB) were then interpolated to create a symmetric, velocity-dependent friction curve $b(\omega)$.

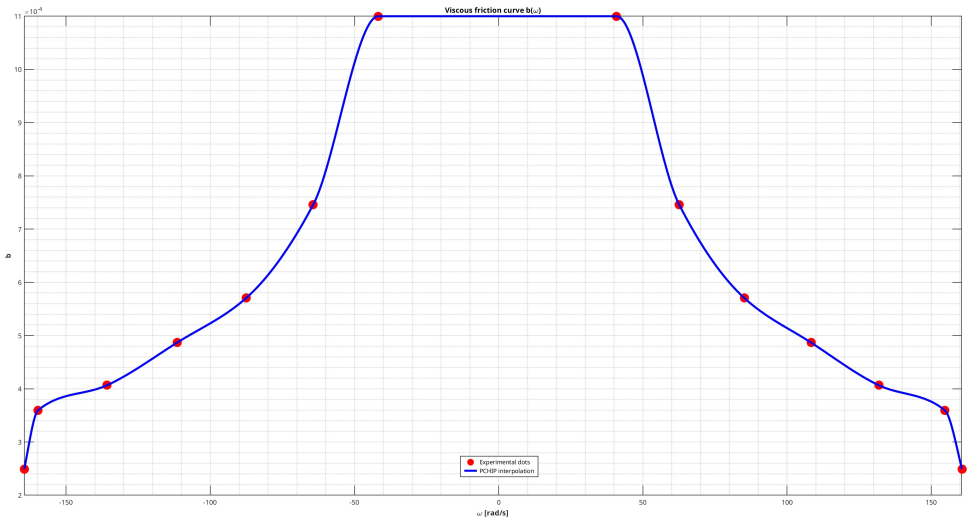


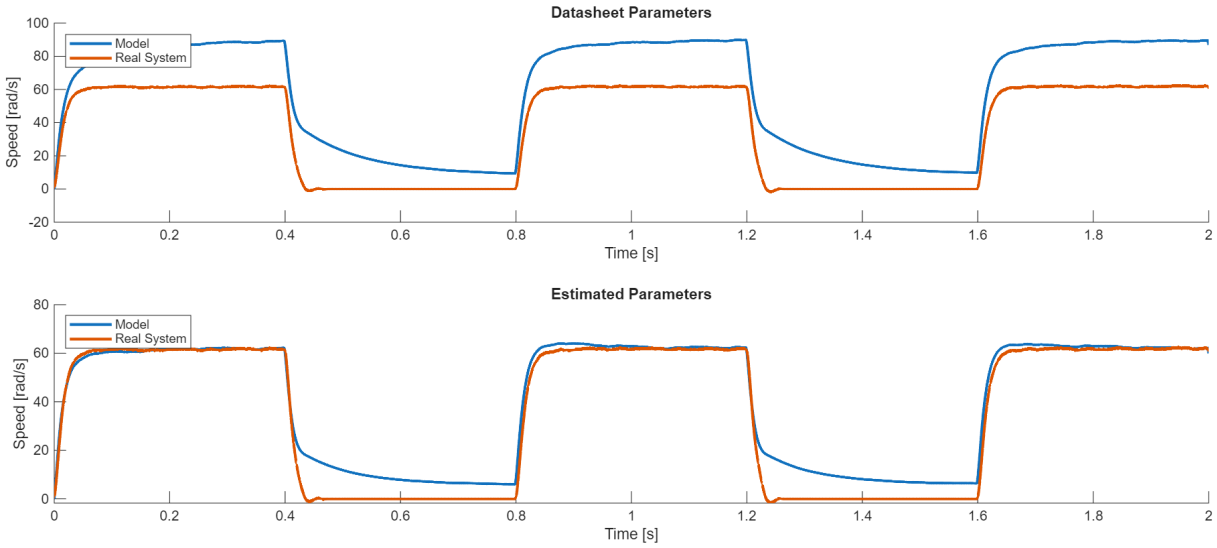
Figure 4.4: Symmetric interpolation of the viscous friction coefficient $b(\omega)$. This curve represents a globally averaged approximation of the motor's mechanical resistance, obtained via the Least Squares algorithm.

While this symmetric curve hides the wear-induced differences between the two rotational directions, it still provides a solid and mathematically stable average baseline of the motor's mechanical behavior.

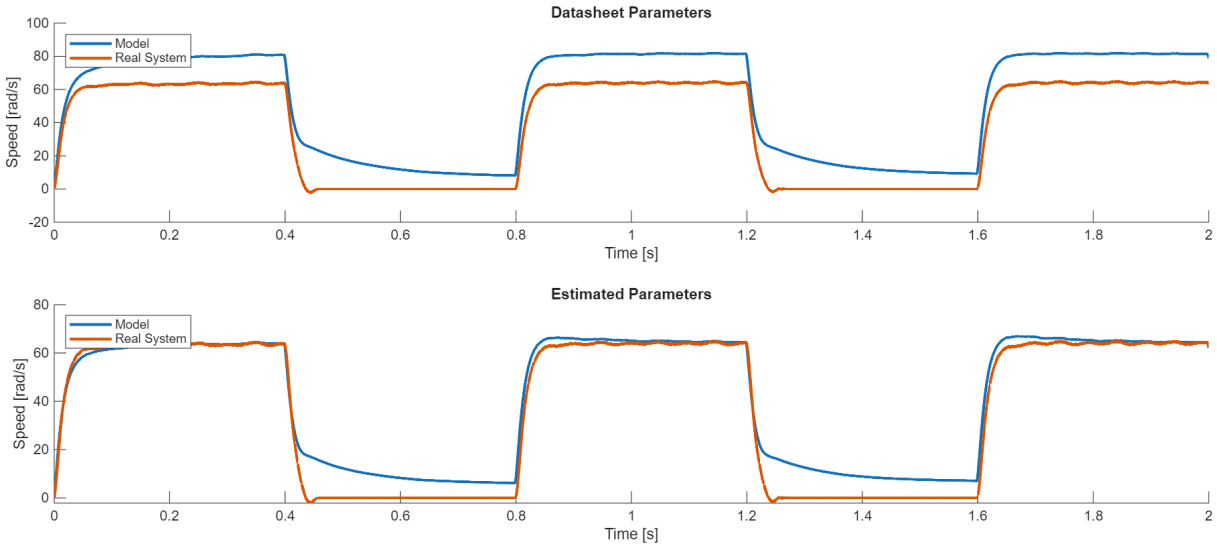
Isolation of the different friction's value

In order to isolate and quantify the magnitude of the asymmetric friction, a dedicated unidirectional test was performed at a constant supply voltage of 20 V. The motor was first driven exclusively in the positive direction, and subsequently, by swapping the input voltage terminals at the motor's side, exclusively in the negative direction (still supplying +20V). For these specific datasets, the viscous friction coefficients were estimated directly using the previously described Least Squares algorithm.

This isolated test yielded a positive viscous friction $b_{pos} = 9.6192 \times 10^{-4}$ Nms/rad and a negative viscous friction $b_{neg} = 8.4296 \times 10^{-4}$ Nms/rad.
Calculating the relative deviation $\left(\frac{b_{pos}-b_{neg}}{b_{neg}} \times 100\right)$, it was determined that the true mechanical resistance in the positive direction is approximately 14.11% higher than in the negative counterpart.



Standard connection (Positive direction)



Inverted connection (Negative direction)

Figure 4.5: Visual comparison of the steady-state velocity during unidirectional square wave tests. Focusing on the real system's response (orange curve), the motor reaches a visibly higher steady-state speed when rotating in the negative direction with inverted terminals (bottom plot, ≈ 64 rad/s) compared to the positive direction (top plot, ≈ 61 rad/s) under similar operating conditions. This is the physical evidence proves that the motor encounters a lower viscous friction resistance ($b_{neg} < b_{pos}$) in the negative rotational direction.

Extraction of the interpolation's parameters

Even if the unidirectional test provided the precise magnitude of the asymmetry at a specific operating point, a complete velocity-dependent model was required. Therefore, to evaluate this asymmetry across the motor's entire operating range, a steady-state analysis was conducted on the data acquired from the multi-voltage square wave tests

$(14, 20, 23, 32, 38, 44$ V). Even if the alternating test sequence introduces some transient disturbances, the relative behaviour at different speeds remains valid and can be exploited to derive a dense, non-linear friction mapping.

To visually understand the data extraction process for the asymmetric friction mapping, it is useful to analyze the raw data acquired during the alternating square wave tests. Figure 4.6 displays the time evolution of the system's output during the ± 14 V test.

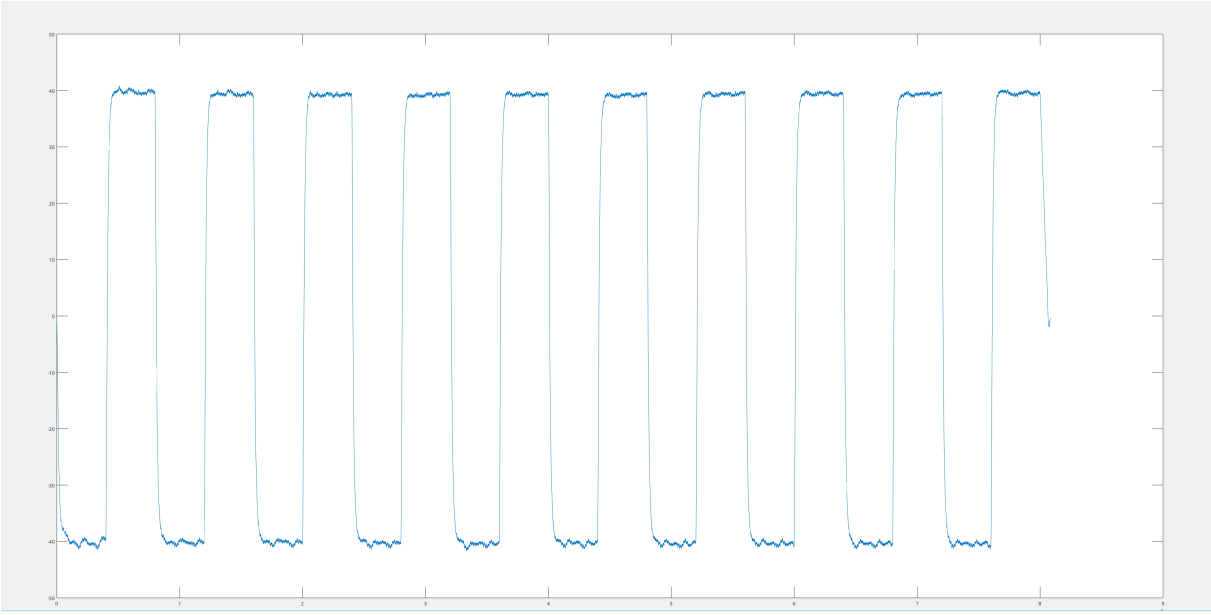


Figure 4.6: Time evolution of the angular velocity during the alternating 14 V square wave test. The positive and negative steady-state plateaus are clearly visible.

The plot is characterized by steep transient phases—occurring immediately after the voltage inversions—followed by flat steady-state regions (plateaus) where the motor operates at a constant speed. At steady state, the angular acceleration becomes zero ($\dot{\omega} \approx 0$). Consequently, the inertial term $J\dot{\omega}$ in the mechanical balance equation cancels out, meaning that the electromagnetic torque generated by the motor strictly balances the viscous friction forces.

To precisely extract the current (i) and velocity (ω) values needed to calculate b_{pos} and b_{neg}, the transient phases were ignored. The extraction focused exclusively on the steady-state plateaus. However, to mitigate the effect of the high-frequency measurement noise visible in the data, relying on a single sample would have been inaccurate.

Therefore, we computed an average value. Within the MATLAB workspace, the acquired data was stored in the `simout` matrix, where the first column (`simout(:,1)`) contains the armature current and the second column (`simout(:,2)`) contains the angular velocity. For each voltage level, the extraction was performed by isolating the final segment of the plateau (just before the subsequent voltage inversion) and calculating the mathematical mean of the last 500 samples of both columns.

This operation was made separately for the upper plateau (yielding $\bar{i}_{pos}$ and $\bar{\omega}_{pos}$) and the lower plateau (yielding $\bar{i}_{neg}$ and $\bar{\omega}_{neg}$). Finally, these clean, averaged steady-state values were substituted into the static balance equation to evaluate the asymmetric

friction components:

$$b_{pos} = \frac{K_m \cdot \bar{i}_{pos}}{\bar{\omega}_{pos}}, \quad b_{neg} = \frac{K_m \cdot \bar{i}_{neg}}{\bar{\omega}_{neg}} \tag{4.29}$$

Once the distinct friction values for both the positive and negative phases of each voltage test were calculated, they were utilized as experimental breakpoints for the interpolation. MATLAB's `pchip` (Piecewise Cubic Hermite Interpolating Polynomial) function was applied to build a continuous asymmetric friction profile $b(\omega)$, avoiding any artificial numerical overshoots between the extracted points. The resulting mapping is presented in Figure 4.7:

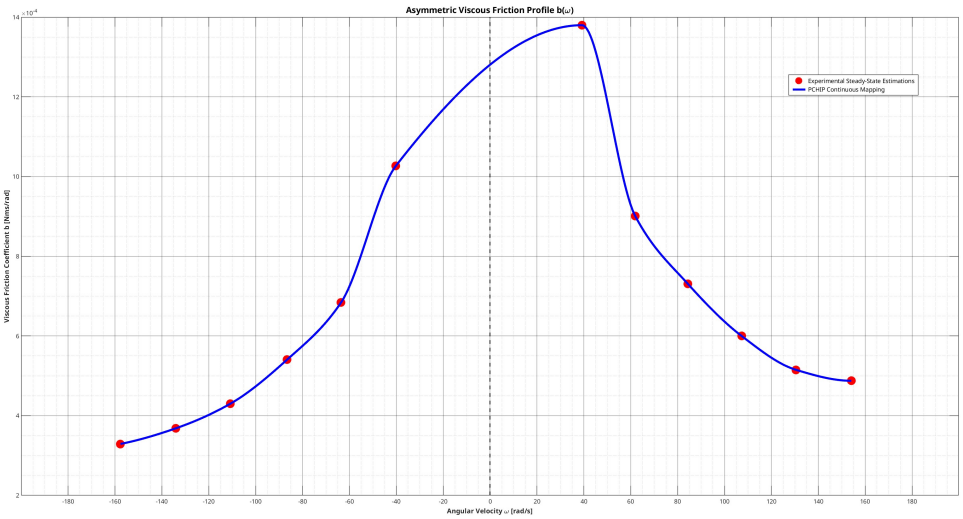


Figure 4.7: It's easy to notice the differences among the values of b for positive and negative ω.

It is important to note that, while the complete asymmetric mapping was derived to thoroughly characterize the motor and was implemented in the full non-linear Simulink validation, a simpler symmetric interpolation curve was employed during the subsequent synthesis of the PI controller. This simplification did not degrade the closed-loop performance because the friction variation between the two rotational directions is relatively small. Meaning that this simplification is compensated by the PI regulator itself. In fact, the integrator absorbs the asymmetric friction difference as a disturbance, still ensuring zero steady-state error.

4.5.4 Simulink transposition of the interpolated value of b

To implement a non-linear, velocity-dependent friction behavior in the Simulink model, MATLAB's `pchip` (Piecewise Cubic Hermite Interpolating Polynomial) function was applied to the extracted experimental points. `pchip` was chosen over standard splines to generate a shape-preserving continuous mapping $b(\omega)$ without introducing artificial, non-physical numerical overshoots between the experimental breakpoints.

The continuous friction profile obtained via the `pchip` interpolation is directly integrated into the Velocity Control Loop within the Simulink environment. This is achieved by employing a 1-D Lookup Table block, which replaces the classic constant parameter b.

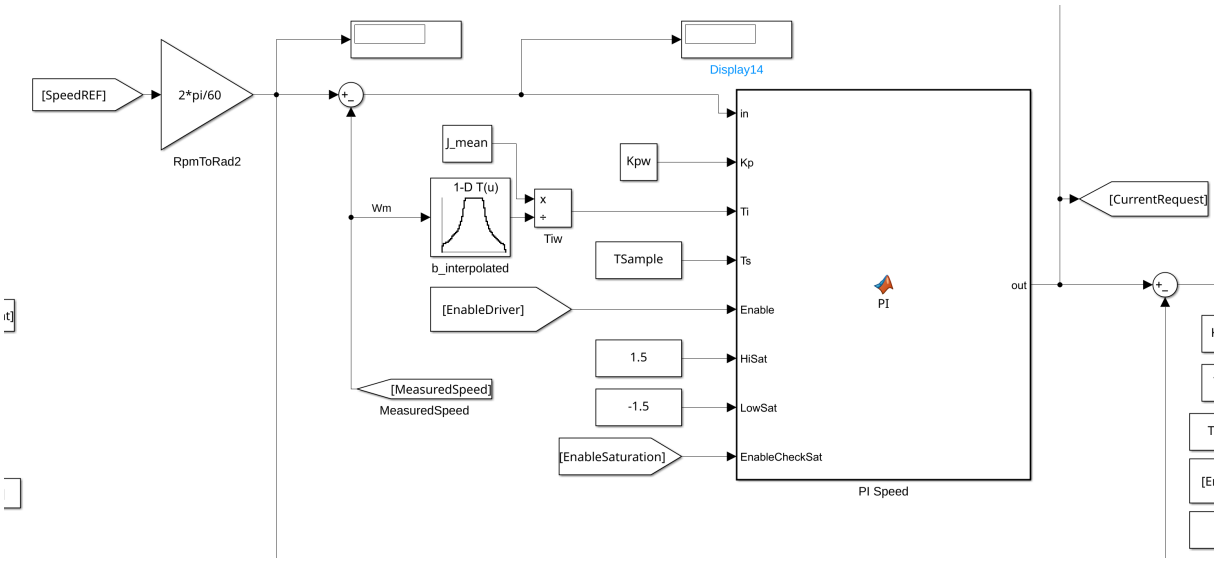


Figure 4.8: 1-D Lookup table implemented in the speed loop.

The dense vector `omega_fit` is assigned as the block's breakpoints vector, while `b_fit` defines the corresponding table data. During execution, the 1-D Lookup Table receives the instantaneous measured angular velocity $\omega(k)$ as an input and performs a real-time interpolation to output the current viscous friction coefficient.

Furthermore, this estimated parameter is used to perform **real-time Gain Scheduling** on the speed controller. The interpolated friction $b(\omega)$ is continuously divided by the equivalent inertia J_{mean} to dynamically calculate the local mechanical time constant, which is then used to update the integral time constant of the PI controller:

$$T_{i\omega}(\omega) = \frac{J_{mean}}{b(\omega)} \tag{4.30}$$

It is important to note the theoretical implications of this adaptive implementation. In a strictly Linear Time-Invariant (LTI) framework, setting $T_{i\omega} = \tau_m$ achieves exact pole-zero cancellation. However, because $b(\omega)$ varies continuously with the system's state (the velocity), the closed-loop system is structurally non-linear and is best described as a **Linear Parameter-Varying (LPV)** system. In such systems, continuous algebraic pole-zero cancellation is not mathematically guaranteed, and standard LTI stability proofs do not strictly apply.

Nevertheless, this Gain Scheduling strategy provides a significant practical improvement: it locally aligns the controller's dynamics with the plant's instantaneous mechanical time constant, drastically reducing the compensation effort required from the feedback loop. The residual dynamic mismatch, along with unmodeled non-linearities (such as static friction or rapid changes in velocity), is then effectively handled by the **integral action** of the PI controller. This synergy between the adaptive, feedforward-like scheduling and the robust integral feedback ensures stable tracking and zero steady-state error across the entire operating range of the motor, avoiding the instability risks of a naive pole cancellation.

4.6 Validation of the Estimated Parameters

To verify the correctness and accuracy of the identification procedure, a validation phase was performed within the Simulink environment. The core validation strategy consists of benchmarking the real physical system's responses against two separate simulation models: one parameterized with the nominal manufacturer datasheet values, and the other with the newly estimated parameters (R_{estim}, L_{estim}, J_{estim}, and the $b(\omega)$ mapping).

The validation process addresses the electrical and mechanical subsystems independently, ensuring that both time constants and steady-state behaviors are correctly captured.

4.6.1 Electrical Subsystem Validation

For the electrical validation, a voltage step input was applied to the motor model under blocked-rotor conditions. The resulting armature current transient collected from the real system was compared directly with the outputs of the nominal and identified Simulink models (`LR_model.slx`).

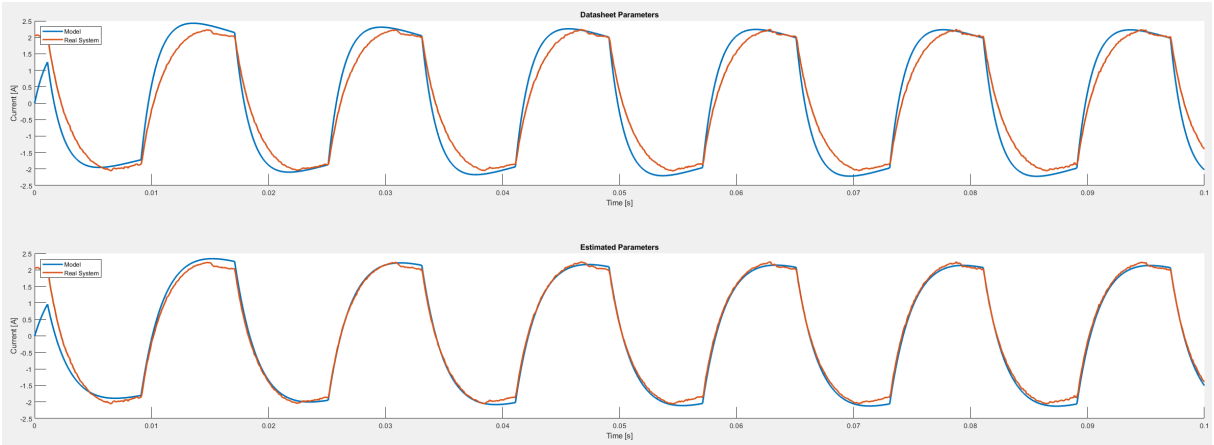


Figure 4.9: Datasheet Vs Estimated R-L parameters

The nominal datasheet parameters ($R = 3.2\ \Omega, L = 4.1$ mH) failed to accurately track the real current dynamics, exhibiting a faster electrical time constant ($\tau_e = L/R$) and a mismatch in the steady-state current. Conversely, the simulation utilizing the estimated parameters ($R_{estim} = 3.1532\ \Omega, L_{estim} = 6.1$ mH) showed a near-perfect overlap with the real acquired current transient. This successful match confirms that the Least Squares algorithm precisely captured the actual electrical properties of the armature circuit, effectively accounting for physical non-idealities and winding temperature variations.

4.6.2 Mechanical Subsystem Validation

A similar validation approach was applied to the mechanical subsystem using the `Jb_model.slx` file. The motor was excited with a square wave voltage input to analyze both the velocity transient and its steady-state behavior. The measured angular velocity $\omega(t)$ was superimposed onto the outputs generated by the nominal and estimated models.

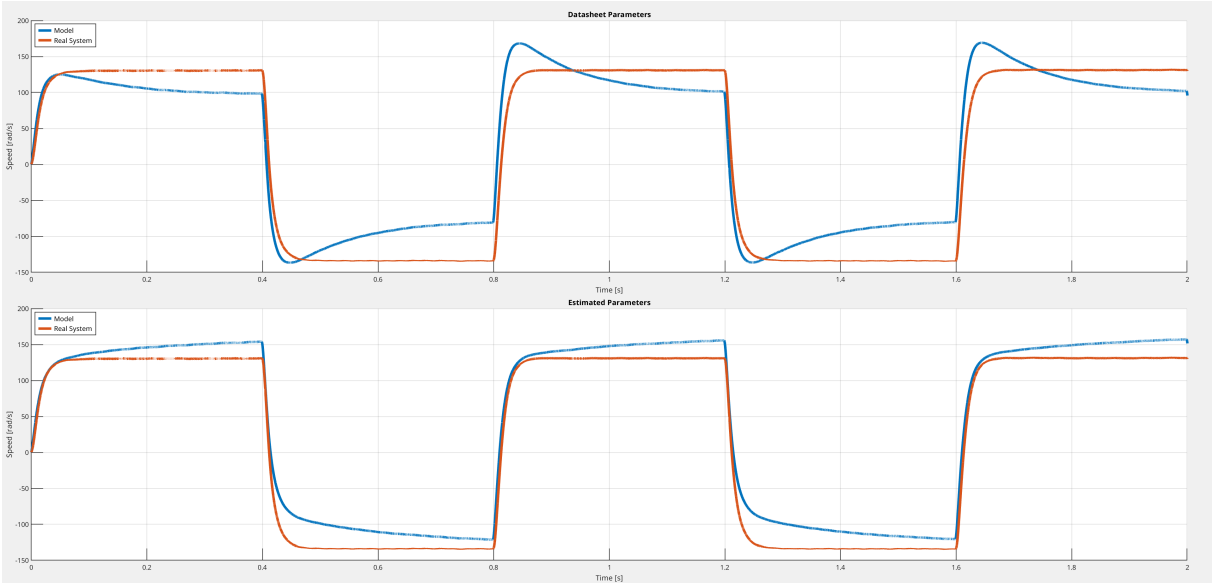


Figure 4.10: Mechanical validation: comparison of the velocity response between the real system, datasheet model, and identified model (38V).

The validation results showed a significant improvement when using the identified parameters. The nominal model, used the manufacturer's rotor inertia and a nominally assumed friction coefficient. This assumptions led to a bad tracking performance, both the transient slope—dictated by the mechanical time constant $\tau_m = J/b$—and the maximum velocity reached at steady state.

On the other hand, the model employing the parameters estimated via the Least Squares algorithm (J_{estim} and the globally averaged constant b_{estim}) behaved much better. By utilizing these experimentally extracted values, the simulated velocity curves reproduced much closely both the correct mechanical time constant and the steady-state plateaus. This result validates the physical correctness of the Least Squares identification procedure, proving that the estimated constant parameters provide a highly reliable baseline approximation of the motor's actual mechanical dynamics.

4.7 PI Controller Designed with Estimated Parameters

To guarantee optimal performance and stability, the parameters of the Proportional-Integral (PI) controllers were analytically synthesized using the Pole-Zero Cancellation (Pole Placement) method. The controller tuning was entirely based on the previously estimated parameters ($R_{estim}, L_{estim}, J_{estim}, b(\omega)$), ensuring that the mathematical compensators perfectly matched the real physical plant.

4.7.1 Inner Current Loop Design

The electrical subsystem acts as a first-order low-pass filter with a pole located at $-R/L$. To govern the current dynamics, a PI controller $C_i(s)$ was designed with the transfer function:

$$C_i(s) = K_{pi}\frac{1 + sT_{ii}}{sT_{ii}} \tag{4.31}$$

To achieve complete cancellation of the electrical pole, the integral time constant of the controller was set equal to the electrical time constant of the motor:

$$T_{ii} = \frac{L_{estim}}{R_{estim}} \tag{4.32}$$

By cancelling the plant's pole, the closed-loop transfer function of the inner loop is reduced to a first-order system with a dominant time constant τ_i. To ensure a highly reactive torque response, the target time constant was chosen as $\tau_i = 1$ ms. So, the proportional gain was computed as:

$$K_{pi} = \frac{L_{estim}}{\tau_i} \tag{4.33}$$

Using the estimated values ($R = 3.1532\ \Omega, L = 0.0061$ H), the exact control gains for the current loop were synthesized and implemented in the Simulink discrete-time environment.

4.7.2 Outer Speed Loop Design

The outer control loop is responsible for tracking the speed reference ω_{ref} by generating the appropriate current reference for the inner loop. According to the cascade control design rules, to avoid dynamic interference between the two loops, the outer loop must be significantly slower than the inner one. Therefore, the target mechanical time constant was chosen to be ten times slower: $\tau_w = 10 \cdot \tau_i = 10$ ms.

Similarly to the electrical subsystem, the mechanical subsystem presents a pole at $-b/J$. The speed PI controller $C_w(s)$ was designed to cancel this dynamic:

$$C_w(s) = K_{pw}\frac{1 + sT_{iw}}{sT_{iw}} \tag{4.34}$$

For the analytical synthesis of the speed controller gains, an average mechanical time constant $\tau_m = 85$ ms was assumed as a baseline, yielding the integral time constant:

$$T_{iw} = \tau_m \tag{4.35}$$

Consequently, the proportional gain to achieve the desired closed-loop bandwidth (τ_w) was calculated as:

$$K_{pw} = \frac{J_{estim}}{\tau_w \cdot K_m} \tag{4.36}$$

It is important to emphasize that while a nominal linearized mechanical time constant (τ_m) was required to analytically calculate the constant K_{pw} and K_{iw} gains (since the friction b is non-linear, meaning that also the mechanical time constant $\tau_m = J/b$ varies continuously), the actual physical plant in the Simulink environment continued to utilize the highly accurate, non-linear $b(\omega)$ PCHIP Lookup Table to better simulate the actual behavior of the motor. Moreover, an anti-windup saturation mechanism was enabled on the integral action to prevent the controller from diverging when the required voltage hit the physical limits of the power drive.

4.7.3 Performance Evaluation with Fixed Estimated Friction

In order to evaluate the improvements brought by the previous estimations, the closed-loop performance was first analyzed by using a fixed value of the estimated friction coefficient (b_{estim}) for the mechanical plant. That value was the one correspondent to a 1000 RPM of nominal speed. This value of speed is also the one used as speed step reference supplied to the system to analyze the transient response and the disturbance rejection capabilities.

No-Load Condition

The first test was conducted under ideal no-load conditions. Figure 4.11 illustrates the step response of the system controlled by the PI tuned with nominal datasheet parameters versus the PI tuned with the experimentally estimated parameters.

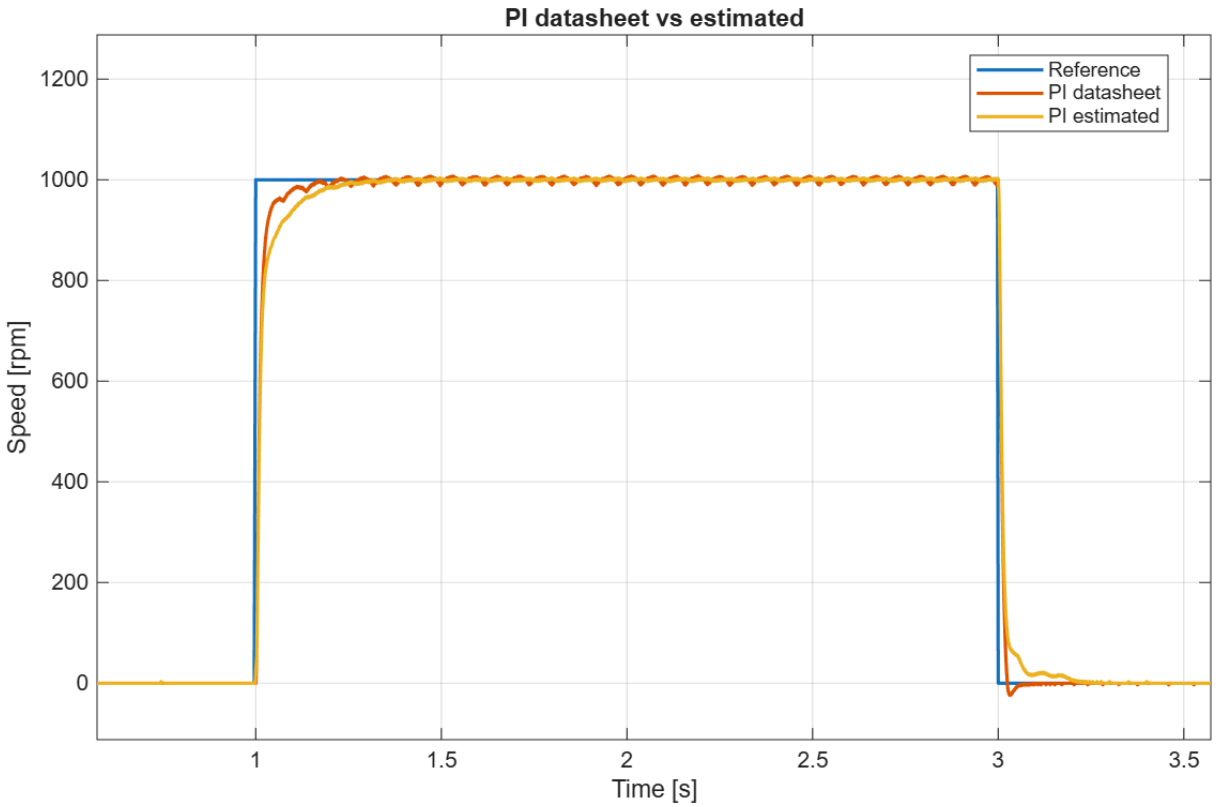


Figure 4.11: Step response at 1000 RPM under no-load conditions. Comparison between the PI controller designed with datasheet parameters and the one designed with estimated parameters.

The comparison clearly shows the better performance obtained through the parameter identification process. The system controlled with the datasheet parameters presents a more unstable response, with visible oscillations both during the transient and the steady state phases due to the wrong values of inertia and friction used for the tuning. On the other hand, the PI synthesized with the estimated parameters guarantees a much smoother and stable curve. It strictly respects the imposed closed-loop speed ($\tau_w = 10$ ms) with much less unwanted vibrations, providing a fast and clean response with almost zero steady-state error.

Load Condition

To evaluate the robustness of the control loops, a constant mechanical load disturbance was subsequently applied to the motor shaft. Figure 4.12 shows how the two identically structured controllers react to the external load.

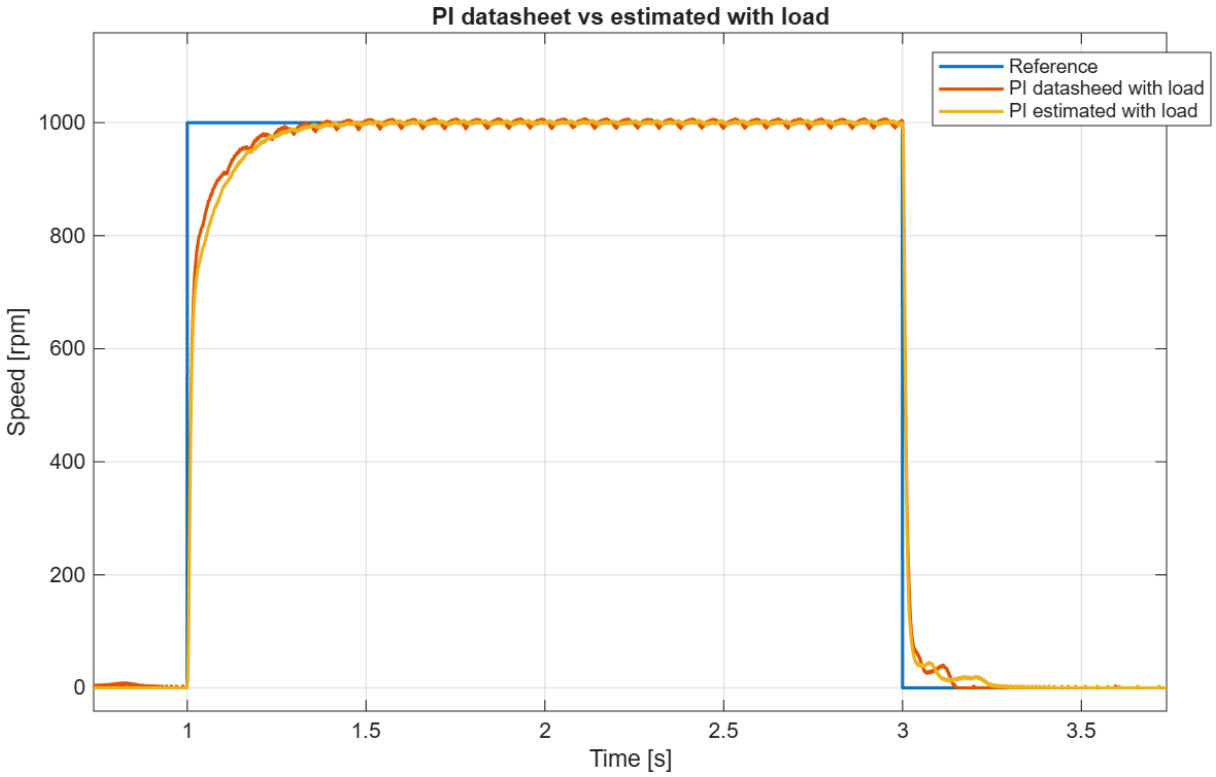


Figure 4.12: Step response at 1000 RPM with an applied load torque. The estimated PI controller shows cleaner dynamic and disturbance rejection.

Once again, the estimated PI controller shows better results than the nominal controller. Thanks to the more precise pole-zero cancellation achieved in the inner current loop using L_{estim} and R_{estim}, the drive handles the external torque demand with a high stability and robustness, reducing the system's oscillations.

Robustness Analysis: Load vs. No-Load

Finally, to have a better visualizations of both of the previous cases, the response of the solely estimated controller was plotted in both conditions (no-load and load) in Figure 4.13.

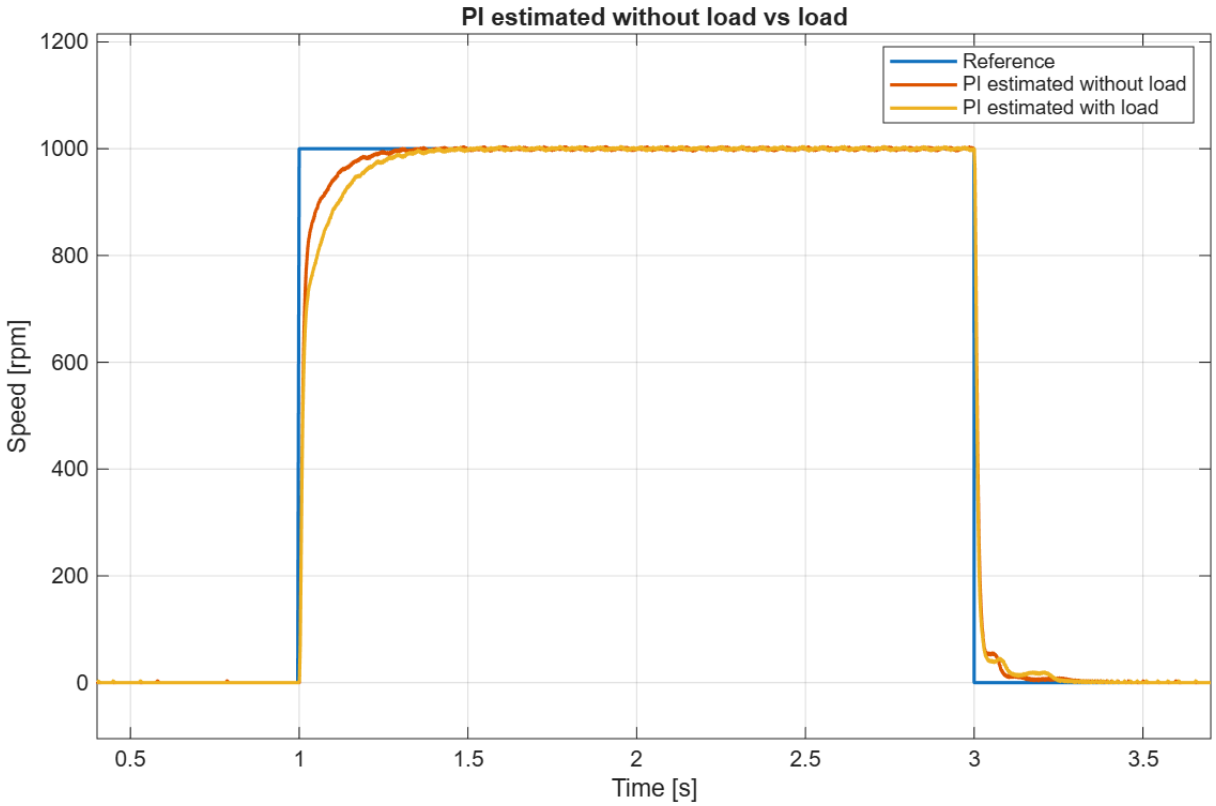


Figure 4.13: Performance consistency of the estimated PI controller: comparison between the unloaded and loaded scenarios.

This graph compares the system's behavior with and without the external load. As shown, the only noticeable difference is that the motor takes slightly longer to reach the steady-state speed when the load is applied. Despite this small delay, the implemented controller proves to be very robust. The tracking performance remains good, and the speed response is smooth, keeping the oscillations highly reduced in both scenarios.

4.7.4 Performance Evaluation with Interpolated Friction

In the previous section, the system was tested using a fixed average friction. To perform a more realistic evaluation, the Simulink plant was updated to use the fully interpolated non-linear friction mapping $b(\omega)$.

Because the friction value changes continuously depending on the motor's speed, a single step response is not sufficient to evaluate the overall system behavior. Therefore, a stair-step speed reference signal was commanded to the system. The trajectory consists of multiple speed steps, increasing by 500 RPM each time, from 0 up to a final speed of 1500 RPM. This specific profile forces the motor to operate across different velocity ranges, allowing us to evaluate the PI controller's ability to handle the variable friction $b(\omega)$ at different operating points.

Figure 4.14 presents the overall speed response of the system following the complete stair trajectory.

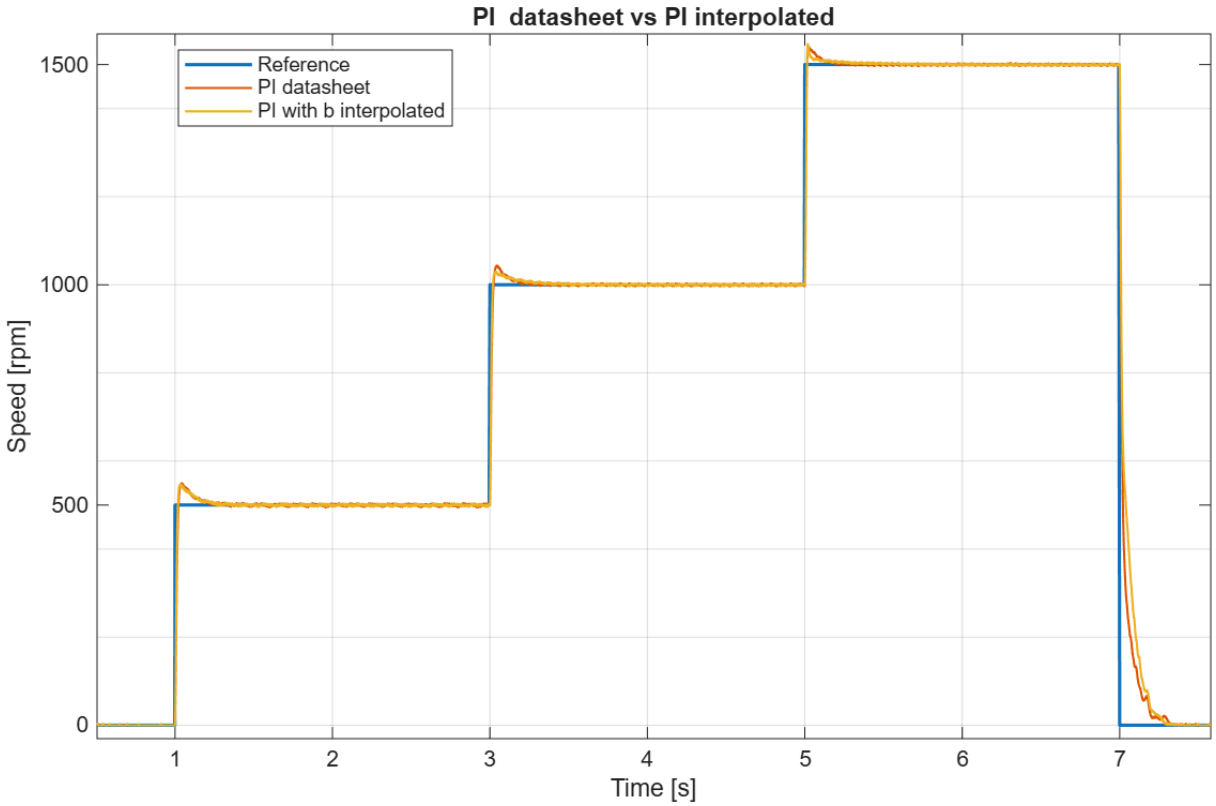


Figure 4.14: Overall closed-loop speed response using a stair-step reference signal (0 to 1500 RPM). The plant model utilizes the non-linear interpolated friction $b(\omega)$.

To better appreciate the subtle effects caused by the changing friction coefficient during the different phases of the test, the following figures present a zoomed-in view of each individual 500 RPM increment. As the speed increases, the value of $b(\omega)$ changes, slightly modifying the mechanical dynamics of the motor. These zoomed plots are intended to show how the estimated controller handles the transient response at these different friction levels.

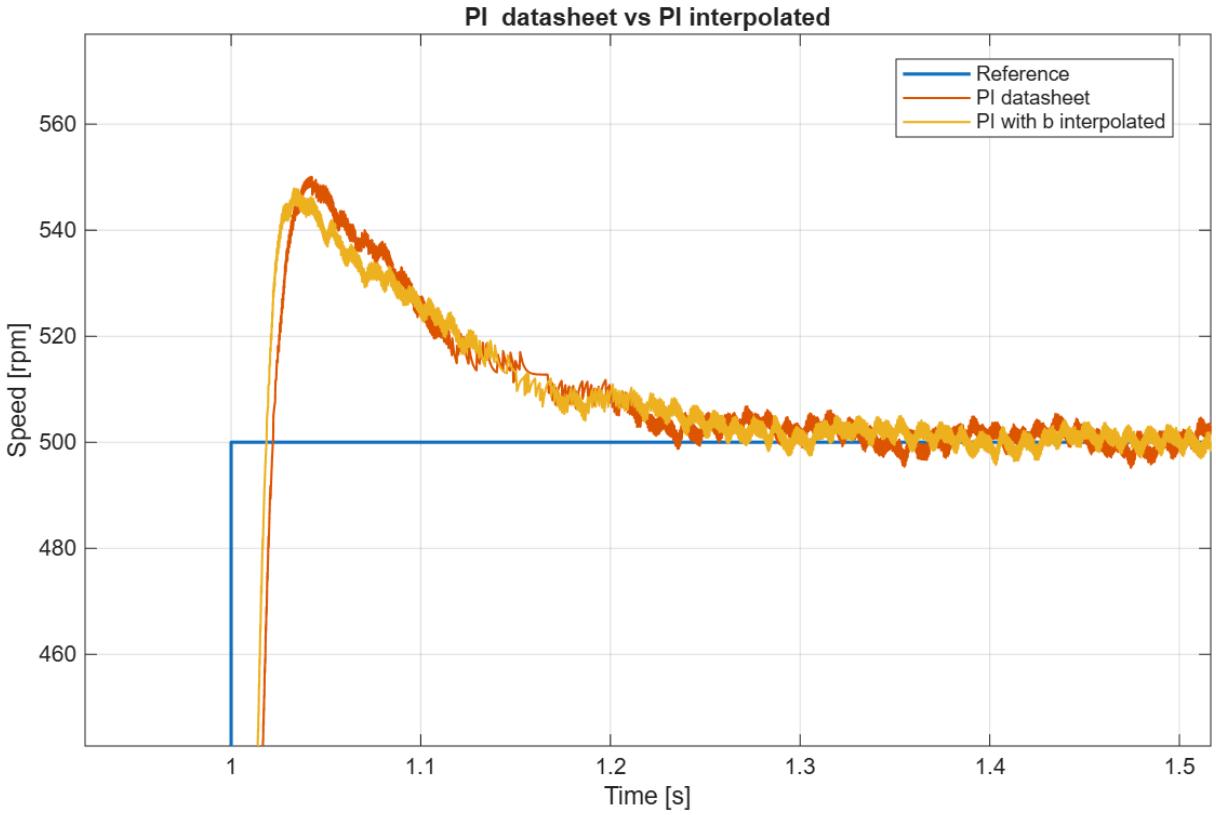


Figure 4.15: Zoomed view of the first step: acceleration from 0 to 500 RPM.

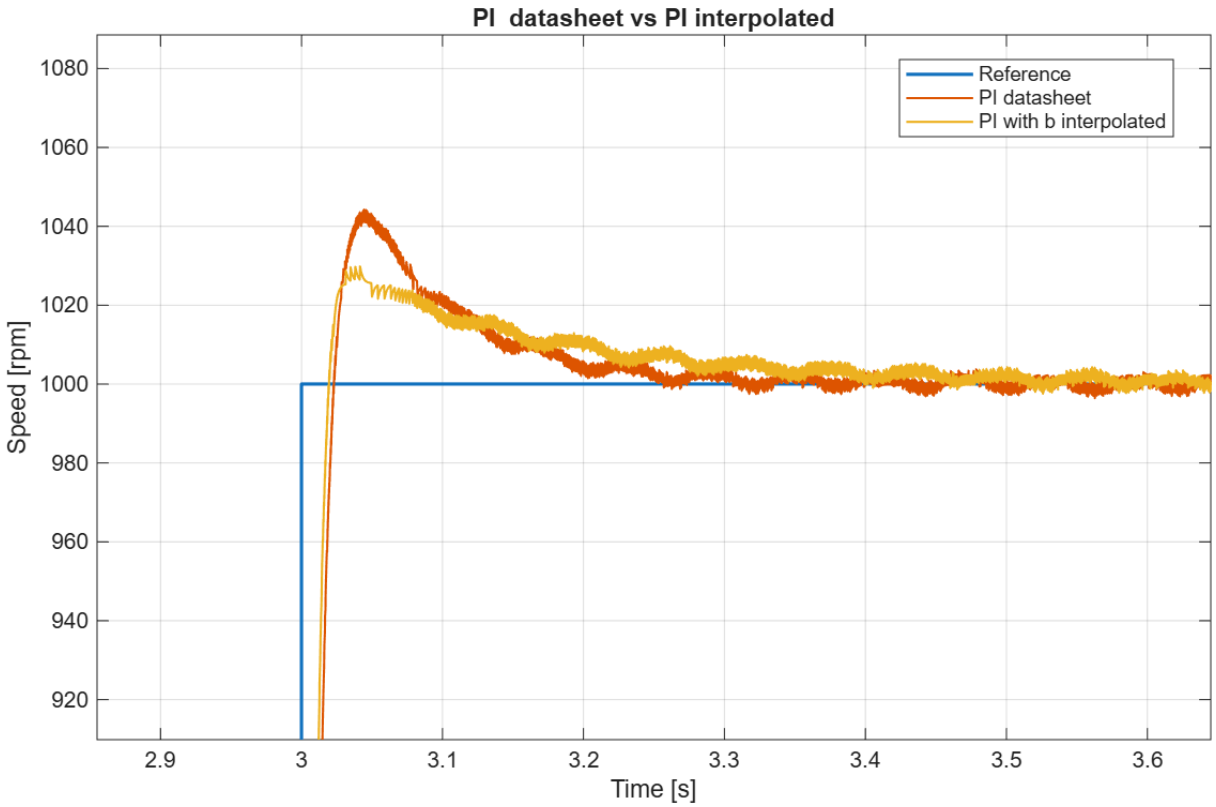


Figure 4.16: Zoomed view of the second step: acceleration from 500 to 1000 RPM.

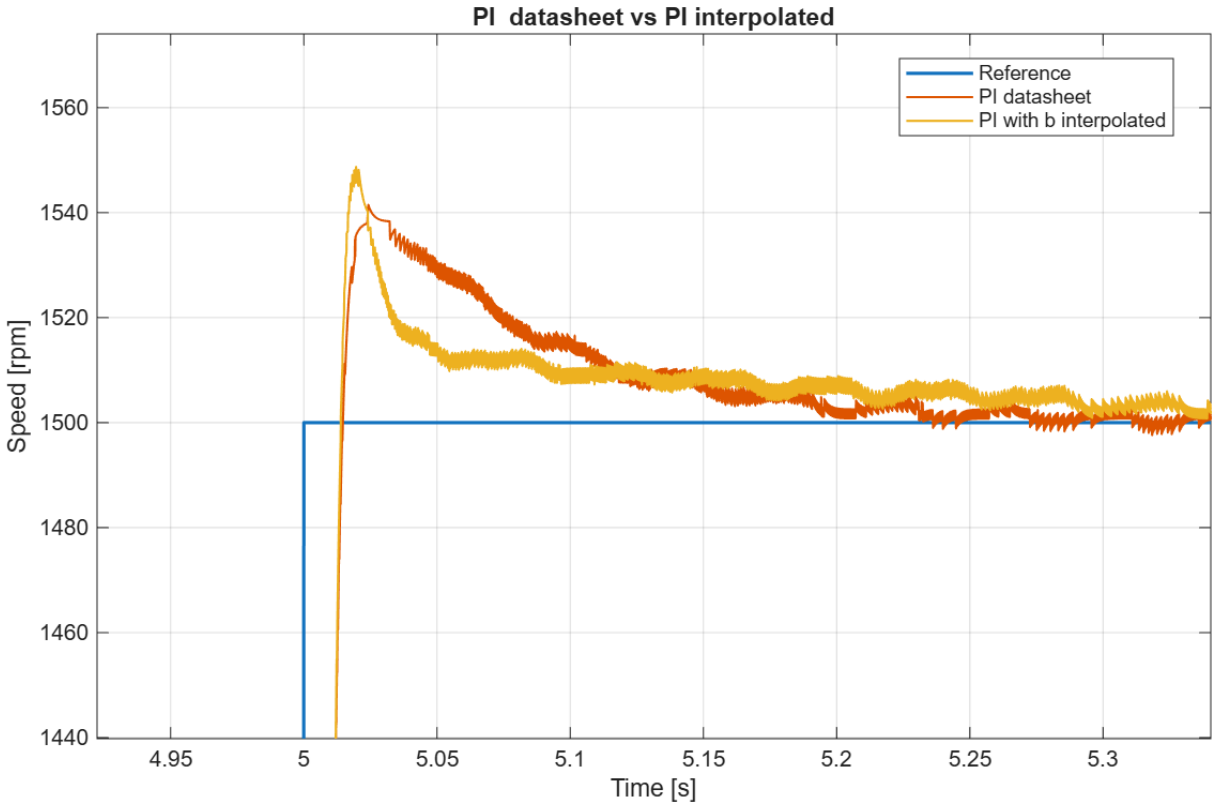


Figure 4.17: Zoomed view of the third step: acceleration from 1000 to 1500 RPM.

4.8 Comparison Between Nominal and Estimated Parameters

While the dynamic and transient differences between the nominal and estimated models were analyzed in the validation section, here it's provided a direct numerical summary of the identified physical quantities. Table 4.1 highlights the discrepancies between the initially considered parameters and the actual parameters of the physical motor utilized in the laboratory.

Table 4.1: Summary of nominal vs. experimentally estimated parameters.

Parameter	**Nominal**	**Estimated (Least Squares)**
Armature Resistance R	3.2 Ω	3.1532 Ω
Armature Inductance L	4.1 mH	6.1 mH
Equivalent Inertia J	7.04×10^{-5} kg m^2	8.127×10^{-5} kg m^2
Viscous Friction b	6.65×10^{-4} Nms/rad	Non-linear asymmetric $b(\omega)$

4.9 Improvements Obtained on the Real System

The parameter estimation process significantly improved the overall control and performance of the real DC motor.

Relying exclusively on the nominal datasheet parameters to tune the controllers would have led to poor and potentially unstable performances. This is particularly true for the

inner current loop, since the experimentally estimated inductance (L_{estim}) was approximately 50% higher than the nominal one. By utilizing the identified R_{estim} and L_{estim} values, the pole-zero cancellation was highly accurate. This guaranteed a fast and stable electrical response with a 1 ms time constant, preventing dangerous current overshoots that could have triggered the hardware protections of the drive.

Regarding the mechanical subsystem, the identification procedure provided a much deeper understanding of the motor's real dynamics. Mapping the non-linear and asymmetric friction into the PCHIP curve $b(\omega)$ allowed the creation of a highly realistic simulation model. At the same time, using the correctly estimated equivalent inertia (J_{estim}) provided a solid and reliable baseline to design the speed loop.

When applied to the real system, the PI controller tuned with these experimental parameters performed much better.The robust tuning of the overall PI controller effectively managed the unmodeled static friction and the asymmetric viscous drag. As a result, the closed-loop system achieved fast and smooth reference tracking ($\tau_w = 10$ ms), robust disturbance rejection under varying load conditions, and an almost zero steady-state error.

Chapter 5

Advanced Discrete-Time Controllers

5.1 Introduction to Discrete-Time Control

In the previous chapters, the digital control of the DC motor was achieved by first designing a continuous-time PI controller and then discretizing it using approximations (e.g., the Tustin method). While effective, this indirect approach introduces mapping errors and performance degradation, especially when pushing the system toward higher bandwidths.

To overcome these limitations and fully exploit the digital nature of the microcontroller, this chapter focuses on direct discrete-time synthesis. By discretizing the plant model first, the control algorithms can be designed directly in the z-domain. This allows for the implementation of advanced analytical strategies, such as Deadbeat and Dahlin controllers, which can achieve precise and highly aggressive transient responses that are theoretically impossible with standard continuous-time methods.

Furthermore, to provide these highly responsive discrete-time controllers with a wider operational margin during the experimental validation, the DC-link voltage of the power supply was increased to $V_{sat} = 32$ V (compared to the 24 V used in previous tests). This higher saturation limit allows the actuators to express their fast dynamics more effectively. However, it also makes the implementation of robust anti-windup schemes strictly necessary to safely manage the aggressive control efforts generated by these algorithms.

5.2 Deadbeat Controller

5.2.1 Theoretical Principle of Deadbeat Control

Considering a digital controller with a one-degree-of-freedom configuration, the deadbeat design aims to impose specific, highly aggressive properties on the closed-loop output signal $Y(z)$. Specifically, when the system is subjected to a standard reference input $V(z)$ (such as a step, ramp, or parabola), the deadbeat controller forces the output to reach its steady-state final value in the minimum possible number of sampling periods, ensuring a null steady-state error and avoiding any inter-sample oscillations (ripple).

Assuming a discrete-time plant transfer function $G_p(z)$, the closed-loop transfer function $G_m(z)$ is defined as:

$$G_m(z) = \frac{D(z)G_p(z)}{1 + D(z)G_p(z)} \tag{5.1}$$

A simplified deadbeat design procedure can be formulated based on the following assumptions:

- The reference input is a step function.
- The plant $G_p(z)$ is stable and minimum phase, meaning its dynamics can be safely canceled by the controller without causing instability.
- The desired closed-loop transfer function is imposed as $G_m(z) = z^{-k_d}$, meaning the output will perfectly track the input after k_d sampling periods.

For physical realizability (causality), the delay parameter k_d must be greater than or equal to the relative degree of the plant r_p (i.e., $k_d \geq r_p$). Under these constraints, the analytical expression of the deadbeat controller $D(z)$ is derived by inverting the closed-loop relationship:

$$D(z) = \frac{1}{G_p(z)} \frac{z^{-k_d}}{1 - z^{-k_d}} \tag{5.2}$$

Since controllers are implemented on digital microcontrollers, this approach is extremely advantageous as it allows the controller to be synthesized directly in the discrete-time domain, entirely skipping the discretization process of a continuous-time equivalent and avoiding the introduction of further mapping errors.

5.2.2 Plant Discretization and Controller Design

To achieve the highest possible accuracy in the plant inversion process, the continuous-time transfer function of the plant, $G_p(s)$, was built utilizing the parameters estimated via the Least Squares algorithm (as detailed in Chapter 4) at an operating speed of 1000 rpm.

The estimated electro-mechanical parameters used for the synthesis are:

$$R = 3.153\ \Omega, \quad L = 0.0061\ \text{H}, \quad J = 8.103 \cdot 10^{-5}\ \text{kg} \cdot \text{m}^2, \quad K_m = 0.11\ \text{Nm/A} \tag{5.3}$$

Given the negligible magnitude of the viscous friction coefficient b, this term was omitted ($b \approx 0$) during the synthesis phase. In fact, considering the numerical values, the mechanical pole associated with the viscous friction is orders of magnitude slower than the electrical dynamics and the control loop bandwidth. Therefore, omitting it simplifies the mathematical model without compromising the validity of the control law. The transfer function linking the requested voltage to the motor speed is therefore:

$$G_{mot}(s) = \frac{K_m}{LJs^2 + RJs + K_m^2} \tag{5.4}$$

In our experimental setup, the control action generated by the algorithm is a normalized duty-cycle signal sent to the PWM. To account for the voltage gain provided by the power amplifier, the transfer function is multiplied by the DC-link voltage ($V_{dc} = 32$ V), yielding $G_p(s) = 32 \cdot G_{mot}(s)$.

The plant was then discretized using a sampling time $T_s = 0.1$ ms with a Zero-Order Hold (ZOH) method. Since the continuous-time transfer function of the motor $G_{mot}(s)$ is strictly proper (featuring zero zeros and two poles), its exact ZOH discretization naturally yields a discrete-time model $G_p(z)$ with a relative degree $r_p = 1$. This intrinsic characteristic mathematically represents the minimum causal delay introduced by the digital nature of the system, as the actuator requires exactly one sampling period to react.

Following the deadbeat synthesis formula (5.2), the desired closed-loop delay was set to perfectly match the plant's relative degree by choosing the minimum realizable

value, $k_d = 1$. This strictly guarantees the causality of the controller, since the required parameter k_d must be greater than or equal to the relative degree r_p of the discretized plant $G_p(z)$, which in our ZOH formulation is exactly 1. The controller $D(z)$ is thus computed as:

$$D(z) = \frac{1}{G_p(z)} \frac{z^{-1}}{1 - z^{-1}} \tag{5.5}$$

The resulting polynomial fraction was then reduced to its minimal realization using the `minreal()` MATLAB function to cancel out overlapping pole-zero pairs, yielding the nominal polynomials B_z (numerator) and A_z (denominator) that define the control law.

5.2.3 Anti-Windup Implementation

While the deadbeat controller theoretically guarantees an extremely fast response, its calculated control action $u(k)$ is typically very aggressive. When dealing with real actuators, the control variable is strictly bounded by the physical limits of the power supply. In our system, the maximum voltage applicable to the DC motor is the DC-link voltage $V_{sat} = 32$ V.

When a large reference step is applied, the high control effort computed by the deadbeat algorithm inevitably exceeds V_{sat}. During this saturation phase, the plant operates in an open-loop condition, and the controller's integral action continues to accumulate the tracking error uncontrollably. This phenomenon, known as *windup*, causes the internal states of the controller to grow disproportionately. Consequently, when the error changes sign, the control action takes a significant amount of time to discharge, leading to severe overshoot, prolonged settling times, and instability.

To mitigate this problem, an anti-windup scheme was incorporated directly into the controller's structure. Following the block diagram synthesis discussed in the course theory, the anti-windup mechanism relies on the definition of an arbitrary polynomial $F(z)$ that mathematically replaces the denominator $A(z)$ during the saturation regime. To ensure stability and a proper discharge of the accumulated error, $F(z)$ must satisfy the following constraints:

- It must be an asymptotically stable polynomial (all roots strictly inside the unit circle).
- Its degree must be greater than or equal to the degree of the controller's numerator $B_z(z)$ (i.e., $\deg\{F(z)\} \geq \deg\{B_z(z)\}$).
- The steady-state gain condition must be verified: $\frac{B_z(1)}{F(1)} \geq 0$.

Since the synthesized controller yields a numerator $B_z(z)$ of 2nd degree, the anti-windup polynomial $F(z)$ must be at least of 2nd degree to satisfy the algebraic constraint ($\deg\{F(z)\} \geq \deg\{B_z(z)\}$). To provide an extra degree of freedom in shaping the discharge dynamics, we opted for a 3rd-degree polynomial. After several tuning iterations to balance the discharge speed without triggering aggressive reactions upon exiting the saturation region, the chosen polynomial was:

$$F(z) = K_f \cdot z(z - 0.95)(z - 0.8) \tag{5.6}$$

with $K_f = 100$. This specific formulation ensures that $F(z)$ has strictly stable real poles close to the unit circle. This avoids high-frequency ringing, while the gain K_f provides a

sufficiently high steady-state value to rapidly discharge the accumulated error when the actuator leaves the saturation boundaries.

The implementation was carried out by extracting the coefficients of $F(z)$ and generating the corresponding feedforward polynomial $F_{feed}(z) = F(z) - A_z(z)$, which is fed back in the anti-windup loop only when the requested voltage exceeds the saturation limits.

5.2.4 Simulation Results: Control Effort and Ringing Effect

To validate the theoretical synthesis and demonstrate the absolute necessity of the anti-windup scheme, the controller was first tested in a Simulink environment. The focus of these preliminary simulations was to analyze the control effort (the requested voltage $u(k)$) generated by the deadbeat algorithm when subjected to a step reference.

As discussed, a pure deadbeat controller requires an immense initial control effort. Figure 5.1 shows the unconstrained control action (red curve) superimposed on the constrained one (blue curve).

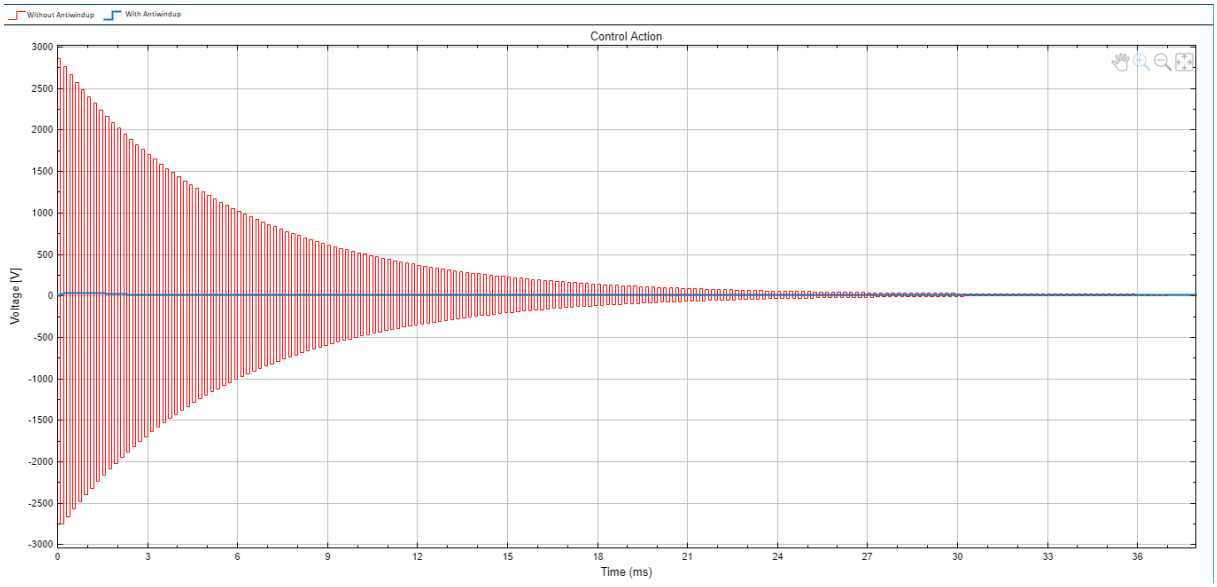


Figure 5.1: Simulated control action without saturation and anti-windup (red) vs with anti-windup (blue). The unconstrained effort exhibits severe high-frequency ringing.

The unconstrained requested voltage reaches physically unfeasible peaks of approximately ± 2700 V and exhibits severe high-frequency ringing, oscillating violently between positive and negative values before decaying. This severe oscillatory behavior is intrinsically linked to the discrete-time synthesis of the Deadbeat algorithm. By expanding the controller's transfer function $D(z)$ and applying the inverse Z-transform, the control law can be expressed as a difference equation in the time domain:

$$u(k) = a_1 u(k-1) + a_2 u(k-2) + b_0 e(k) - b_1 e(k-1) + b_2 e(k-2) \tag{5.7}$$

As typically occurs in Deadbeat controllers synthesized for electro-mechanical plants, the coefficients of the past error terms ($e(k-1)$, $e(k-2)$) exhibit alternating signs and large magnitudes. This implies that at every sampling instant, the controller aggressively reverses its numerical action to force the tracking error to zero in minimum time. This alternating mathematical sign physically translates into the violent high-frequency ringing observed in the control effort.

Applying a strict physical saturation limit without an appropriate discharge mechanism would result in the aforementioned windup phenomenon. Figure 5.2 illustrates the effectiveness of the implemented anti-windup polynomial $F(z)$ during the initial transient.

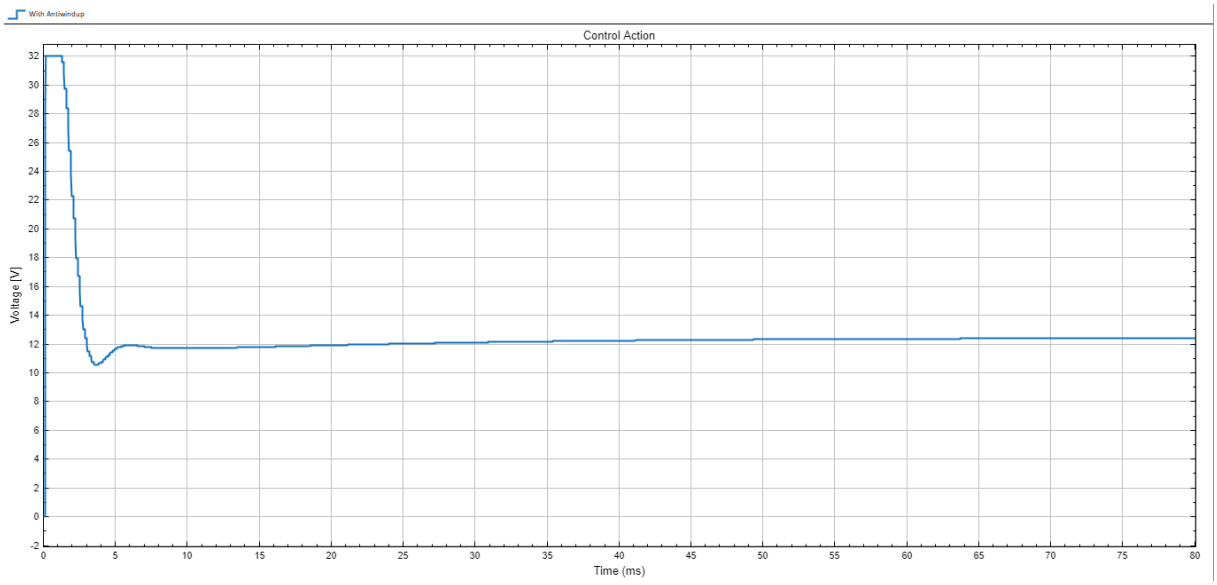


Figure 5.2: Detailed view of the simulated control action with the active anti-windup scheme and a 32 V saturation limit.

As evident from the detailed plot, the control action correctly saturates at the 32 V upper bound without accumulating unbounded integral error. As soon as the system dynamics catch up to the reference, the requested voltage smoothly decreases and settles around the steady-state value required to maintain the steady-state speed, completely suppressing the aggressive ringing effect.

To fully appreciate the impact of the anti-windup scheme on the system's dynamic performance, the speed response of the simulated model was analyzed. Figure 5.3 compares the response to a 100 rad/s step reference for the deadbeat controller with and without the anti-windup feedback.

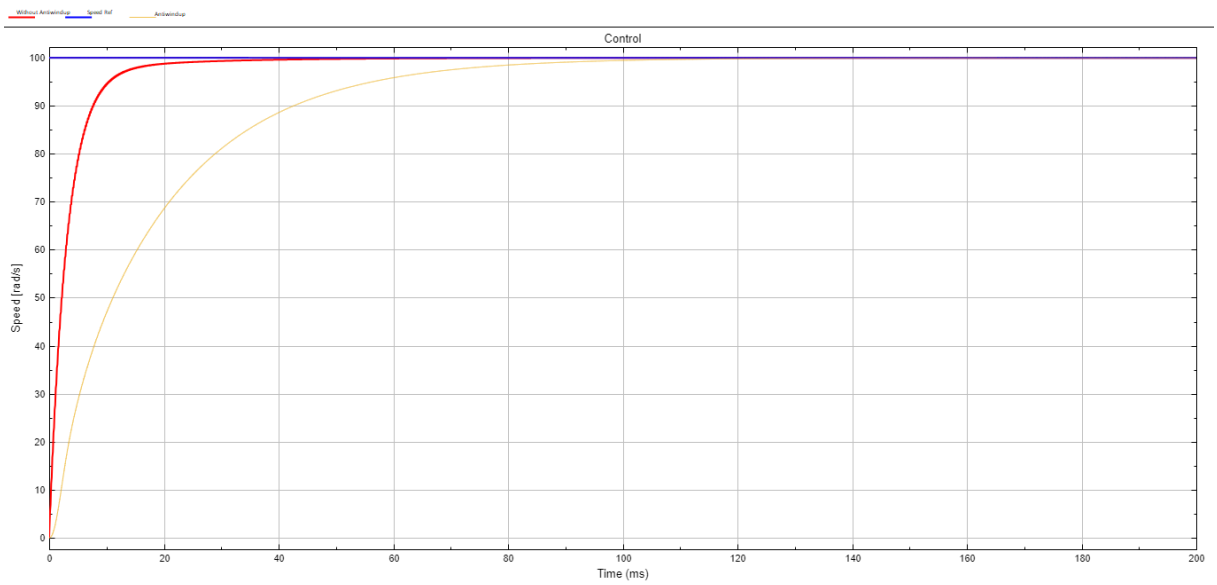


Figure 5.3: Simulated speed response to a 100 rad/s step reference: controller without anti-windup (red) vs. controller with anti-windup (yellow).

The dynamic behavior observed in the plot reveals a significant difference in performance.

Without the anti-windup scheme (red curve), the system exhibits the fastest possible response, reaching 100 rad/s in approximately 15 to 20 ms. Because the calculated control effort heavily exceeds the physical limits of the inverter, the output remains solidly saturated at the maximum available voltage (32 V). This effectively acts as a "bang-bang" control. However, this simulated speed trajectory is deceptive. It completely hides the severe windup phenomenon occurring inside the controller: during saturation, the internal states accumulate unbounded errors. In the real system, this unconstrained accumulation leads to severe high-frequency ringing in the control variable $U(z)$, overshoot, and long delays during deceleration.

Conversely, when the anti-windup scheme is active (yellow curve), the system takes considerably longer—over 100 ms—to approach the steady-state value. This slower response is necessary to ensure stability. When the controller hits the saturation boundary, the difference between the unconstrained and saturated control effort is fed back through the asymptotically stable polynomial $F(z)$. This bounds the internal states of the controller. Consequently, the trajectory is no longer a pure maximum acceleration, but a "softened" response governed by the roots of $F(z)$. While slower, this mechanism is strictly required to suppress internal ringing, avoid integrator windup, and guarantee stable control signals.

5.2.5 Experimental Results on the DC Motor

Once the controller synthesis and the anti-windup scheme were validated in simulation, the algorithm was deployed on the microcontroller to drive the real DC motor. The experimental tests were performed by applying a square wave speed reference (from 0 to 1000 rpm) to evaluate both the transient and steady-state behaviors. To thoroughly test the robustness of the control, the experiments were conducted under two different mechanical conditions: unloaded motor (WITHOUT Load) and motor subjected to an

active resisting torque obtained by connecting a braking resistor to the coupled generator (WITH Load).

Figure 5.4 illustrates the complete sequence of the step response, comparing the deadbeat controller equipped with the anti-windup scheme (blue line) against the standard formulation without anti-windup (red line).

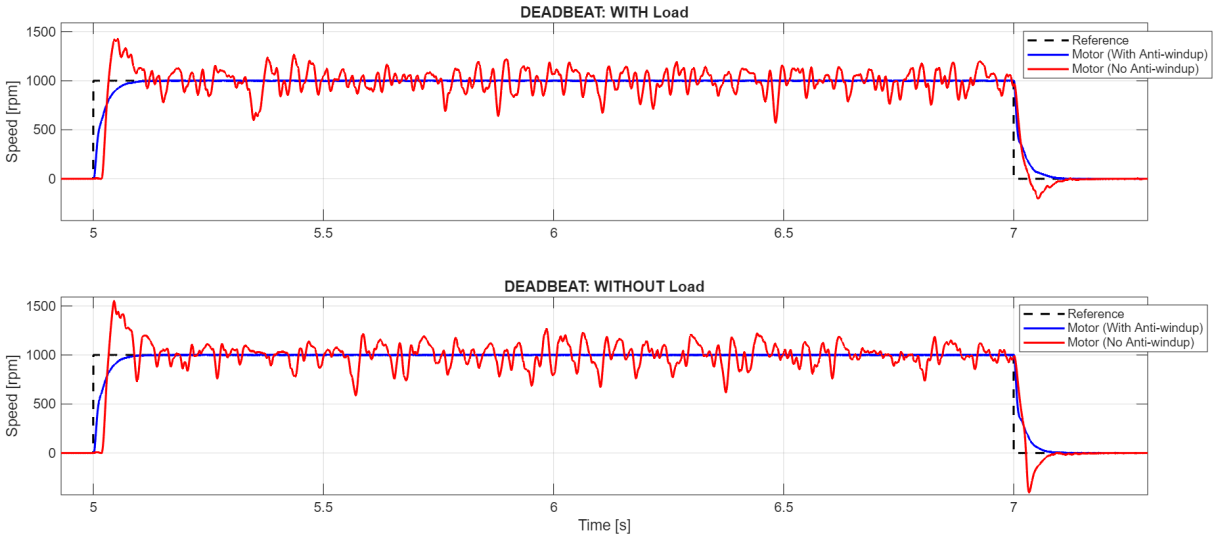


Figure 5.4: Experimental speed response to a sequence of steps ($0 - 1000$ rpm). Comparison between the implementation with (blue) and without (red) anti-windup, both loaded and unloaded.

The overall response clearly demonstrates that without the anti-windup filter, the system exhibits a highly oscillatory and erratic behavior, even during the steady-state phase at 1000 rpm. This is the mechanical manifestation of the control effort *ringing* observed in the simulations: the actuator is constantly hammered by a control signal alternating between positive and negative saturation, exacerbated by unmodeled physical nonlinearities such as the friction of the rubber joint. Conversely, the implementation with the $F(z)$ anti-windup polynomial (blue curve) completely suppresses these oscillations, ensuring a perfectly stable and flat steady-state tracking.

To better highlight the critical differences in the transient dynamics, detailed zoomed-in views of the rising and falling edges were extracted.

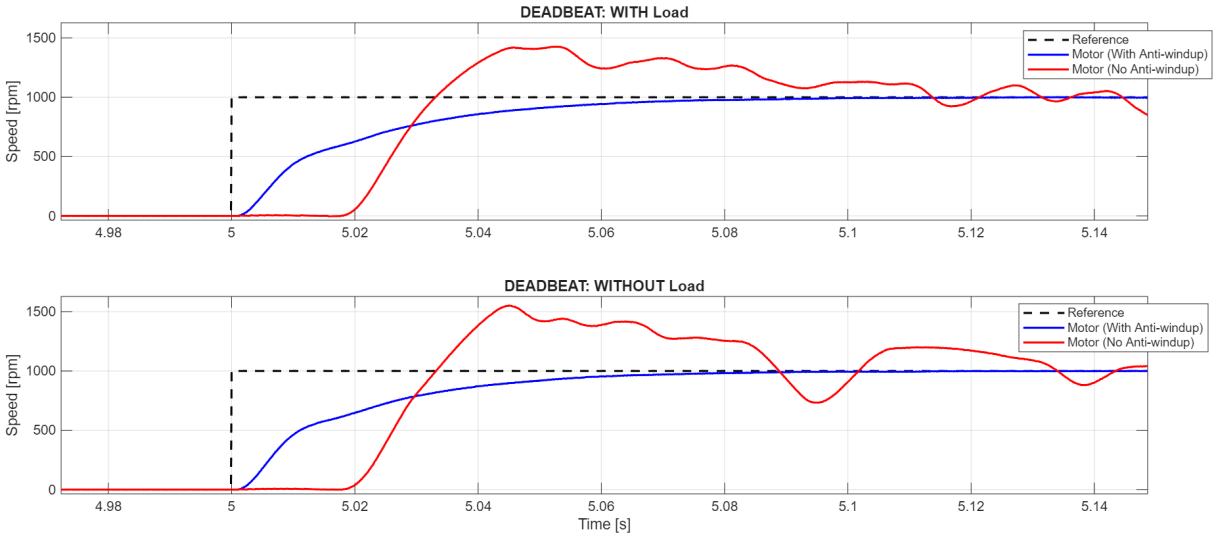


Figure 5.5: Detailed view of the rising edge. The absence of anti-windup (red) leads to severe initial delay and massive overshoot.

As shown in Figure 5.5 (Rising Edge), the absence of the anti-windup scheme severely degrades the acceleration phase. The red curve shows a noticeable dead-time before the motor begins to accelerate effectively, followed by a massive overshoot (reaching over 1500 rpm). This happens because the integral action charges excessively, forcing the actuator deep into saturation. On the other hand, the blue curve (with anti-windup) maintains a smooth, strictly controlled, and monotonic trajectory, completely avoiding the overshoot. As physically expected, the "WITH Load" response is slightly slower than the "WITHOUT Load" one due to the increased rotor inertia, but the controller handles both conditions flawlessly.

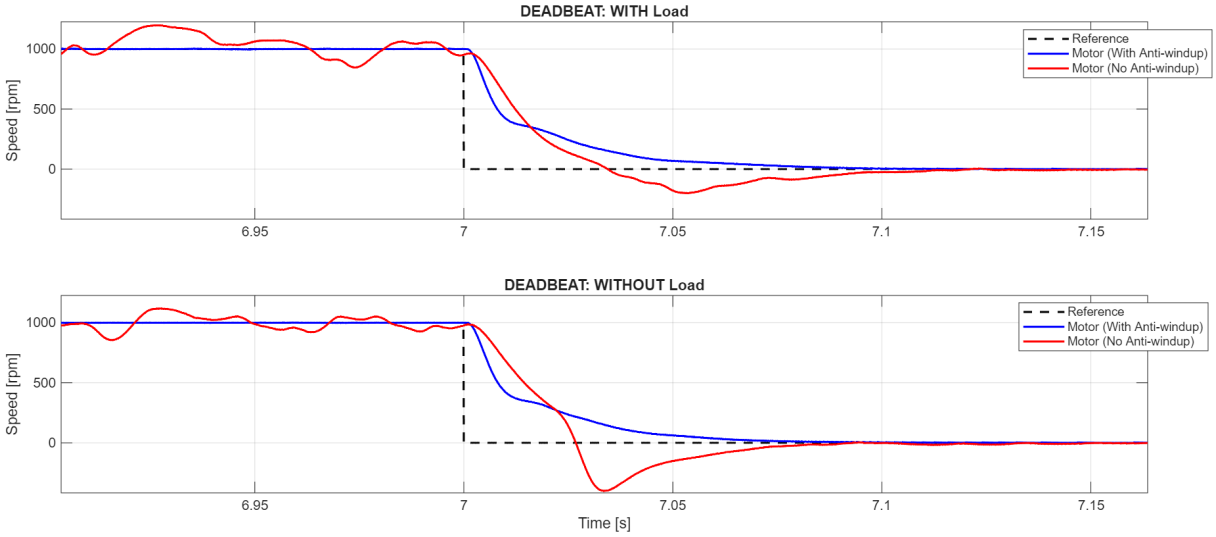


Figure 5.6: Detailed view of the falling edge. The integral windup discharge causes a severe undershoot (bump) in the uncompensated system (red).

Finally, Figure 5.6 (Falling Edge) provides the most striking evidence of the windup phenomenon. When the reference abruptly drops from 1000 rpm to 0, the standard implementation (red curve) holds a massive accumulated positive value in its internal integrator from the steady-state phase. When the error becomes negative, the integrator

takes a considerable amount of time to discharge, causing a severe delayed "bump" (a heavy undershoot below 0 rpm) before the motor finally settles.

Conversely, the implementation with the properly tuned $F(z)$ polynomial (blue curve) bounds the integral state during the saturated regime. As a result, as soon as the reference drops, the control action immediately reacts, leading the motor to a smooth and rapid deceleration without crossing the zero-speed axis. This proves the absolute necessity of the anti-windup network for deploying discrete-time analytical controllers in real-world applications.

5.3 Dahlin Controller

5.3.1 Theoretical Principle of Dahlin Control

While the Deadbeat controller aims to bring the output to the steady-state in the minimum possible number of time steps, this often requires an extremely aggressive control action, leading to severe actuator saturation and ringing phenomena. To overcome these limitations, the Dahlin control algorithm can be employed.

The Dahlin controller can be interpreted as a modification of the Deadbeat controller designed to produce a smoother, exponential response rather than an immediate step. According to the theory provided in the course, the design specification requires the closed-loop system to behave like a continuous-time first-order system with a delay θ. For a unit step input, the desired continuous-time output is:

$$Y(s) = \frac{e^{-\theta s}}{\lambda s + 1}\frac{1}{s} \tag{5.8}$$

where λ is the desired time constant of the first-order system, which acts as the main tuning parameter. A smaller λ results in a faster system response, but requires a larger control effort.

Moving to the discrete-time domain, the desired output becomes:

$$Y(z) = \frac{(1 - e^{-T/\lambda})z^{-N-1}}{(1 - z^{-1})(1 - e^{-T/\lambda}z^{-1})} \tag{5.9}$$

where T is the sampling time and $N = [\theta/T]$ represents the delay in terms of sampling periods.

By considering the closed-loop transfer function relationship $G_m(z) = Y(z)/V(z)$, the Dahlin discrete-time controller $D(z)$ can be synthesized analytically by inverting the plant dynamics $G_p(z)$:

$$D(z) = \frac{1}{G_p(z)}\frac{G_m(z)}{1 - Gm(z)} = \frac{1}{G_p(z)}\frac{(1 - e^{-T/\lambda})z^{-N-1}}{1 - e^{-T/\lambda}z^{-1} - (1 - e^{-T/\lambda})z^{-N-1}} \tag{5.10}$$

5.3.2 Plant Discretization, Controller Design and Tuning

The synthesis of the Dahlin controller was performed directly in the discrete-time domain using MATLAB. To ensure the highest possible accuracy in the plant inversion process, the continuous-time transfer function of the plant, $G_p(s)$, was built utilizing the parameters

estimated via the Least Squares algorithm (as described in Chapter 4) at an operating speed of 1000 rpm:

$$R = 3.153\,\Omega, \quad L = 0.0061\,\text{H}, \quad J = 8.103 \cdot 10^{-5}\,\text{kg m}^2 \tag{5.11}$$

It is worth noting that, given the extremely small value of the viscous friction coefficient ($b \approx 0.0005$), the term was considered negligible ($b \approx 0$) during the synthesis phase to simplify the mathematical model without compromising the overall validity of the control law.

The plant was then discretized using a sampling time $T_s = 0.1$ ms. For the desired dynamics, a delay $\theta = 0.0001$ s was imposed. Since $\theta = T_s$, the discrete delay parameter results in $N = \lfloor \theta / T_s \rfloor = 1$.

The main tuning parameter of the Dahlin controller is λ. A proper selection of λ is crucial to find the best trade-off between the speed of the response and the magnitude of the required control action.

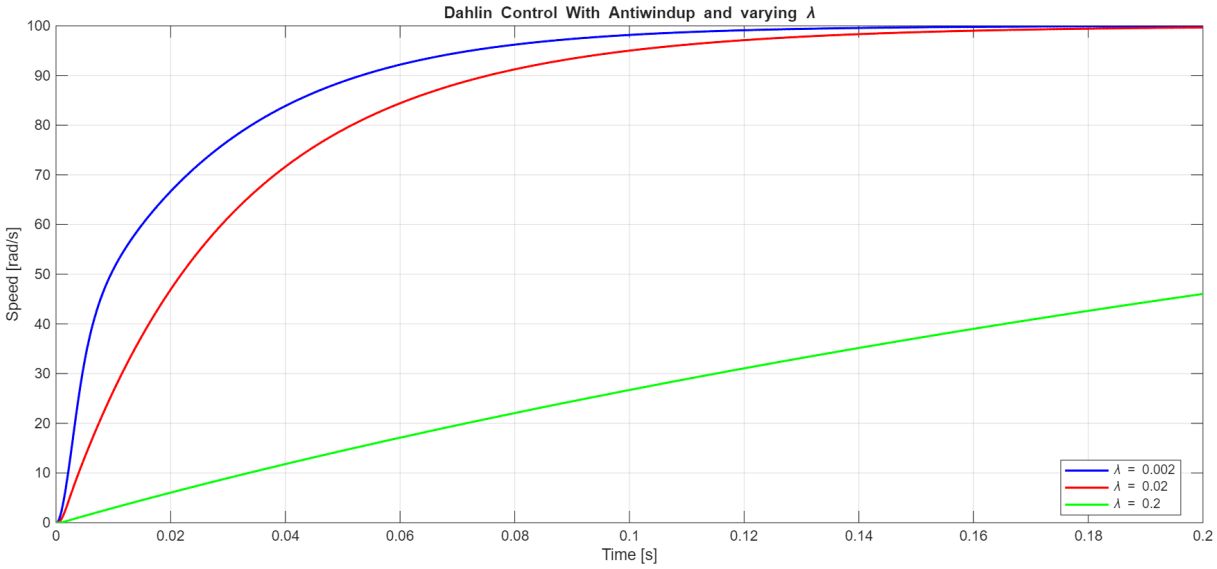


Figure 5.7: Dahlin Control: Effect of varying the tuning parameter λ on the speed response.

As shown in Figure 5.7, decreasing the value of λ makes the system respond significantly faster. However, a very small λ brings the Dahlin controller closer to a pure Deadbeat behaviour, requiring an aggressive control effort that easily exceeds the physical limits of the voltage supply. After several iterations, a value of $\lambda = 0.02$ was chosen, as it provides a fast settling time while maintaining a relatively smooth control variable.

5.3.3 Anti-Windup Implementation and Simulation Results

Despite the smoother response compared to the Deadbeat controller, the Dahlin algorithm still attempts to force the system dynamics to match the reference model. When high speed references are requested, the required voltage heavily saturates at the physical limits of the power supply.

To prevent the integral action of the controller from charging uncontrollably during saturation periods (windup phenomenon), an anti-windup scheme based on the block

diagram synthesis proposed in the literature was implemented. This approach relies on defining a proper polynomial $F(z)$ that satisfies the following constraints:

- $F(z)$ must be an asymptotically stable polynomial.
- $\deg(F(z)) \geq \deg(B(z))$, where $B(z)$ is the numerator of the controller $D(z)$.
- The steady-state gain condition must be verified: $\frac{B(1)}{F(1)} \geq 0$.

Since the Dahlin controller yields a 2nd-degree numerator $B(z)$ exactly like the Deadbeat, the $F(z)$ polynomial must be at least of 2nd degree. Maintaining the same 3rd-degree structure used previously to optimally shape the discharge dynamics, the chosen polynomial was:

$$F(z) = K_f \cdot z(z - 0.95)(z - 0.8) \tag{5.12}$$

with $K_f = 100$. This formulation ensures that $F(z)$ has stable poles and a sufficiently high steady-state gain to rapidly discharge the accumulated error when the actuator exits the saturation region.

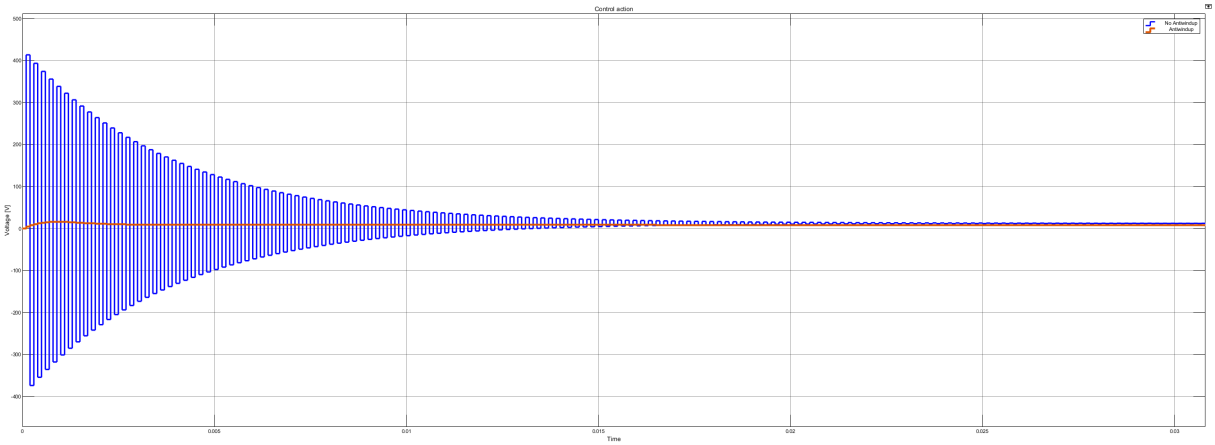


Figure 5.8: Control action (voltage) requested by the Dahlin controller with the implemented Anti-windup scheme.

The effect of the anti-windup implementation is visible in the control action plotted in Figure 5.8, where the controller manages the saturation boundaries efficiently without causing severe instability or ringing when returning to the linear operating region.

To better appreciate the effectiveness of the proposed anti-windup polynomial $F(z)$, a detailed view of the control action (voltage) generated by the Dahlin controller with the anti-windup scheme enabled is presented in Figure 5.9.

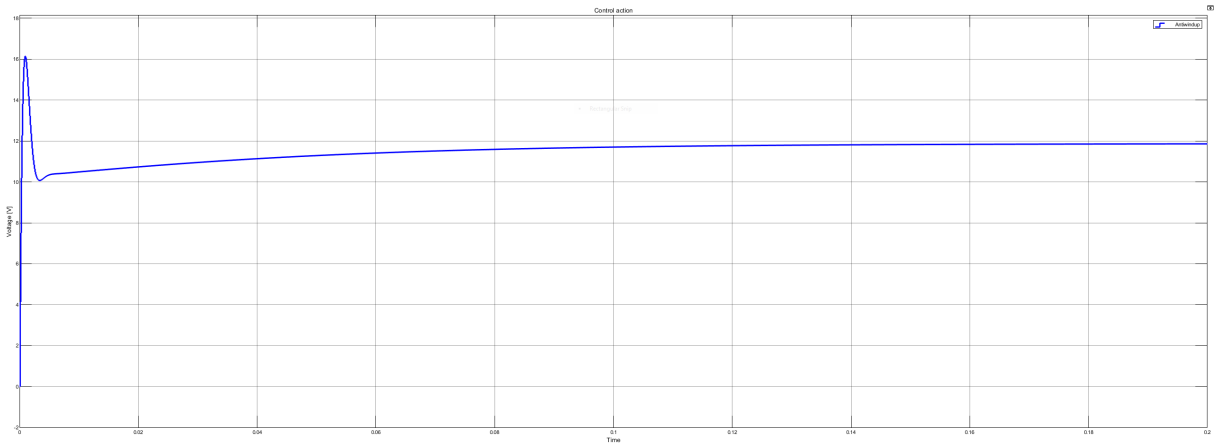


Figure 5.9: Detailed view of the control action (voltage) for the Dahlin controller with the anti-windup scheme enabled.

As can be observed in the detailed plot, the control effort is extremely smooth and well-behaved. During the initial transient phase, the voltage exhibits a sharp but contained peak (around 16 V) to rapidly accelerate the motor dynamics. Immediately after, it smoothly settles to the steady-state value required to maintain the target speed (which compensates for the back-electromotive force and viscous friction).

Notably, the aggressive "ringing" effect—which is typically associated with discrete-time analytical controllers featuring negative real poles, as discussed in the theoretical lectures—is completely suppressed. This confirms that the choice of $F(z)$ not only efficiently manages actuator saturation, but also drastically improves the overall quality of the control signal fed to the real motor, avoiding dangerous high-frequency stress on the electrical components.

5.3.4 Experimental Results on the DC Motor

Once the controller synthesis and the anti-windup scheme were validated, the algorithm was deployed on the microcontroller to drive the real DC motor. The tests were performed by applying a square wave speed reference (from 0 to 1000 rpm) to evaluate both the transient and steady-state behaviors. To thoroughly test the robustness of the control, the experiments were conducted under two different mechanical conditions: unloaded motor (WITHOUT Load) and motor subjected to an active resisting torque obtained by connecting a braking resistor to the coupled generator (WITH Load).

Figure 5.10 illustrates the complete sequence of steps, comparing the Dahlin controller equipped with the anti-windup scheme (blue line) against the standard formulation without anti-windup (red line).

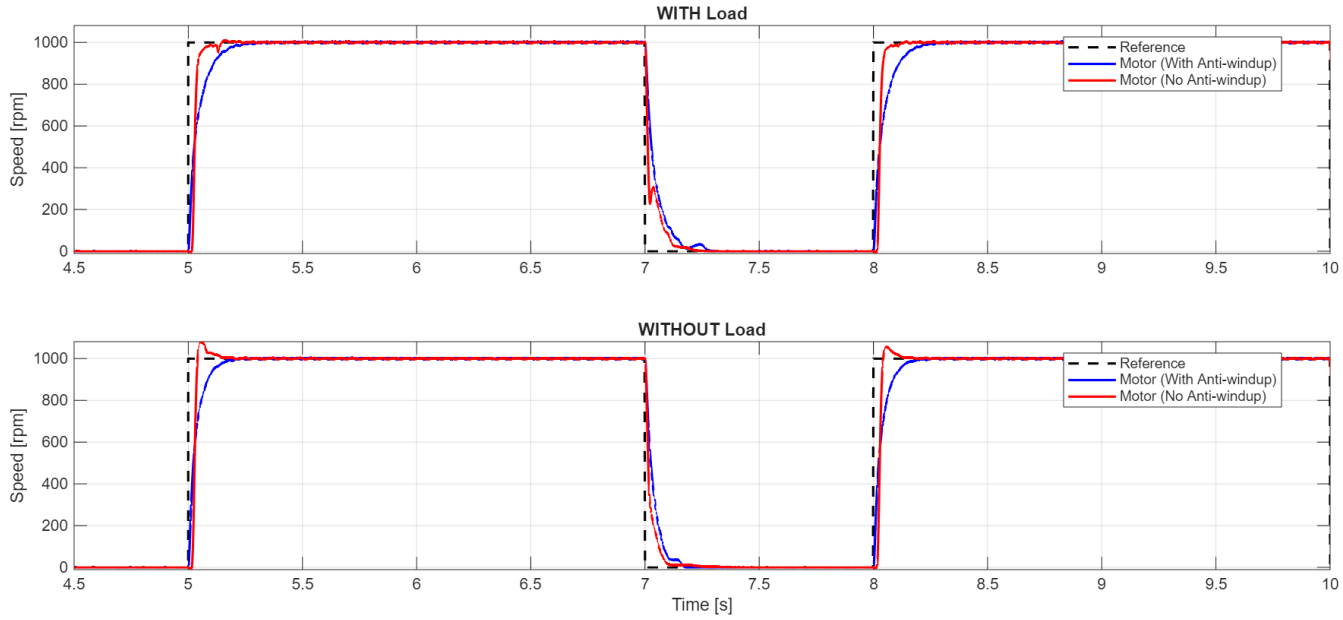


Figure 5.10: Experimental speed response to a sequence of steps ($0-1000$ rpm). Comparison between the implementation with (blue) and without (red) anti-windup, both loaded and unloaded.

To better highlight the critical differences between the two implementations, detailed zoomed-in views of the rising and falling edges were extracted.

Rising Edge and Settling Time Analysis

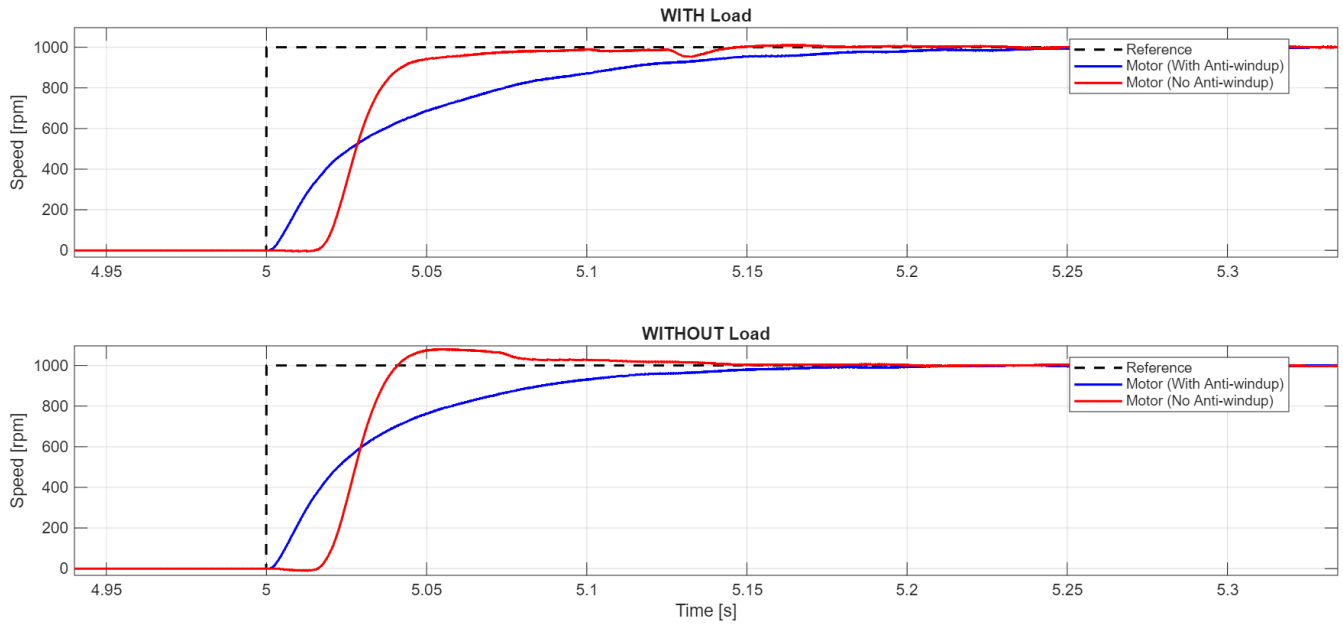


Figure 5.11: Detailed view of the rising edge. The absence of anti-windup (red) leads to visible overshoot, especially in the unloaded condition.

As shown in Figure 5.11, during the large initial step, the absence of the anti-windup scheme (red curve) allows the integral action to charge excessively. This forces the actuator into deep saturation, making the system rise slightly faster but causing an inevitable

overshoot, which is particularly evident in the "Without Load" scenario. On the other hand, the blue curve (with anti-windup) maintains a much smoother and strictly controlled trajectory, completely avoiding the overshoot.

To quantitatively evaluate the closed-loop time constant achieved on the real setup, cursor measurements were taken during the laboratory sessions under the same operational conditions. Figures 5.12 and 5.13 show the comparison between the uncompensated system and the one with the active anti-windup scheme.

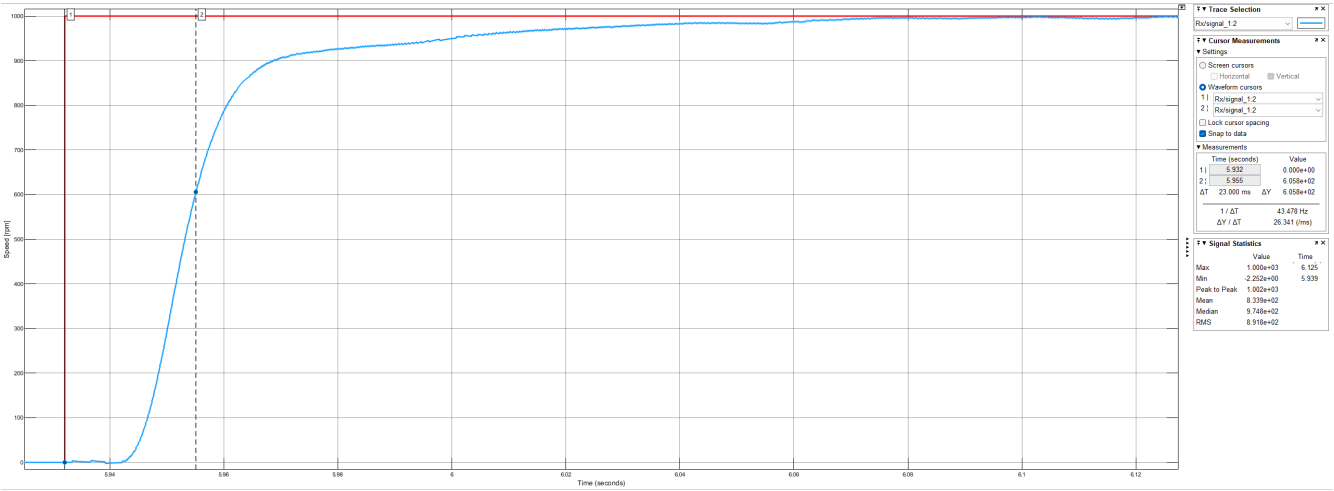


Figure 5.12: Cursor measurement of the closed-loop time constant (approx. 63% of the steady-state value). Implementation WITHOUT anti-windup: $\Delta T \approx 23$ ms.

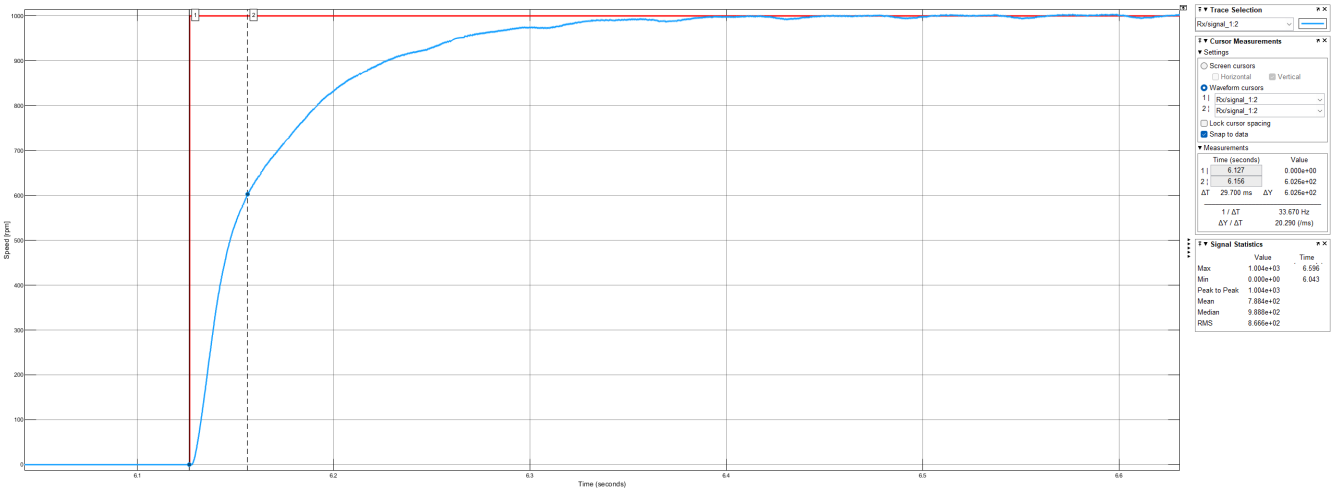


Figure 5.13: Cursor measurement of the closed-loop time constant. Implementation WITH anti-windup: $\Delta T \approx 29.7$ ms.

The measurements focus on the time required for the motor speed to reach approximately 63% of its final value, representing the time constant of the equivalent first-order system. The theoretical design parameter was set to $\lambda = 0.02$ s (20 ms).

As shown in Figure 5.12, the implementation without anti-windup reaches the threshold in $\Delta T \approx 23$ ms, which is relatively close to the theoretical expectation of 20 ms. In this uncompensated scenario, the controller ignores the physical limits and requests a massive voltage, keeping the actuator fully saturated at its maximum capability. This pseudo

"bang-bang" behavior forces a very fast initial acceleration, which perfectly explains the shorter rise time.

Conversely, Figure 5.13 shows that the implementation with the anti-windup scheme reaches the 63% threshold in $\Delta T \approx 29.7$ ms. This slight increase in the rise time is a direct consequence of the anti-windup action. When saturation occurs, the anti-windup feedback mechanism modifies the internal dynamics of the controller to keep the states bounded. This necessary intervention inherently alters the transient response of the system compared to the purely linear, unconstrained mathematical model.

Ultimately, the observed delay of roughly ~ 7 ms is an expected and acceptable trade-off: the system sacrifices a small amount of initial speed to ensure bounded internal states, overall stability, and a safe recovery from the saturation regime.

Falling Edge and Windup Discharge

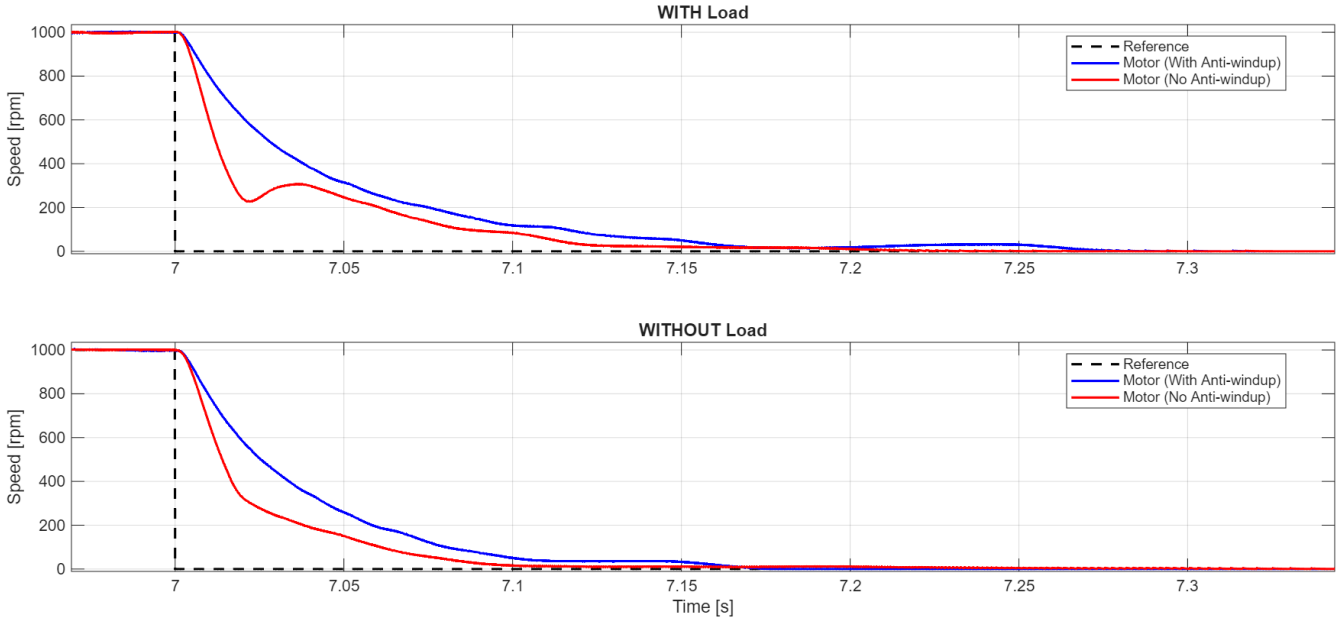


Figure 5.14: Detailed view of the falling edge. The integral windup discharge causes a severe "bump" in the uncompensated system (red).

The most striking evidence of the windup phenomenon occurs during the falling edge (Figure 5.14), when the reference abruptly drops from 1000 rpm to 0. In the standard implementation (red curve), the integrator holds a massive accumulated positive value from the steady-state phase. When the error becomes negative, the integrator takes a considerable amount of time to discharge, causing a severe delayed "bump" (re-acceleration) before the motor finally settles to zero.

Conversely, the implementation with the properly tuned $F(z)$ polynomial (blue curve) limits the integral state during the saturated regime. As a result, as soon as the reference drops, the control action immediately reacts, leading the motor to a smooth, monotonic, and rapid deceleration, proving the absolute necessity of the anti-windup network for real-world applications.

Chapter 6

Final Comparison Between Controllers

After detailing the design, implementation, and individual tuning of the different control architectures, this chapter provides a direct experimental comparison of their performances. The controllers evaluated in this final stage represent the best versions developed during the laboratory sessions:

- **Cascade PI Control**: Tuned utilizing the offline estimated mechanical and electrical parameters (as discussed in Chapter 4) and equipped with the software anti-windup scheme.
- **Deadbeat Controller**: Synthesized in the discrete-time domain and supported by the $F(z)$ anti-windup polynomial to manage actuator saturation.
- **Dahlin Controller**: Designed for a smoother exponential response ($\lambda = 0.02$) and similarly protected by the discrete anti-windup scheme.

6.1 Comparison Criteria

To comprehensively evaluate and rank the controllers, a qualitative and graphical analysis was conducted based on the system's response to a standard step sequence ($0 \rightarrow 1000 \rightarrow 0$ rpm). The comparison relies on the following fundamental criteria:

- **Transient Speed and Responsiveness**: Evaluated primarily through the rise time. It measures how fast the controller drives the motor to reach the desired speed reference.
- **Stability and Overshoot**: Assesses the controller's ability to smoothly transition into the steady-state phase without exceeding the reference or oscillating, both during the rising edge and the falling deceleration (undershoot).
- **Control Effort and Saturation Management**: Evaluates how much the controller "stresses" the physical hardware. Highly aggressive algorithms (like the Deadbeat) request extreme voltage peaks, forcing the power supply into prolonged saturation and relying heavily on the anti-windup mechanism to not lose stability.

- **Robustness to Load Variations**: Evaluated by comparing the unloaded motor's performance against the loaded scenario. The load is introduced by connecting the braking resistors to the coupled brushless generator, which imposes an active resisting torque on the motor shaft.

6.2 Performance in Unloaded Condition

The first comparative test was conducted with the motor in an unloaded state (decoupled from the braking resistors). The system was subjected to a step reference from 0 to 1000 rpm, and then back to zero. Figure 6.1 illustrates the overall superimposed responses of the three control architectures.

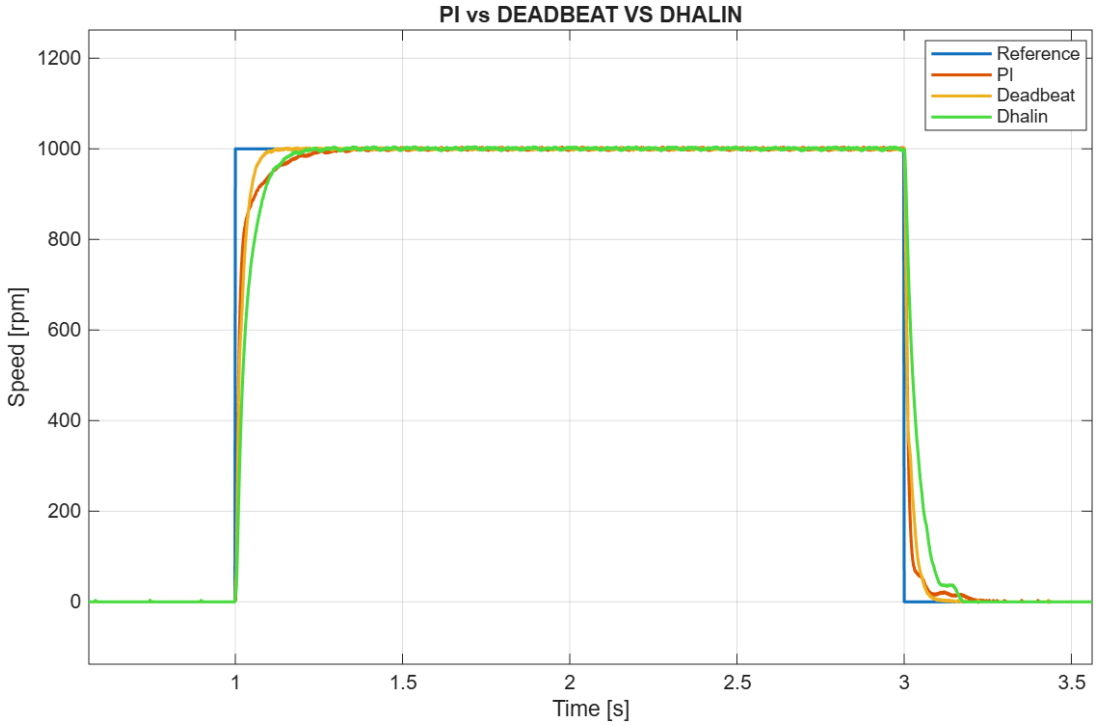


Figure 6.1: Experimental comparison of PI, Deadbeat, and Dahlin controllers without load.

Detailed zoomed-in views of the transient phases (Figures 6.2 and 6.3) have been extracted to better appreciate the dynamic differences. By thoroughly analyzing these transient behaviors, the distinct characteristics of each synthesis method clearly emerge:

- **Deadbeat (Yellow Line)**: The Deadbeat controller provides the absolute best performance in this scenario. True to its theoretical formulation, it guarantees the fastest rise time. Remarkably, thanks to the highly effective design of the $F(z)$ anti-windup polynomial, it completely suppresses the expected ringing. As clearly visible in the detailed transient plots, it tracks the reference with **no appreciable overshoot** and smoothly returns to zero with **zero undershoot**.
- **Cascade PI (Orange Line)**: The Cascade PI exhibits a very fast initial response. However, looking closely at the zoomed rising edge, it is the slowest to fully reach and settle at the exact steady-state value. This "long tail" effect demonstrates that

while the offline parameter estimation provides a stable system, the integral action takes considerably more time to completely eliminate the residual error compared to the analytical discrete controllers. The falling edge confirms this behavior, showing a minor delay near the zero mark before discharging completely.

- **Dahlin (Green Line)**: The Dahlin response reflects its design specification ($\lambda = 0.02$), rising more gently than the Deadbeat. However, the detailed view of the falling edge highlights a peculiar behavior: instead of a smooth exponential decay, the green curve shows a visible deceleration plateau (bump) before finally reaching zero speed. This hesitation is likely caused by the controller's less aggressive nature struggling to smoothly overcome the unmodeled non-linearities of the real motor (such as static friction) at very low speeds.

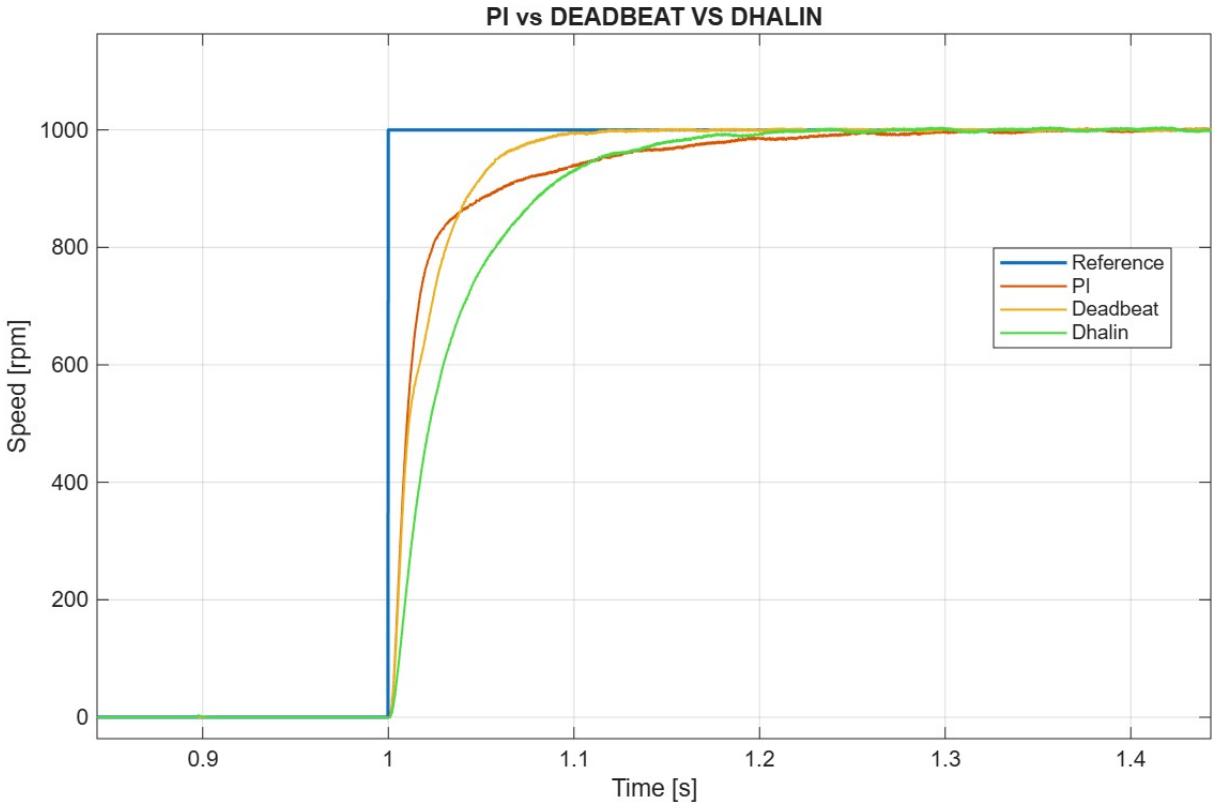


Figure 6.2: Detailed view of the rising edge (Unloaded condition).

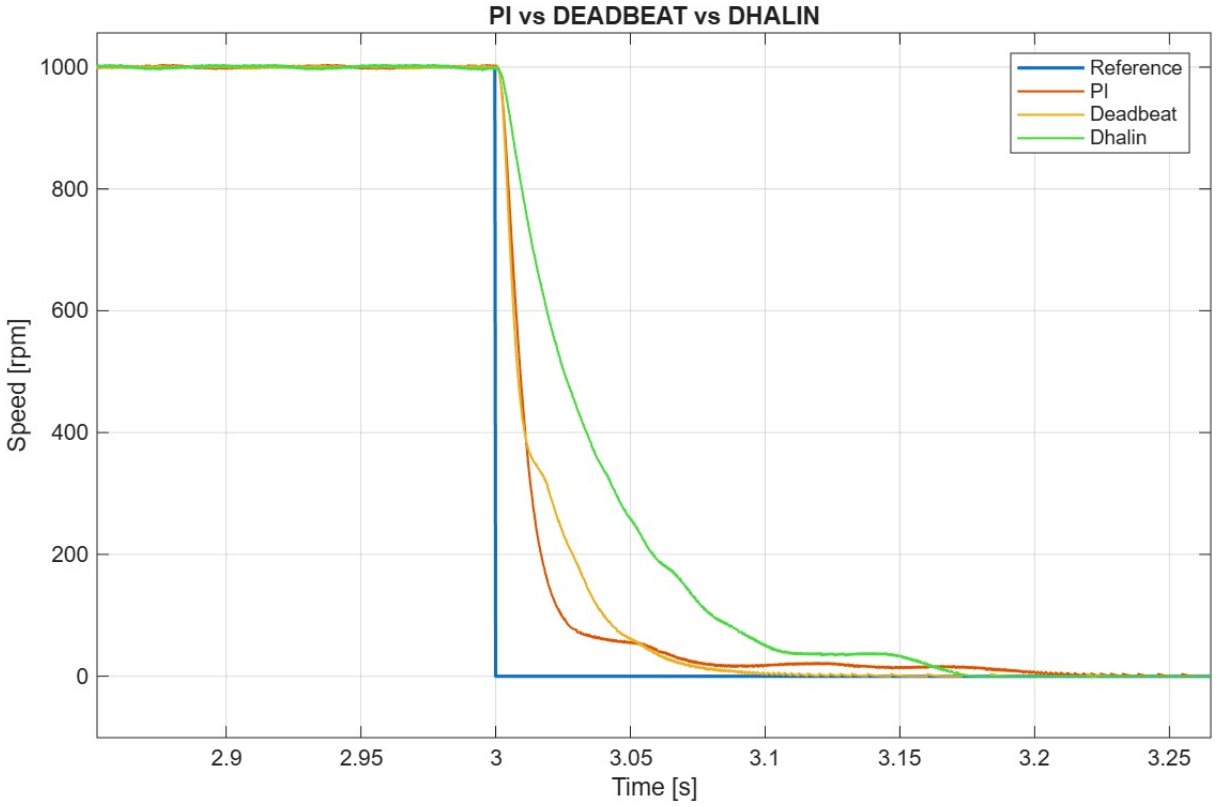


Figure 6.3: Detailed view of the falling edge (Unloaded condition).

Steady-State Considerations A highly magnified view of the steady-state phase (Figure 6.4) reveals a highly oscillatory, overlapping behavior for all three controllers.

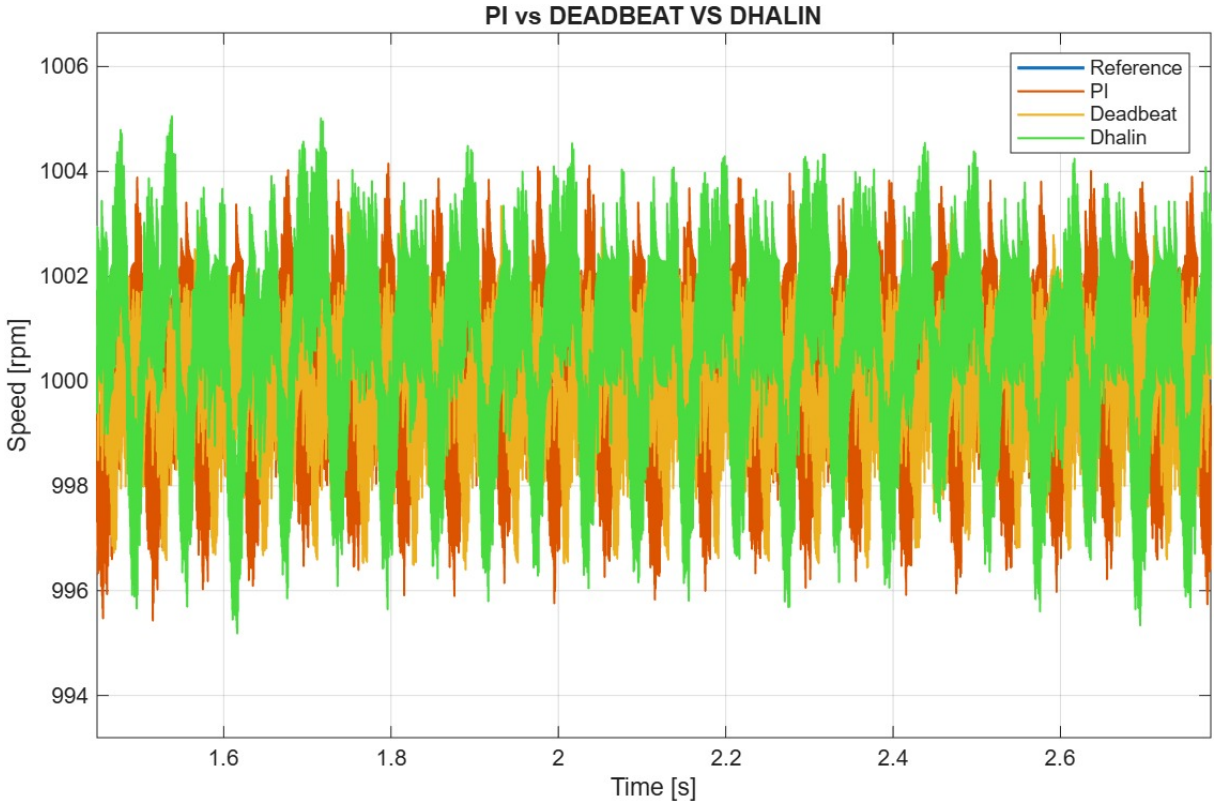


Figure 6.4: Detailed view of the steady-state tracking performance at 1000 rpm.

While it may visually appear noisy, this behavior is physically expected and actually represents a highly satisfactory result. The high-frequency micro-oscillations are bounded within a narrow band of approximately ± 4 rpm. This phenomenon is entirely attributable to the encoder's spatial quantization, the discrete nature of the digital speed filter, and the PWM switching ripple. Given the 1000 rpm operating point, this corresponds to a relative error of less than 0.5%, confirming the excellent steady-state tracking precision of the entire control architecture.

6.3 Performance in Loaded Condition

To evaluate the robustness of the designed controllers, a second set of experiments was conducted by coupling the DC motor to the brushless generator connected to the braking resistors. This setup introduces an active resisting torque proportional to the speed, significantly altering the mechanical dynamics of the plant. The superimposed responses to the same $0 \rightarrow 1000 \rightarrow 0$ rpm step reference are shown in Figure 6.5.

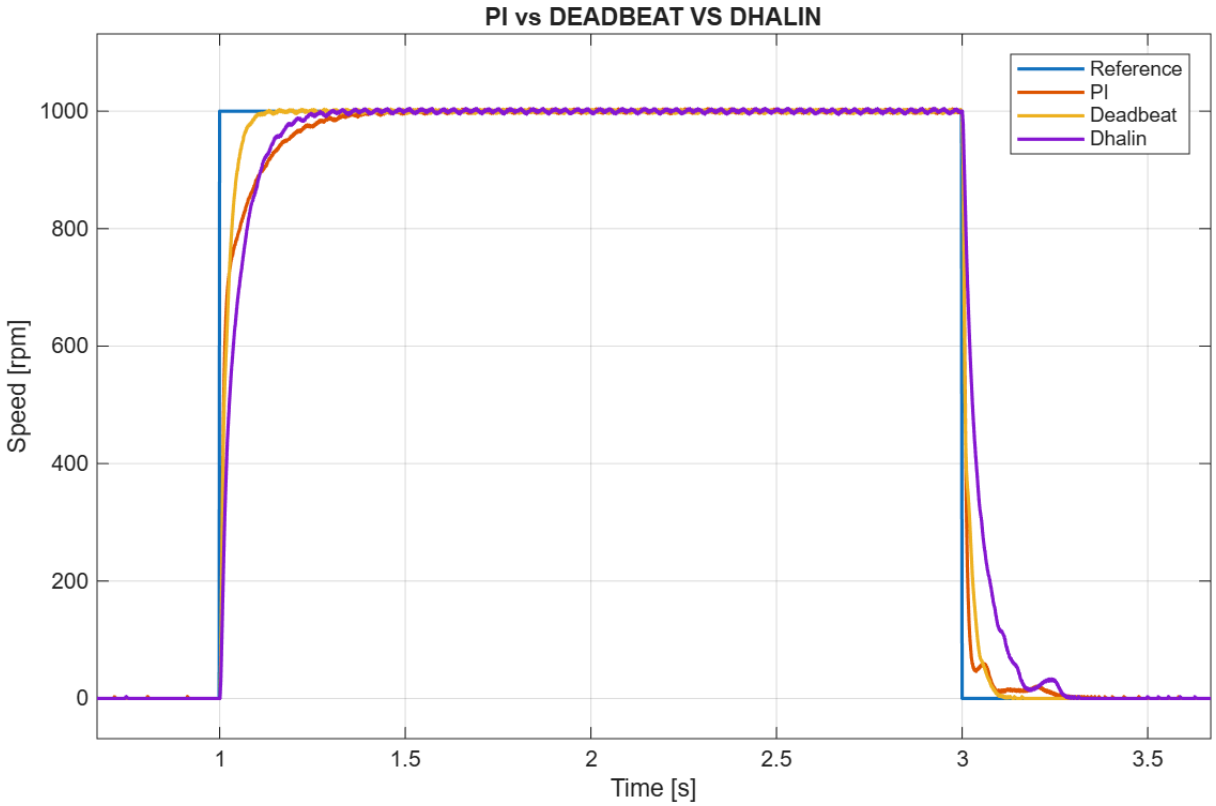


Figure 6.5: Experimental comparison of PI, Deadbeat, and Dahlin controllers with active load.

The introduction of the load drastically amplifies the differences between the control strategies, highlighting their varying degrees of robustness:

- **Deadbeat (Yellow Line)**: The Deadbeat controller demonstrates exceptional robustness, remaining virtually unaffected by the active load. It proudly maintains its superiority, delivering the fastest rise time with **zero overshoot**. Furthermore, its falling edge remains razor-sharp, smoothly and immediately dropping to zero without any undershoot or hesitation. The aggressive nature of the algorithm, perfectly tamed by the $F(z)$ anti-windup polynomial, easily overpowers the resisting torque.

- **Cascade PI (Orange Line)**: The Cascade PI clearly struggles more in this scenario. The "long tail" effect observed in the unloaded test is severely amplified here. The initial acceleration is fast, but the integral action takes a substantial amount of time to overcome the opposing load torque and eliminate the steady-state error. Additionally, a minor delayed bump is visible during the falling edge before the motor completely stops.

- **Dahlin (Purple Line)**: The Dahlin controller is the most negatively impacted by the load. While its rising edge maintains a relatively smooth trajectory, its performance completely degrades during the falling edge. The slight hesitation observed in the unloaded case morphs into a **massive deceleration plateau (bump)**. The less aggressive control effort struggles to properly discharge and manage the unmodeled dynamics introduced by the loaded generator, resulting in a severe delay before the speed finally settles to zero.

6.4 Final Discussion

The comparative analysis clearly highlights the specific strengths and limitations of each control strategy, demonstrating how theoretical designs perform under real-world physical constraints.

The **Deadbeat controller** emerges as the overall best performer. Although theoretically prone to severe ringing and actuator stress, the implementation of the $F(z)$ anti-windup polynomial completely resolves these vulnerabilities. It provides the fastest tracking, zero overshoot, and outstanding robustness against active load variations, proving to be the superior choice when high dynamic performance is required.

The **Cascade PI** stands out as a highly reliable and balanced solution. Thanks to the offline parameter estimation, it achieves stable tracking without stressing the power supply as aggressively as the Deadbeat. However, its integral action is visibly slower in rejecting load disturbances, leading to longer settling times (the "long tail" effect). It remains the best compromise between performance and hardware preservation.

Finally, the **Dahlin controller** is suitable only when a smooth, conservative control effort is strictly prioritized over speed. While it performs adequately in unloaded conditions, its lack of aggressiveness makes it highly susceptible to unmodeled dynamics and resisting torque, resulting in poor deceleration transients and severe delays.

Ultimately, the Deadbeat combined with a properly tuned anti-windup network proved to be the most effective architecture for this DC motor setup, successfully bridging the gap between theoretical speed and practical robustness.

Chapter 7

Conclusions

The main objective of this project was the development and practical implementation of different digital control strategies for a DC motor. This work successfully connected theoretical mathematical models with practical application, managing real physical constraints such as sensor noise and actuator voltage limits.

An important step for the proper operation of the system was the offline estimation of electrical and mechanical parameters using the Least Squares algorithm. This procedure led to a model that is much more faithful to reality than the manufacturer's datasheet, allowing the identification and management of an asymmetric and non-linear viscous friction. Parallelly, the implementation of anti-windup algorithms proved to be an absolute necessity for all developed controllers, maintaining system stability and preventing accumulated errors during the saturation phases of the power supply.

As discussed in the previous comparison, moving from a standard control approach (Cascade PI) to advanced discrete-time techniques (Deadbeat and Dahlin) requires a careful balance between the desired response speed and the effort imposed on the hardware.

In summary, combining precise physical model identification with the software management of electrical limits guaranteed the success of the entire project. The Deadbeat controller, paired with its specific anti-windup polynomial, proved to be the most suitable choice for this motor, ensuring fast reaction times and excellent practical robustness.